opentext™

iClub
A Portfolio India Initiative

Volume 6

TECH JOURNAL 2026

THINK . INNOVATE . WRITE



Green Code Effect:
Efficient Architecture for
GenAI

ENTERPRISE AI
THE RISE OF
RESULTS
-AS-A-SERVICE

Magic Memory

eBPF
From Basics to
Production

Migrating from Apache Tiles to Thymeleaf with Spring 6

Biocomputing
The Living Future of Information Processing

Demystifying Vertex AI Vector Search:
The Secret Sauce for LLMs

Beyond the Basics
XSS Prevention strategies for custom frameworks

Spec-Driven Development
When AI Meets Discipline

Technology AI Security Machine Learning Agentic Development Processes Protocols



Editorial Board

Anju Dagilya
Kamal Maheshwari
Lokesh Murthy Venkatesh
Praveen Kumar Sahukar
Soumya K
Srijith Kochunni
Vamsi Krishna
Vidhya K
Yogita Naik

Contributing Authors

Abani Ranjan Samal
Abhinandan Narayan
Akhil Rampally
Aruna KS
Atish Mohanty
Bharatesh
Chandrashekar Ramachandran
Chandru Karunanithi
Deeksha R
Deepika Singh
Dharun Kumar
Dinesh Kumar Sahoo
Ganapathi CH
Gaurav Kumar
Gopala Krishna
Haretha Varshini S
Harinath Jarugula
John Divakar
Joswitha Palipe
Kamal Maheshwari
Kratagya Jadoun Singh
Krishna Kumar Parthasarathy
Kumar Chandrashekar
Lida Joy
Lokesh Murthy Venkatesh
Nancey
Nithin Kumar Chidamabara
Noor Mohammed Rizwan
Prabhu Vinod Tikkiseti
Preetham Wagle
Rahul R
Rajasekhar Danturti
Santhosh Kumar J
Sethu Ramalingam
Shivani Srivastava
Sri Darshan Sreedharan
Srijith Kochunni
Vamshi Krishna Balusu
Vamsi Krishna

Cover Design

Shiva Shanker Musini

Book Composition

Arul Kumar Kannaiyan



iClub Tech Journal

April 2026

AI Analogous

Biocomputing: The Living Future of Information Processing 5
Kumar Chandrashekar

Demystifying Vertex AI Vector Search: The Secret Sauce for LLMs 7
Harinath Jarugula

The Intelligent Kernel: How AI is Changing Power Management and Scheduling in Linux 9
Gaurav Kumar

Vector Databases in Modern Engineering Systems 11
Chandrashekar Ramachandran

Vector Databases - The Hidden engine behind enterprise AI 13
Lida Joy

AI In Development

Automating Software Installation Faster with AI-Assisted Development 15
Sethu Ramalingam

Debugging a Critical NFS Performance Issue Using AI-Assisted Kernel Tracing 17
Vamshi Krishna Balusu

From Zero to Hero: Building a Production-Ready Patch Management UI with GitHub Copilot 19
Rajasekhar Danturti

The Dependency Context Gap - Leveraging AI to solve invisible breaking changes 22
Preetham Wagle

Why-Every-Software-Product-Needs-Its-Own-AI-Model 24
Gopala Krishna

Why we dread the notification word “Failed”? 27
Nithin Kumar Chidamabara

AI In Software

AI Models That Sleep: How Artificial Rest Cycles Improve Model Stability 29
Nancey

Applied Accelerated AI: A Systems Approach to Real-World Performance 31
Aruna KS

Creating a club of digital thinkers 33
Srijith Kochunni

Engineering an AI-Augmented Diagnostic Engine for Enterprise Log Analysis 35
Lokesh Murthy Venkatesh

From Software Tools to Business Outcomes - The rise of Results as a service in Enterprise AI 37
Kamal Maheshwari

The Edge Advantage: Why Small Language Models (SLMs) Are the Giant Leap for Enterprise AI 40
Santhosh Kumar J

The Great Compute Repatriation: Why Your Old Laptop Is the Future of Your Smart Home? 41
Sri Darshan Sreedharan

The Unified Backend 2026: High-Performance AI with Java 25 and PostgreSQL 18 43
Prabhu Vinod Tikkiseti

Introduction to Agentic AI and building your own agent 46
Joswitha Palipe

AI Processes

Agile in the Age of AI: Redefining Scrum for Intelligent Development 48
John Divakar

GitHub Copilot in the Agile Sprint Cycle: A Framework for Maximizing Team Velocity and Quality 51
Lokesh Murthy Venkatesh

Spec-Driven Development: When AI Meets Discipline 53
Vamsi Krishna

The Green Code Effect: Architecting for Efficiency in the GenAI Era 56
Chandru Karunanithi

The Silent Technical Debt introduced by AI-Assisted Development 58
Akhil Rampally

Cloud Computing

Engineering Resilience in Cloud-Native Systems with OpenShift 60
Haretha Varshini S

Development

Apache TLS Upgrade Deprecation Impact 62
Shivani Srivastava

Beyond the Basics : XSS Prevention strategies for custom frameworks 64
Noor Mohammed Rizwan

Establishing Trust Anchors: Formal Verification of Multi-Tiered Certificate Chains 66
Rahul R

Importance of a Reliability Setup for Product Releases 68
Abhinandan Narayan

Observability-First Design: The Modern Successor to TDD for Microservices 70
Krishna Kumar Parthasarathy

Migrating from Apache Tiles to Thymeleaf with Spring 6 73
Dinesh Kumar Sahoo

Scaling File Uploads: A Merkle Tree–Based Approach to Resilient Multi-Gigabyte Transfers 76
Kratagya Jadoun Singh

Protocols Standards

eBPF - From Basics to Production 79
Deepika Singh

Magic Memory 82
Ganapathi CH

Mercurial Cores 84
Ganapathi CH

Realtime communication with WebRTC 86
Atish Mohanty

The Container is Dead: Why WASI is the “Universal USB” of Cloud Computing 88
Gaurav Kumar

Technology

An Analytical Review of Ansible + Semaphore UI as a Lightweight Control Plane for Non-Functional Testing 90
Bharatesh

Migrating Existing Xamarin.Forms App to .NET MAUI 92
Abani Ranjan Samal and Dharun Kumar

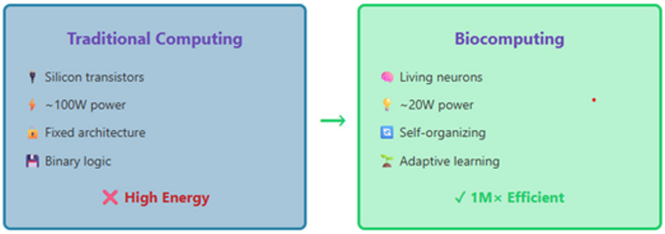
Reimagining Patch Management Through Dependency-Aware Risk Visibilit 95
Kratagya Jadoun Singh

Surviving the Grey Zone 97
Deeksha R

The Architecture of Entropy A Story of systems, failure and survival 99
Sri Darshan Sreedharan

Biocomputing: The Living Future of Information Processing

Kumar Chandrashekar, Lead Software Engineer - Connected (CMX)



Introduction

As we navigate through 2025, traditional silicon-based computing approaches physical and energy efficiency limits. Data centers consume 1-2% of global electricity, with projections suggesting exponential growth as AI workloads intensify. Biocomputing—a revolutionary paradigm harnessing living neurons as computational units—offers a solution.

Unlike neuromorphic computing that mimics biological systems using silicon, biocomputing directly utilizes neurons, DNA, and proteins as the computing substrate. This distinction is crucial: we're not building computers inspired by biology; we're making biology compute.

The Problem Being Addressed

Energy Crisis in Computing

Training a single large language model consumes as much energy as 100 American homes use annually. GPT-3 required approximately 1,287 MWh—equivalent to 552 tons of CO_2 emissions.

Key Statistics:

- Traditional computers: ~100 watts for desktop systems
- Human brain: ~20 watts for 86 billion neurons with massively parallel computations
- Energy efficiency gap: The brain is approximately 1 million times more energy-efficient per operation

Architectural and Physical Limits

Modern computing faces two critical barriers:

- The Von Neumann bottleneck (where data constantly shuttles between separated memory and processing units)
- Physical scalability limits as transistors approach atomic scales (3nm nodes).

  Biological neural networks solve both—processing and memory are intrinsically linked at synapses, and physical laws don't constrain biological growth as they do silicon miniaturization.

Findings and Current State of Research

Breakthrough Developments (2023-2025)

Research institutions have developed systems using neural organoids—3D clusters of human neurons derived from stem cells—as computational units.

Key Achievements:

- Longevity: Neurons maintained functional for 100+ days
- Scale: Arrays with up to 1,000,000 neurons
- Efficiency: >1000x improvement over silicon
- Learning: Successfully trained for pattern recognition and logic operations

The Solution: Biocomputing Architecture

Core Components

1. Biological Processing Units (BPUs): Neurons process information through spike timing, synaptic plasticity, and neurotransmitter chemistry—far more complex than binary transistors.

Parameter	Silicon Computing	Biocomputing (Current)	Theoretical Biocomputing
Energy Efficiency	1x (baseline)	100-1000x better	1,000,000x better
Parallel Processing	Limited (thousands)	Massive (millions)	Extreme (billions)
Learning Speed	Fixed architecture	Adaptive (real-time)	Self-optimizing
Lifespan	5-10 years	100+ days (improving)	Years (goal)
Setup Time	Minutes	Days-Weeks	Hours (goal)

2. Microelectrode Arrays (MEAs): High-density electrode arrays (10,000+ channels) bidirectionally interface with organoids, stimulating and recording neural activity at microsecond resolution.
3. Bioreactor Systems: Sophisticated life-support maintaining temperature (37°C), pH (7.35-7.45), nutrient perfusion, and waste removal to keep neurons alive.

Training Biological Circuits

Unlike explicit programming, biocomputing uses reinforcement learning protocols. Stimulation patterns are applied, responses monitored, and desired patterns rewarded with dopaminergic signals.

Neural networks self-organize through Hebbian learning—"neurons that fire together, wire together." Training requires 2-5 days for simple logic operations versus milliseconds for silicon, but biological systems are training themselves through growth and reorganization.

Biocomputing's Impact on AI and Neural Networks

Synergistic Relationship

The relationship is bidirectional: AI optimizes stimulation protocols and manages bioreactor parameters, while biocomputing provides energy-efficient inference, novel computing paradigms (temporal coding, spike-timing plasticity), and inspiration for artificial network architectures.

Real-World AI Applications

- Edge AI Devices: Miniaturized biocomputing chips consuming microwatts could extend battery life from hours to weeks while running sophisticated AI models.
- Adaptive Control Systems: Inherent plasticity suits continuous adaptation in autonomous vehicles, smart grids, or prosthetics that integrate with neural patterns.
- AI Training Acceleration: Mature biocomputing clusters could train models with dramatically reduced energy costs, democratizing AI development.

Challenges and Limitations

Ethical Considerations: Organoid consciousness (current consensus: no—insufficient complexity), sourcing human-derived cells, and evolving regulatory frameworks.

Technical Hurdles: Biological variability hinders reproducibility, scaling to billions of neurons remains difficult, no standardization exists, and debugging living neurons is far more complex than silicon.

Where Traditional Computing Remains Superior

- Deterministic Tasks: Silicon excels at precise, repeatable calculations—database queries, cryptography, numerical simulations—with unmatched speed and reliability.
- Memory & Storage: Conventional systems store terabytes with perfect fidelity and transmit data at gigabit speeds. Biological memory is reconstructive and unsuitable for verbatim recall or instant network transfer.
- Network Communication: Digital systems seamlessly transfer datasets globally with error-correction; biological systems lack this capability.

Timeline Reality Check: Current biocomputing parallels 1950s transistor computers. Commercial viability is likely 10-15 years away.

Conclusion

Biocomputing represents a paradigm shift as profound as the transition from vacuum tubes to transistors. By harnessing evolution's 3.5-billion-year R&D investment in biological neural networks, we're poised to overcome silicon computing's fundamental limitations.

The statistics are compelling: 1,000,000x better energy efficiency, massive parallel processing, and self-organizing architectures. Research institutions have demonstrated working proof-of-concept systems today.

For AI, biocomputing offers sustainable scaling. As models grow exponentially—from GPT-3's 175 billion parameters to future trillions—energy demands threaten to make advanced AI economically and environmentally untenable. Biocomputing could enable AI's trajectory without depleting planetary resources.

Challenges remain: longevity, standardization, scalability, and ethical considerations. However, biocomputing complements rather than replaces traditional computing—excelling in adaptive learning, parallel processing, and efficiency while silicon dominates precision, memory, and deterministic operations.

The future of computing isn't choosing silicon or biology—it's leveraging both paradigms for their unique strengths.

References and Further Reading

- Kagan et al. "In vitro neurons learn and exhibit sentience when embodied in a simulated game-world" - Neuron (2022)
- Cai et al. "Organoid Intelligence: Biocomputing with Human Brain Organoids" - Frontiers in Science (2023)
- Patterson et al. "Carbon Emissions and Large Neural Network Training" - arXiv (2021)
- Marković et al. "Physics for Neuromorphic Computing" - Nature Reviews Physics (2020)
- Various research papers on biocomputing and neural organoids - Nature, Science, Frontiers (2023-2025)

Technical Glossary

- Organoid: 3D cell culture that mimics organ structure/function.
- iPSC: Induced Pluripotent Stem Cells - adult cells reprogrammed to embryonic-like state.
- MEA: Microelectrode Array - device for interfacing with neural tissue.
- Hebbian Learning: Synaptic plasticity principle ("cells that fire together, wire together").
- Von Neumann Architecture: Traditional computer design separating memory and processing.
- Neuromorphic: Hardware designed to mimic neural structure (but using silicon).

Demystifying Vertex AI Vector Search: The Secret Sauce for LLMs

Harinath Jarugula, Principal Software Engineer - Endpoint Management

Introduction

If you've been following the world of Large Language Models (LLMs) like Gemini or GPT-4, you've likely heard the term "Vector Database"— or specifically, Google's Vertex AI Vector Search. To a beginner, these terms can feel like a word salad of data science jargon. However, at its heart, a vector database is simply a way to give an AI a "long-term memory" of your specific data.

Here is a breakdown of how it works, the missing pieces you need to know, and why it's essential for modern AI applications.

The Core Concept: From Words to Math

Computers don't understand the "feeling" of a sentence; they only understand numbers. To bridge this gap, we use two critical processes: Chunking and Embeddings.

Chunking

Imagine you have a 500-page manual. An LLM cannot "read" and remember all 500 pages at once due to memory limits (context window). Chunking is the process of breaking that manual into smaller, bite-sized pieces—such as paragraphs or specific sections—so the AI can handle them efficiently.

Embeddings

Once we have these chunks, we pass them through an embedding model. This model converts the text into a long list of numbers called a vector.

Think of an embedding as a set of coordinates in a massive, multidimensional library. Words with similar meanings (like "dog" and "puppy") will have coordinates very close to each other in this invisible mathematical space.

How Vertex AI Vector Search Works

This is where the magic happens. You have millions of these vectors. You can't just store them in a standard spreadsheet; searching them would be too slow.

Vertex AI Vector Search is a high-performance database designed to store these "coordinate" vectors in a structure called an index.

The "Secret Sauce": ScaNN

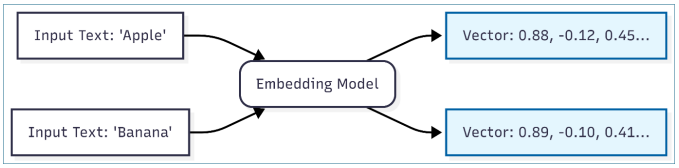


Figure 1 The Embedding Process

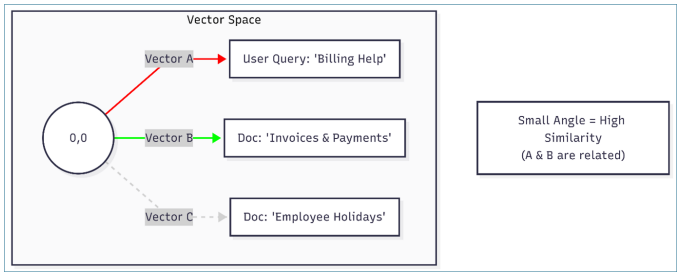


Figure 2 Visualizing Cosine Similarity

Why use Vertex AI instead of a basic open-source tool? Speed. Vertex uses a Google-proprietary algorithm called ScaNN (Scalable Nearest Neighbors). It compresses the vectors effectively and lets you search through billions of items in mere milliseconds with high accuracy.

Finding the Needle: Cosine Similarity

When a user asks a question, the system converts that question into a vector. The database then needs to find which "chunks" of your data are most relevant. It does this using Cosine Similarity.

Imagine two arrows pointing from the centre of a graph. Cosine similarity measures the angle between those arrows.

- Small Angle: The chunks are semantically similar (for example, "Reset Password" and "Change Login").
- 90° Angle: They have nothing in common (for example, "Reset Password" and "Banana Recipe").

Why LLMs Need It: The Power of RAG

The most popular use for Vertex AI Vector Search is Retrieval-Augmented Generation (RAG).

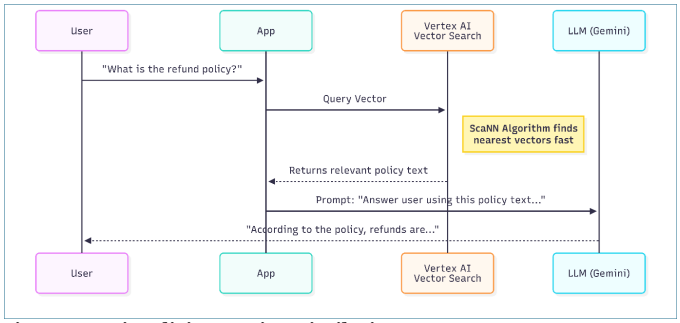


Figure 2 Visualizing Cosine Similarity

LLMs are smart, but they are frozen in time—they only know what they were trained on. If you want a chatbot to answer questions about your company's internal 2026 Q1 policies, the LLM won't know them.

The RAG Workflow

1. Search: The user asks a question.
2. Retrieve: Vertex AI Vector Search finds the most relevant "chunks" from your private data using cosine similarity.
3. Augment: The system hands those chunks to the LLM and says: "Using only this information, answer the user's question."
4. Generate: The LLM provides an accurate, "grounded" answer without hallucinating.

Beyond Just Text: Multimodality and Hybrid Search

While we often talk about text, Vertex AI Vector Search is multimodal. This means vectors can represent different kinds of data:

- Images: Searching for a specific shoe style by uploading a photo.
- Video: Searching within video frames as if they were text.

Pro Tip: Hybrid Search

Sometimes, vector search alone isn't enough. If a user searches for a specific product SKU like "XJ-900," a vector search might return "XJ-800" because they are "mathematically similar."

Vertex AI supports Hybrid Search, which combines the semantic understanding of vectors with the precision of traditional keyword search, giving you the best of both worlds.

Conclusion

Vertex AI Vector Search is the bridge between the raw power of LLMs and the specific, private data of your business. By turning information into coordinates (Embeddings), breaking it down (Chunking), and finding matches (Cosine Similarity), it transforms a general AI into a specialized expert tailored to your own data.

The Intelligent Kernel: How AI is Changing Power Management and Scheduling in Linux

Gaurav Kumar, Sr. Software Engineer - Open Enterprise Server

The Linux kernel has always been the engine of our digital world, expertly handling resources and processes. Imagine if it could think for itself, though. What if it could figure out what was needed, make things better, and save energy, instead of just following rules? AI is about to change how Linux handles power and schedules tasks. Machine learning isn't just another program for Linux anymore; it's becoming a core part of it.

The Need for More Intelligent Kernels

Linux's power management and task scheduling tools have long relied on clever but fixed rules. These guidelines work well for routine tasks, but they fall short when things get hectic and unpredictable. Imagine a server that simultaneously runs multiple virtual machines, an AI language model, and a live data analysis. Every minute, even every second, the best way to distribute CPU power or manage energy for such a mix is different.

AI can help with this. Imagine a system that learns from what's happened before. It then figures out what'll be needed next and changes how it uses resources to either make things more productive or save energy, instead of sticking to strict rules.

Intelligent AI in Practice

Although the term "smart AI" sounds like something from a science fiction film, it actually refers to tiny, unique AI models, or agents, that operate directly within the operating system. These agents monitor temperature, power consumption, CPU usage, and input/output waiting times. They use this information to make wise choices.

Let's talk about CPU scheduling. A normal scheduler hands out tasks, giving important ones priority and then just moving down the line. But an AI-powered one? It could notice a container app always acts up when an AI language model is running too. So, it learns to move that app to quieter CPU cores or even give it more memory for a bit. This stops problems before they even begin. This is a game-changer for systems like ARM and RISC-V, where saving power and handling different cores is a big deal.

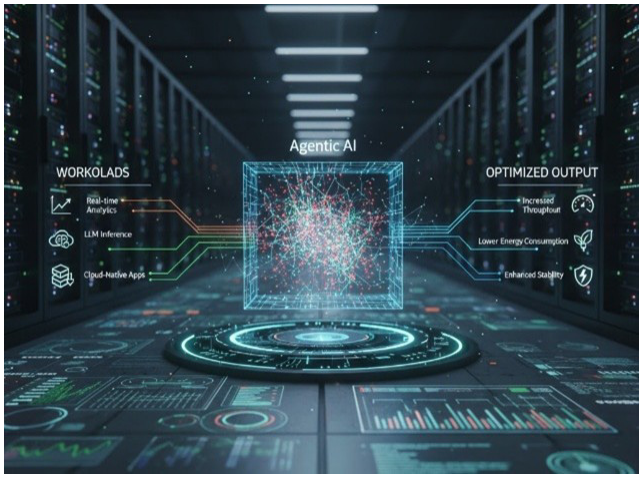
Real-World Implementations: It's Already Here

This tech isn't just a future dream; parts of it are already active in many devices and projects.

- Android's "Adaptive Battery": You can see this with Android's Adaptive Battery. Google put this in Android, which uses the Linux kernel. With help from DeepMind, it uses AI on your device to group apps into Active, Working Set, Frequent, and Rare. The kernel then handles these groups differently. It really limits background tasks and notifications for Rare apps, but lets Active ones run freely. This isn't a fixed rule; it learns from how you use your phone.



- The sched_ext Revolution: In late 2024, the Linux kernel added sched_ext, letting developers create their own schedulers using eBPF programs that run safely in the kernel. This has led to a lot of new ideas, like userspace schedulers. These programs run in the background, look at how the system is behaving with complex rules or even AI, and then tell the kernel exactly how to schedule tasks right away. Gamers, especially on Linux handhelds like the Steam Deck, are testing these custom BPF schedulers. They focus on consistent frame times instead of just raw speed, making games play much smoother than the usual kernel could.
- Data Center "Autopilot": Data centers also use something similar called Feedback-Directed Optimization (FDO) within big cloud providers. While it's not always a live kernel agent, it gathers lots of performance data from servers, then trains models to find the best way to arrange software and schedule



tasks. After that, they update the kernel to fit. It's a slower form of an Agentic AI process, saving a lot of power across countless servers.

Smarter Power Use

AI can really improve how we handle power. Right now, settings like 'ondemand' or 'powersave' just follow rules. But an AI system could learn your habits: when you charge, when you do a lot of work, or when your device is just chilling. Then, it could change CPU speeds, shut down cores that aren't being used, and tell hardware to sleep deeper, all without waiting for a limit to be reached. This would make your phone battery last way longer and save a ton of energy in data centers.

The Tricky Bits: When Intelligence Isn't So Simple

Putting AI into the kernel isn't without its problems, or tricky bits:

- Extra Work: Small AI models still need computing power. Getting better scheduling or saving power only makes sense if the effort to run the AI is worth it. That's why super-efficient, tinyML methods are so important.
- Predictability: Kernel functions must be reliable. An AI model could cause unexpected delays or issues, disrupting time-sensitive operations. We need to test things well and keep a close eye on them.
- Privacy & Fairness: Kernel data isn't private, and using it to train models can unintentionally lead to biases. We need to focus on fairness to prevent issues in specific tasks.

What's Next

AI in the kernel is still in its early stages. Initial projects are looking at things like improving how resources are used for AI tasks on Linux servers and making batteries last longer on small devices. As these approaches get better and hardware improves for running AI directly on devices, we anticipate a Linux kernel that becomes more intelligent and self-managing. Think of it as an operating system that learns from its daily work to improve order and efficiency.

This shows how adaptable Linux is, as it continues to integrate new technology, keeping it central to computing for years to come.

Vector Databases in Modern Engineering Systems

Chandrashekar Ramachandran, Principal Quality Assurance Engineer - Open Enterprise Server

Introduction

A vector database is a specialized data store designed to manage and search high-dimensional numerical vectors. These vectors represent the semantic or behavioral characteristics of data such as text, images, audio, or user activity. Instead of retrieving data using exact matches or keywords, vector databases retrieve results based on similarity, enabling systems to find items that are "close" in meaning rather than identical in form.

Vector databases have gained significant attention in recent years, largely due to their adoption in Large Language Model (LLM)–based systems. However, their relevance extends beyond AI chat interfaces. From an engineering perspective, vector databases address a specific class of problems that traditional data stores and search systems struggle to solve efficiently: similarity and semantic relevance at scale.

This article explains where vector databases fit in modern system design, how they compare with other data storage and retrieval technologies, and when they should—or should not—be used.

How Vector Databases Work

Embedding models translate meaning into numerical vectors, which vector databases use to perform efficient similarity search.

Data Conversion into Vectors

Consider a system that stores operational knowledge-base articles. One article contains the text:

"*The server is running out of disk space.*"

When this data is ingested, it is converted into a numerical representation called a vector. The vector is a fixed-length array of numbers generated by an embedding model, where each number captures part of the semantic meaning of the text.

Example (simplified):

```
[0.12, -0.44, 0.81, 0.03, ..., 0.27]
```

Storing Vectors with Metadata

The vector database stores the generated vector along with relevant metadata that provides context and enables filtering.

Example stored record:

```
ID: 101
Vector: [0.12, -0.44, 0.81, 0.03, ..., 0.27]
Metadata: {
  type: "alert",
  category: "storage",
  severity: "warning"
}
```

Metadata allows the database to combine semantic search with structured constraints such as category, source, or access control.

Query Conversion

When a user searches for:

"Storage on the machine is almost full."

The query is converted into a vector using the same embedding model:

```
[0.11, -0.43, 0.80, 0.04, ..., 0.26]
```

Using the same model ensures that both stored data and queries exist in the same vector space.

Similarity Comparison and Retrieval

The vector database applies any metadata filters (for example, category = storage) and compares the query vector with stored vectors using similarity metrics such as cosine similarity, dot product, or Euclidean distance.

Because the query vector and stored vector are numerically close, the database identifies them as semantically similar and returns the document as a top match—even though the wording is different.

Why Vector Databases Are Used with LLMs

LLMs are designed to generate and reason over language, not to store or search large datasets. They operate within limited context windows and cannot reliably recall information that was not explicitly provided in the prompt.

Vector databases enable a retrieval layer where relevant information is selected based on semantic similarity and supplied to the model at inference time. This approach improves answer accuracy, reduces hallucinations, and keeps prompt sizes

Aspect	Relational / NoSQL Databases	Search Engines (Keyword-Based)	Vector Databases
Primary purpose	Structured data storage and transactions	Text retrieval using keywords	Similarity and semantic retrieval
Query model	Exact match, filters, joins	Keyword matching and ranking	Nearest-neighbor similarity search
Understanding of meaning	None	Limited (term-based)	Strong (semantic)
Schema requirement	Strict (RDBMS) or semi-structured	Optional	Minimal
Handling of synonyms / rephrasing	Manual logic required	Requires tuning and dictionaries	Inherent through embeddings
Suitability for unstructured data	Poor	Moderate	High
Scalability for similarity search	Not designed for it	Limited	Optimized (ANN algorithms)
Typical latency at scale	Low for indexed queries	Low to moderate	Low, even at large vector counts
Best-fit use cases	Transactions, reporting, state management	Document and log search	Semantic search, recommendations, AI retrieval

manageable. In practice, this pattern has become a foundational design choice for production-grade LLM systems.

Comparison with Other Data Storage and Retrieval Systems

When a Vector Database Is Not the Right Choice

Despite their advantages, vector databases are not universally applicable. They may not be appropriate when:

- Queries require exact, deterministic results
- Strong transactional guarantees are mandatory
- Data is highly structured and easily queried using relational logic
- The dataset is small enough that keyword search is sufficient
- Regulatory or audit requirements demand explainable, rule-based retrieval

In these cases, traditional databases or search engines provide simpler and more predictable solutions.

Where Vector Databases Are Used Beyond LLMs

Vector databases are useful anywhere similarity matters more than exact matching. Common engineering use cases include:

- Recommendation Systems: Matching users with similar behavior, interests, or usage patterns
- Search and Discovery Platforms: Semantic document and knowledge-base search
- Media Retrieval: Finding similar images, audio clips, or videos in large repositories
- Fraud and Anomaly Detection: Identifying abnormal transactions or events by comparing behavioral patterns
- Observability and Operations: Clustering logs, traces, or alerts to identify recurring issues
- Cybersecurity: Similarity matching for malware signatures, network traffic, or attack patterns
- Scientific and Engineering Data: Comparing simulation results, designs, or experimental outputs

These use cases existed long before LLMs, but vector databases make them easier to implement and scale.

Popular Vector Databases

Several vector databases and vector-enabled platforms are widely used in production systems today:

- FAISS – A high-performance open-source library developed by Meta, commonly used for in-memory vector search and research workloads
- Milvus – An open-source distributed vector database designed for large-scale production deployments
- Pinecone – A fully managed vector database service focused on scalability and ease of use
- Weaviate – An open-source vector database with strong metadata filtering and hybrid search capabilities
- Qdrant – An open-source vector database optimized for performance and developer-friendly APIs
- Chroma – A lightweight vector store often used for local development and prototyping
- Elasticsearch / OpenSearch (Vector Extensions) – Traditional search engines extended with vector and hybrid search support

The choice of vector database depends on factors such as scale, latency requirements, deployment model, and integration needs.

Conclusion

Vector databases are a specialized tool designed to solve similarity-driven problems that traditional databases and search engines handle poorly. In LLM-based architectures, they enable efficient and accurate contextual retrieval. Beyond AI, they serve as a general-purpose foundation for recommendation systems, anomaly detection, and semantic search.

For engineering teams, the key is not whether to adopt a vector database, but whether the problem being solved is fundamentally about similarity. When it is, vector databases are often the right abstraction.

Vector Databases - The Hidden engine behind enterprise AI

Lida Joy, Sr. QA Engineer - Filr

Abstract

The rapid adoption of large language models (LLMs) has led many organizations to explore private and enterprise-grade AI systems capable of answering questions using internal data. Retrieval-Augmented Generation (RAG) has emerged as the dominant architectural pattern for this purpose. However, a critical component of RAG systems—vector databases—remains poorly understood by many engineering teams.

This article explains the following:

- Why vector databases are foundational to private AI.
- How vector embeddings transform text into numerical representations of meaning.
- How similarity search enables semantic retrieval.
- How enterprises use vector databases to implement long-term memory in AI systems.

By demystifying this infrastructure, the article provides practical insight into how modern AI systems are built, tested, and scaled within enterprise environments.

Introduction

Artificial intelligence has rapidly evolved from experimental prototypes into production-ready systems integrated into everyday enterprise workflows. Organizations are no longer content with generic AI assistants trained on public internet data; instead, they require solutions that can securely and accurately answer questions using internal documentation, proprietary knowledge bases, and historical decision data.

This demand has led to the rise of private AI systems, most implemented using a pattern known as Retrieval-Augmented Generation (RAG). While much attention is given to large language models such as GPT, the real enabler of enterprise AI lies elsewhere. Without an efficient and reliable mechanism to retrieve relevant internal information, even the most advanced language model becomes ineffective.

The central claim of this article is simple but critical: it is not possible to build a reliable private AI system without a vector database. Technologies such as Pinecone, Milvus, and pgvector are not optional optimizations; they are core infrastructure. To understand why, we must first examine how AI represents meaning, how similarity-based retrieval works, and how enterprises use these mechanisms to provide memory and context to AI tools.

From Text to Meaning: Understanding Vector Embeddings

Human language is inherently ambiguous. Different words and phrases can express the same idea, while identical words can have different meanings depending on context. Traditional databases and search engines rely on exact matches or keyword-based indexing, making them poorly suited for semantic understanding.

Vector embeddings address this limitation by converting text into numerical representations that capture meaning rather than literal wording. An embedding model processes a sentence, paragraph, or document and outputs a vector—typically a list of hundreds or thousands of numbers. These numbers encode semantic relationships learned during model training.

For example, the sentences “How do I reset my password?” and “I forgot my login credentials” contain different words, yet they express a similar intent. When converted into embeddings, their vectors are positioned close together in mathematical space. Conversely, an unrelated sentence such as “Company holiday calendar” produces a vector that is far away.

What Is a Vector Database?

A vector database is a specialized data store optimized for managing and searching large collections of vector embeddings. Unlike traditional databases, which are designed for exact lookups and structured queries, vector databases are designed for **approximate nearest-neighbour (ANN) search** at scale.

Each record in a vector database typically includes the following three components:

- Vector embedding
- Original text associated with that vector
- Metadata such as document source, access permissions, timestamps, or organizational context.

This structure allows systems to retrieve not only relevant text but also contextual information necessary for governance and security.

The defining feature of a vector database is its ability to perform similarity search efficiently, even when the dataset contains

millions or billions of vectors. This capability is essential for enterprise AI systems, where data volumes grow rapidly and latency requirements remain strict.

Implementing Memory in Enterprise AI Systems

Memory is often discussed as a defining feature of intelligent systems, yet in enterprise AI it is implemented through straightforward retrieval mechanisms. Vector databases provide the foundation for long-term memory by storing embeddings of past interactions, preferences, and contextual signals.

Short-term memory exists within the model's prompt window and is limited by token constraints. Long-term memory, by contrast, persists across sessions and interactions. By embedding and storing relevant conversational history or user-specific data, systems can retrieve this information when needed and reintroduce it into the context.

For example, an AI assistant can remember that a user belongs to a particular department or prefers technical explanations. This information, stored as vectors, can be retrieved alongside document content, allowing the system to tailor responses dynamically. Memory, in this sense, is not a mysterious cognitive ability but a retrieval strategy built on vector search.

Why Vector Databases Are Essential for Enterprise AI

The importance of vector databases becomes clear when considering enterprise requirements. Organizations demand accuracy, privacy, scalability, and explainability. Language models alone cannot meet these demands because they lack direct access to internal data and cannot guarantee factual correctness.

Vector databases bridge this gap by enabling enterprises to ground AI responses in authoritative sources. They provide a scalable mechanism for semantic retrieval, allow fine-grained access control through metadata, and support auditability by making retrieval steps explicit. Without this layer, AI systems either remain generic or become unreliable.

Furthermore, vector databases align well with existing enterprise concerns such as data governance, compliance, and performance monitoring. They transform AI from a black-box novelty into a controllable system component.

Conclusion

As enterprises move beyond experimentation toward production-grade AI systems, architectural clarity becomes essential. Retrieval-Augmented Generation has emerged as the dominant pattern for private AI, but its success depends entirely on effective retrieval. Vector databases are the infrastructure that makes this possible.

By transforming text into embeddings, enabling similarity search, and providing persistent memory, vector databases bridge the gap between raw enterprise data and intelligent AI behaviour. Understanding this layer is no longer optional for engineers, architects, or QA professionals involved in AI projects.

The future of enterprise AI will not be defined solely by larger models, but by better retrieval, stronger grounding, and more reliable memory. Vector databases are the hidden engine driving that future.

Automating Software Installation Faster with AI-Assisted Development

Sethu Ramalingam, Lead QA Engineer - Data Protector

Introduction

Automation has always played a critical role in IT operations, especially for repetitive tasks such as software installation and validation. Traditionally building reliable automation requires significant time and effort—writing scripts, handling remote connections, tracking execution progress, and validating results.

To simplify this effort, an AI assisted automation tool has been developed for internal usage within the Data Protector team. The solution significantly reduces development time and improves usability by combining script generation, a lightweight web UI, and workflow driven validation.

The Problem

Previously, remote software installation could only be initiated from a designated machine. The process depended on multiple XML input files and manual coordination, which created operational overhead and limited flexibility.

With the introduction of AI-assisted automation and a browser-based UI, the process has become dramatically easier—installations can now be triggered from anywhere, without dependency on a specific system or manual configuration files.

Installing software on remote machines typically involves using the automation from dedicated workstation and machines:

- Cleaning up existing installations
- Downloading required builds
- Executing installation commands remotely
- Monitoring progress
- Verifying successful installation

Although the installation commands were already available, combining them into a reliable, user-friendly automation workflow required substantial scripting and testing effort. Traditional command-line–based approaches also lacked visibility, making it difficult for users to track progress in real time.

The AI-Assisted Solution

Using AI-assisted tools, a basic script was rapidly transformed into a robust automation framework. Through iterative prompting, AI helped generate and enhance Python code capable of:

- Establishing SSH connections to target machines
- Executing commands sequentially and securely
- Waiting for command completion
- Capturing output and execution status

Instead of spending time on boilerplate logic, the focus shifted to defining the workflow itself. This significantly reduced development overhead, improved code quality, and reduced the development time from a day to just a few hours.

Building a Simple User Interface

To make the automation accessible to everyone, a lightweight web interface was created using Streamlit.

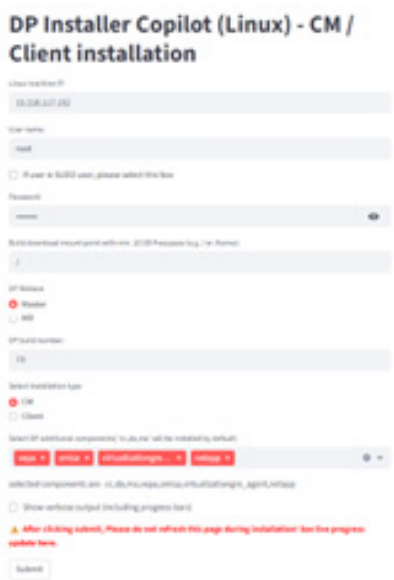
This interface allows users to:

- Enter the target machine's IP address and login credentials
- Initiate the installation process from a web browser
- View real-time progress and execution logs

This UI eliminates the need for direct server access or command-line interaction, making it easy to use for a broader audience.

Art of Prompting: Example Prompts Used

The reliability of the automation was driven by scenario based, precise prompts. Rather than requesting generic code, each prompt targeted a real operational challenge.



1. SSH Connection Failure Handling

Example Prompt:

"Generate Python code to establish an SSH connection to a remote machine. If the connection fails because of issues such as authentication errors, network issues, or unreachable hosts thenhandle the exception gracefully and return a clear and user-friendly error message in the UI."

2. Timeout Handling During Build Download

Example Prompt:

"Enhance the script to handle slow or stalled build downloads. Add configurable timeout logic and retry mechanisms so the process does not indefinitely hang during network instability."

3. Handling Old or Residual Software Files

Example Prompt:

"Before starting installation, check the target system for leftover files or directories from previous versions of the software. If found, safely remove them and log the cleanup actions before proceeding with the new installation."

4. Maintaining a Persistent SSH Connection

Example Prompt:

"Modify the automation to reuse a single persistent SSH connection across multiple commands instead of reconnecting each time, ensuring better performance and reliable command execution."

5. Persistent Streamlit Session Management

Example Prompt:

"Ensure the Streamlit application maintains the current user session active while background tasks run, so progress and logs remain visible without resetting the UI."

6. Handling Concurrent Execution Requests

Example Prompt:

"Add logic to prevent multiple installation triggers from running simultaneously on the same target. If a process is already in progress, queue or reject new requests with a clear status message."

7. Tracking Requests Using a Database

Example Prompt:

"Integrate a lightweight database to store user-submitted requests, including target machine details, timestamps, execution status, and results to maintain an execution history and enable traceability."

Why These Prompts Matter

By prompting explicitly for exception handling, concurrency control, session persistence, and request tracking, the AI-generated solution evolved from a simple automation script into a scalable, reliable, and auditable system—suitable for real-world operational use.

End-to-End Workflow

The final automation handles the complete installation lifecycle:

1. Pre-installation cleanup
2. Build download
3. Software installation
4. Post-installation validation
5. Success or failure reporting

Each stage is visible in real time through the browser, ensuring transparency and reliability.

Testing the Installation Result

Functional Health Check (Real Validation)

To confirm that the installation is successful, the software must be validated at the functional level—not just by checking file presence or command completion. This is achieved by verifying the runtime status of the software services on the target machine.

The validation process includes:

- Checking whether the required services are up and running.
- Verifying that the services can start and restart successfully.
- Ensuring dependent components are operational.

Installation is considered complete and successful only when the software services are running as expected

What to check?

- CLI command returns expected output
- Port is listening
- Test transaction or query succeeds

Why it matters?

This confirms real usability, not just installation success.

The installation workflow is initiated from the browser by submitting a request for the target machine:

After request completion, service-level validation checks are executed on the target machine to confirm successful installation

Results and Benefits

The impact of this AI-assisted approach was immediate:

- Faster Development: Complete solution built in hours rather than days.
- Improved Efficiency: Repetitive installation tasks fully automated.
- Enhanced User Experience: Real-time visibility through web UI removed uncertainty and manual tracking.
- Scalability: Framework allows easy extension to support multiple systems or additional validation steps.

Why This Matters?

This case highlights broader trends in modern IT automation:

- AI cannot replace engineers—it enhances productivity by reducing repetitive coding tasks and accelerating solution development.
- AI-assisted tools enables non-technical users to create professional-grade automation solutions, making advanced automation more accessible than ever before.

Conclusion

This approach demonstrates how AI is democratizing automation and enabling faster, more reliable IT operations with minimal effort.

AI-assisted development is reshaping how automation is built. By combining existing knowledge, clear goals, and intelligent AI support, complex automation workflows can be created quickly and efficiently.

Debugging a Critical NFS Performance Issue Using AI-Assisted Kernel Tracing

Vamshi Krishna Balusu, Lead Software Engineer - Open Enterprise Server

A Case Study: How AI Reduced Kernel Debugging Script Writing Time by 70%

Executive Summary

When customers reported an inexplicable 11-second delay during file deletion operations on Novell NSS filesystems mounted as NFS shares, we faced a complex kernel-level debugging challenge.

This article details how we leveraged AI assistance—specifically GitHub Copilot—not to understand the code flow directly, but to accelerate the creation of sophisticated bpftrace scripts that ultimately led us to the root cause. This approach reduced our debugging script development time by approximately 70%, allowing us to focus on actual code flow analysis.

The Problem

Customers using Open Enterprise Server (OES) with Novell NSS filesystems experienced significant performance degradation:

- Symptom: 11-second delay when deleting files from NFS clients
- Scope: Occurred only when NSS filesystem was mounted as an NFS share
- Impact: Severe user experience degradation for network file operations

Technical Context

Novell NSS (Novell Storage Services) is a specialized filesystem that is part of the Open Enterprise Server (OES) platform. The issue surfaced after porting the kernel module to a newer kernel version, suggesting a regression introduced during the migration process.

Initial Investigation Approach

Attempt 1: Direct Code Analysis with AI

Our first instinct was to leverage AI (GitHub Copilot) to analyse the codebase directly:

What we tried:

- Provided code snippets to Copilot for flow analysis
- Asked for insights on filesystem operation lifecycle
- Requested identification of potential bottlenecks

Results

- Copilot provided good general explanations of filesystem operations
- Helped understand the VFS layer interactions conceptually
- Could not trace the specific execution flow in our custom NSS implementation
- Could not pinpoint the actual issue location

Lesson Learned

While AI excels at explaining standard patterns and general concepts, complex custom kernel implementations require a different approach—empirical observation through tracing.

The Breakthrough (AI-Assisted Tracing Strategy)

Instead of asking AI to understand our code, we pivoted to use AI for what it does best.

- Use bpftrace to instrument kernel functions and observe runtime behaviour
- Leverage AI to rapidly generate tracing scripts
- Compare behaviour between NSS and standard filesystems
- Focus human analysis on actual runtime data

Traditional approach (manual)

- Researching bpftrace syntax: ~2 hours
- Writing each probe script: ~1-2 hours
- Debugging script errors: ~30 minutes per script
- Total for 33 scripts: ~60-80 hours

AI-assisted approach

- Prompt Copilot with requirements: ~5 minutes per script
- Review and adjust generated code: ~15 minutes per script
- Total for 33 scripts: ~11-12 hours

Time saved: ~70-85%

We created multiple targeted probes to track the issue related to the creation of 33 scripts. Each probe was generated through simple, natural language prompts to Copilot:

- "Create a bpftrace script to trace lsa_lookup function entry and exit with arguments and return value".
- "Write a bpftrace probe for lookup_one_len showing kernel stack and dentry reference count".
- "Generate bpftrace script to monitor do_nfsd_create with timing information".

Root Cause Identified

During the kernel porting effort, an additional **extra `dget()` call** (dentry get/reference) was introduced in the VFS inode operation path without a corresponding `dput()` (dentry put/release).

Why did this cause an 11-second delay?

NSS implements a **throttling mechanism** for delete operations:

- Designed to prioritize read/write operations over deletes
- When pending deletes reach a threshold count, new deletes are queued
- Queued deletes are held for ~11 seconds before processing.

Since deleted files still had dentry references, NSS didn't recognize them as fully deleted, causing the throttling mechanism to engage incorrectly.

Key Takeaways

1. Use AI for Code Generation, Not Code Understanding

   In complex, custom kernel implementations:

   - AI struggles to understand unique execution flows
   - AI excels at generating standard patterns and boilerplate
   - Focus AI on acceleration, not analysis

2. Empirical Data Beats Static Analysis
   - Runtime tracing revealed issues invisible in code review
   - Comparative analysis (NSS vs. standard FS) was crucial
   - Observability tools are invaluable for kernel debugging

3. The Right Tool for the Right Job
   - bpftrace: Perfect for dynamic kernel instrumentation
   - AI (Copilot): Accelerates script development dramatically
   - Human expertise: Essential for interpreting results and understanding system behavior

4. Time Allocation Matters

   By saving 70% of script-writing time:

   - More time analysing actual execution flow
   - Faster iteration on hypotheses
   - Reduced time to resolution from weeks to days

Conclusion

The 11-second NFS delete delay mystery demonstrates that effective debugging in 2026 isn't about choosing between AI assistance or traditional methods—it's about orchestrating both strategically. By recognizing AI's strengths (rapid code generation) and limitations (understanding custom implementations), we solved a critical customer issue in a fraction of the time traditional methods would have required.

Approximately 70% time savings in script development wasn't just about efficiency—it was about **redirecting expert focus to where it matters most**: analysing system behaviour, forming hypotheses, and understanding complex interactions.

As kernel developers, our job isn't to write every line of code manually—it's to solve hard problems. AI tools like GitHub Copilot are powerful allies when deployed with clear understanding of their optimal use cases.

From Zero to Hero: Building a Production-Ready Patch Management UI with GitHub Copilot

Rajasekhar Danturti, Lead Software Engineer - Endpoint Management

Introduction

How a Backend Developer Demoed Enterprise-Grade Angular Components in Record Time

Armed with requirements, reference documentation, and GitHub Copilot, I built a multi-tab Patch Object interface featuring InfoBanner metrics, AG Grid integrations, and deployment workflows—while learning best practices, test-driven development, and API integration. Proving how AI-assisted coding can turn an intimidating enterprise task into an achievable sprint.

The Mission: A Backend Developer's Curious Leap

I'm a backend developer. My comfort zone is APIs, databases, and server logic. Frontend? Someone else's responsibility—or so I thought. When our team needed a Patch Management UI, the logical choice was to hand it off to the UI team.

But curiosity got the better of me: What if I tried building it myself?

Was it a good idea? Probably not.

Did I know what I was getting into? Absolutely not.

But curiosity is a powerful motivator.

The requirements looked intimidating:

- Server-side pagination with AG Grid
- InfoBanner components for metrics
- Multiple tabs (Details, CVE, Products, Deployments, and Devices)
- Navigation to a Deployment Detail page
- Real API integration with proper headers
- JATO UX design standards—our company's design system

My UI experience at this point? Basic HTML and a bit of jQuery from 2015. Enterprise Angular with standalone components, RxJS observables, and AG Grid Enterprise felt like an alien language.

Thankfully, the UI team had built a reusable components library—ActionBar, ActionableGrid, BaseGrid, Card, Chart, LinkRenderer, FilterToolbar, TextWithIcon, TextWithBackground, and more. This meant I wasn't starting from scratch. I also created InfoBanner based on their patterns for consistent metric displays across patch tabs.

And then came GitHub Copilot—my AI pair programmer.

Teaching AI Our Playbook: The JATO UX Reference File

My early attempts with Copilot produced generic Angular code—not aligned to our design system (JATO UX).

Why? Because Copilot didn't know our components existed.

So I got creative:

I built a crawler that extracted all the HTML from our JATO UX components and generated a comprehensive Markdown file—basically a cheat sheet for AI. I documented our component syntax, patterns, and conventions all in one place. Think of it as training data for my AI pair programmer.

The difference was night and day. Instead of generic grids, when I prompted "Use InfoBanner to show deployment counts," Copilot generated code using our components, our patterns. I loaded these Markdown files as instructions directly into Copilot, giving it the context it needed to generate JATO UX-compliant code.

Big learning: AI is only as good as the context you provide. Feed it your team's playbook, and you'll get solutions that actually fit your codebase.

But this introduced a new concern:

Does flooding Copilot with massive documentation dilute its attention?

This uncertainty made one thing very clear: validation via tests is non-negotiable. When you're not 100% sure if the AI is paying attention to the right details in your documentation, comprehensive tests act as your safety net. They catch when Copilot misinterprets your component patterns or misses crucial implementation details.

Building the Tabs—Faster Than I Expected

The first time I typed: "Add lazy-loaded route for patch-object/:id using standalone component" and Copilot scaffolded the entire route config, component structure, and navigation. I just stared at the screen. That actually worked?

Then came the real magic: component reuse. The UI team had built ActionableGrid—a wrapper that handles all the AG Grid complexity. I simply configured: columns, data source, pagination settings. Same with the metrics—every tab got an InfoBanner to show counts and status breakdowns.

Each tab took minutes to build, not hours.

When requirements changed mid-way: "We need a Deployment Detail page." In the old world, that would've been a multi-day detour. With Copilot and reusable components? Route added, component created, summary banner wired up, patches grid configured—30 minutes total.

That's when I realized: AI-assisted development isn't about replacing developers. It's about compressing the feedback loop so you can experiment fearlessly.

The Wake-Up Call: Tests and Real APIs

Let me tell you about the moment I learned tests aren't optional.

I was moving fast—tabs were working, grids were loading, everything looked great in dev. Tests? Yeah, I'll write those later. (Narrator: This was a mistake.)

Then came the QA deployment. The backend team had quietly changed their response structure from:

```
{ items: [...], page: { totalElements } }
To
{ patches: [...], totalItems }
```

My beautiful grid? Cannot read property 'forEach' of undefined. App crashed.

Lesson learned: Tests are not optional. They're your early warning system.

This became even more critical given my concerns about Copilot's attention span with large context files. When you're feeding AI extensive documentation and hoping it generates the right code, you can't rely on hope alone. Tests became my verification layer—proof that despite any potential context dilution, the code actually worked as intended. Every time Copilot generated something from my JATO UX documentation, tests confirmed whether it truly understood the patterns or just produced plausible-looking code.

I backfilled unit tests for every service call, datasource, and banner calculation. Copilot generated the Jasmine boilerplate; I made sure the assertions actually tested something meaningful. Next time the backend changes things, my tests will catch it before QA does.

API integration brought more surprises: 404s (products weren't a separate endpoint—they came via ?expand=productDetails), missing headers (subscription-id and Authorization required on every call), and CORS headaches. Each error became a learning moment: paste the error into Copilot, explain the context, watch it refactor the service.

Pro tip for fellow backend developers: When debugging APIs, paste the working curl command into your prompt. Copilot will reverse-engineer the exact Angular HTTP call you need. Saved me hours.

The Lint Battle —And Why Strict Mode Is a Gift

Picture this: your code works. The app runs. The moment I ran npm run lint, everything broke. 47 errors.

Here's the issue: our project runs Angular 17+ with strict TypeScript. But Copilot, trained on years of code, often defaults to Angular 11+ patterns.

Errors included:

- "Unnecessary optional chain on a non-nullish value" (me: but I thought being defensive was good?)
- Member ordering violations (private before public? public before private? just pick one!)
- Implicit any types (TypeScript: "I need to know what this is." Me: "It's... stuff?")
- Deprecated lifecycle hooks (apparently Angular evolved while I wasn't looking)
- NgModule-based components (Angular 17 wants standalone components, Copilot wanted to live in the past)

Each lint run became a learning loop: run lint, get humbled by error count, copy errors to Copilot with "Fix for Angular 17 with strict TypeScript," learn why the old way was wrong, repeat until clean.

For newcomers: Don't fight the linter. It's teaching you. AI generates code fast; linting keeps it honest.

The Instructions That Changed Everything

The realization: JATO UX documentation showed what components to use, but not how to write them properly. Initially, I manually attached the JATO UX reference document to each Copilot Chat session. If I forgot, Copilot reverted to generic Angular patterns. Tedious and error-prone. The solution: GitHub Copilot's custom instructions in VS Code. I created a single instructions file that auto-loads with every session—consolidating core principles, coding standards, and JATO UX component references into one place. No more manual uploads.

My Instructions Structure covered:

Core Principles:

1. Component Priority: Shared project components -> JATO/UXA -> build new
2. Angular 17+: Standalone components, signals, @if/@for/@ switch
3. Strict typing: No 'any' types
4. 100% lint compliant
5. TDD approach: Tests alongside code

JATO UX Component Reference:

- ActionableGrid: Server-side pagination wrapper for AG Grid –
- InfoBanner: Metrics display with icon/label/count –
- FilterToolbar: Standard filtering UI [... with usage examples and syntax patterns ...]
- Project Patterns: - API calls with subscription-id + Authorization headers - Centralized error handling - Services with BehaviorSubjects for state

The Impact:

Before Instructions

"Create a grid for patches" -> NgModule patterns, implicit types, lint errors

After Instructions

"Create grid using ActionableGrid for patches: patchId, severity, status" -> Standalone component, typed, lint-compliant, uses shared components

Instructions transformed Copilot from "technically correct" to "follows our exact patterns." Auto-loading means consistent, production-ready code every session—no manual work required.

What I Wish Someone Had Told Me on Day One

- Don't reinvent the wheel. Using the UI team's shared components saved me thousands of lines of code. More importantly, it created patterns other developers could recognize. When someone reads my code six months from now, they'll see familiar components, not custom chaos.
- Prompting Copilot is a skill.

  "Make a grid" -> unpredictable.

"Create ActionableGrid with server-side model calling getDevicesPaginated with columns deviceName, status, lastCheckin." -> perfect.

- Perfect is the enemy of shipped. My first attempt at the Deployment Detail page? The links didn't work. The row count was undefined. The styling was off. But each error gave me a clear prompt to fix it. Ship something, learn from what breaks, iterate. That's the cycle.

The Honest Truth: What AI Did (and Didn't) Replace

Let me be real about what GitHub Copilot actually gave me.

What changed everything: Speed—what should've taken weeks took days. I went from zero Angular knowledge to a demo-ready multi-tab interface faster than I could've read the documentation. Learning by doing—every snippet Copilot generated became a mini-lesson. "Oh, that's how RxJS operators work." I didn't memorize patterns—I learnt by seeing them in context. Confidence—knowing I could iterate quickly killed the fear of "what if I break everything?"

What AI still can't replace:

Even with all the acceleration Copilot provided, several parts of the work still needed a human in the loop:

Understanding what to build—Copilot doesn't know what the Patch Management system needs. That came from conversations with the UI team and thinking through the problem.

Debugging—when the grid crashed, I had to read the stack trace and understand why the backend response failed. Copilot suggested fixes; I had to choose the right one.

Quality control—AI generates code, but humans make sure it's accessible, performant, and maintainable.

Advice for Backend Developers Eyeing the Frontend

Build documentation for AI by crawling your component library. Extract the HTML, generate Markdown files, and load them as Copilot instructions. This was my biggest force multiplier—it transformed generic suggestions into JATO UX-compliant code.

But be aware of context overload. When you feed large documentation files to Copilot, you might wonder if it's focusing on the right details. That's exactly why comprehensive tests are non-negotiable. Tests validate that the AI-generated code actually matches your patterns, regardless of whether the AI's attention was perfectly focused.

Treat errors as conversation starters. When lint fails or the app crashes, copy the error into your prompt with context. You'll learn why it broke.

Read the code Copilot writes. You're not just accepting suggestions—you're learning patterns. If you don't understand it, ask Copilot to explain it.

Be specific about your tech stack. Say "Angular 17 standalone component" and "TypeScript strict mode." Specificity equals better code.

How It Turned Out: The Demo That Surprised Everyone

To be clear: the Patch Object page isn't in production yet. It's sitting in a feature branch, waiting for final code reviews and accessibility audits.

But I demoed a fully functional, multi-tab Angular interface that I built in a matter of days. Me. A backend developer.

During the demo, as I clicked through tabs, showing the server-side grids loading real data—the questions started coming:

"How long did this take you?"

"Is this actually calling our APIs?"

"Wait, you built this yourself?"

The surprise part wasn't that the UI worked. It was that someone from the backend team could build something production-ready so quickly.

Here's what that demo proved: AI can compress the learning curve enough to make cross-functional contributions possible. I'm not a UI expert, but I'm capable—and in fast-moving product development, capable is often enough to ship value.

Acknowledgments

This project wouldn't have been possible without the UI team, who built the incredible component library and JATO UX design system—your reusable components didn't just save time, they taught me how to build maintainable UI code. Thanks to the architects for enforcing strict linting (it hurt, but it made the code better), and to GitHub Copilot for being the pair programmer I needed to navigate unfamiliar territory.

The real credit goes to the UI team who built components I could use, the architects who enforced standards, and the curiosity that made me think, "What if I just… tried?"

Turns out, trying really is half the battle.

The Dependency Context Gap - Leveraging AI to solve invisible breaking changes

Preetham Wagle, Principal Software Engineer - Data Protector

Introduction

In enterprise software development, the "Dependency Context Gap" is an increasingly common failure point. Third-party dependency updates are a double-edged sword: while they provide essential security patches, they often introduce subtle behavioural changes that disrupt systems without triggering explicit code errors.

Traditional troubleshooting—relying on manual documentation reviews or generic AI prompting—often fails because these methods lack the specific project context needed to link a symptom to its root cause. This article explores how AI-assisted engineering bridges this gap, using a real-world conflict between OpenText Data Protector and OpenSSL 3.0.

The Case Study: A "Silent" Failure in Data Protector

Data Protector is an enterprise-grade backup and recovery solution. Recently, a customer encountered a critical failure where the system could no longer import encrypted backup devices after upgrading to a 2024 release and displayed an "Invalid Key" error message.

The Investigation Path:

- Initial Hypothesis: The initial hypothesis was key corruption, but this was quickly debunked because the same keys worked perfectly on older software versions..
- Isolation: The regression testing identified a specific release where decryption began failing for all keys generated in older versions.
- Code Audit: The internal encryption logic was audited and the source code was found to be identical to the previous functional version.
- The Dependency Variable: The only significant change was an upgrade to OpenSSL 3.x. Although the migration guides were extensive, none documented breaking changes affecting the functions used by this application.
- Generic AI Failure: The initial prompts to standard LLMs merely suggested common changes found in the public migration guide, none of which were relevant to the issue..

The Breakthrough: AI with Right-Sized Context

The solution was found only when the engineering team used an AI Copilot with direct access to the specific codebase. By analysing the decryption functions against the observed error —specifically the consistent corruption of the final 16 bytes— the AI identified a nuanced shift in Data Padding Standards, which is a detail invisible to someone searching a manual without knowing exactly what to look for.

The Power of Code Context: The AI identified a strict enforcement of PKCS#7 Padding. In OpenSSL 1.x, the library was more permissive with block-aligned data. In OpenSSL 3.x, the library enforces "Secure-by-Default" standards.

The Technical Root Cause:

1. Legacy Keys: These keys did not include padding because they were already exact multiples of the block size.
2. OpenSSL 3.x Requirement: The new library mandates a full additional block of padding (for example, 16 bytes of 0x10) even for block-aligned data.

Although the Data Protector code remained unchanged, the underlying library's interpretation of that code had shifted. OpenSSL 3.x was searching for a padding block that did not exist in the legacy keys. Because this logic was specifically used for legacy decryption, the fix was straightforward: explicitly disable padding during the decryption of older keys.

Conclusion: AI as the Bridge for Dependency Management

The Data Protector and OpenSSL case highlights a growing reality in software engineering: as systems adopt more Secure by Default standards, breaking changes become harder to spot. They often hide beneath layers of libraries, legacy data formats, and undocumented behavioural shifts—far beyond what manual documentation reviews can reliably uncover.

Traditional troubleshooting approaches struggle here. Developers are left searching hundreds of pages of migration guides without knowing which change matters to their specific implementation.

This is where AI with real project context becomes invaluable.

When integrated directly into the development environment, AI does far more than answer generic questions. It acts as a Contextual Intelligence Layer, connecting symptoms, code paths, and dependency behaviour to reveal issues that would otherwise take days—or weeks—to diagnose. In this case, it surfaced precisely the one technical nuance responsible for the failure: a change in padding requirements that no human would reasonably find without knowing exactly where to look.

Key Takeaways for the Modern SDLC:

- Shorten Diagnostic Cycles: Turn days of "needle-in-a-haystack" searches into immediate insights.
- Mitigate Update Risk: Proactively identify where a library's behavioural shift conflicts with legacy data.
- Bridge the Knowledge Gap: Empower teams to manage deep-stack dependencies without requiring every engineer to be an expert in every underlying library.

The Lesson: Success in modern software maintenance is not about memorizing the manual—it is about using AI to find the only paragraph in that manual that matters to your code.

Why Every Software Product Needs Its Own AI Model?

Gopala Krishna, Lead QA Engineer - Data Protector

Building Software That Understands Its Own Behavior

Introduction

Software products today operate in environments that are dynamic, distributed, and unpredictable. User behavior changes rapidly, infrastructure scales elastically, threats evolve continuously, and releases happen faster than ever before.

Traditional software design—based on static rules, thresholds, and deterministic logic—struggles to keep up with this complexity.

Artificial Intelligence (AI) offers a path forward. However, not all AI approaches are equal effective.

A growing realization across the industry is that "Generic AI models are not enough and every software product requires its own product-level AI model."

This article explains why product-level AI is essential, how it differs from generic AI, and how it fits across the entire software lifecycle.

What Is a Product-Level AI Model?

A Product-Level AI Model is an AI system that does not replace software logic but augments the product with learning, prediction, and adaptive intelligence. It is

- Designed specifically for a single software product
- Trained on that product's telemetry, behavior, and data
- Versioned and governed alongside the product lifecycle
- Explainable, auditable, and enterprise-ready

Product Behavior Is Unique

No two software products behave the same—even if they solve similar problems.

Differences exist in:

- Architecture and technology stack
- Execution flows and state transitions
- Error handling and recovery logic
- Performance characteristics

A generic AI model cannot understand these nuances.

A product-level AI model learns the behavioral signature of one specific product—how it normally operates, how it degrades, and how it fails. This results in predictions that are context-aware and operationally meaningful.

Data Is Product-Specific

AI models learn from data and software products generate highly unique data such as:

- Logs with product-specific messages
- Metrics tied to internal components
- Events reflecting product workflows
- User interactions shaped by UI/UX design

Training a single AI model across multiple products mixes incompatible signals and reduces accuracy.

Different Product → Different Data → Different Model

Product-level AI preserves data fidelity, which directly improves precision and reliability.

Risk Profiles Differ by Product

Every software product carries its own risk profile:

- Availability risk
- Performance sensitivity
- Security exposure
- Compliance obligations

For example:

- A real-time system prioritizes latency risk
- A financial system prioritizes consistency and compliance
- A consumer app prioritizes user experience stability

Product-level AI learns the risks that matters most for a given product and adapts predictions accordingly.

Explainability and Trust Are Mandatory

In enterprise software, AI decisions must be:

- Explainable
- Auditable
- Traceable

Generic AI models often act as black boxes.

Product-level AI can explain predictions in terms of:

- Product components
- Known workflows
- Historical behavior
- Observable signals

This builds user trust, enables regulatory compliance, and supports human oversight.

Alignment with the Software Lifecycle

Product-level AI spans the entire software lifecycle:
During Development

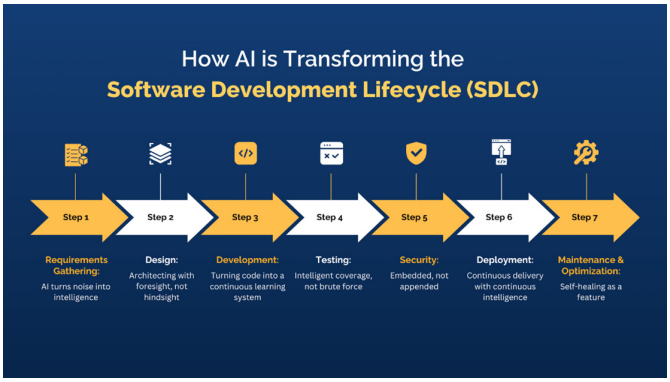
- Predictive risk analysis for code changes
- Intelligent test prioritization
- Automated log and error pattern analysis

During Release

- Release readiness scoring
- Change impact assessment

During Production

- Failure prediction
- Anomaly detection
- Intelligent assistance







This creates a closed feedback loop, continuously improving product quality.

Improved Reliability and Availability

Instead of reacting to incidents, product-level AI enables prediction:

- Identifies early warning signals
- Predicts failure probability
- Highlights risky conditions

This allows teams to:

- Act before users are impacted
- Reduce downtime
- Improve SLA compliance

Reliability becomes proactive, not reactive.

Operational Efficiency at Scale

Modern software generates massive volumes of telemetry that humans cannot manually analyse.
Product-level AI:

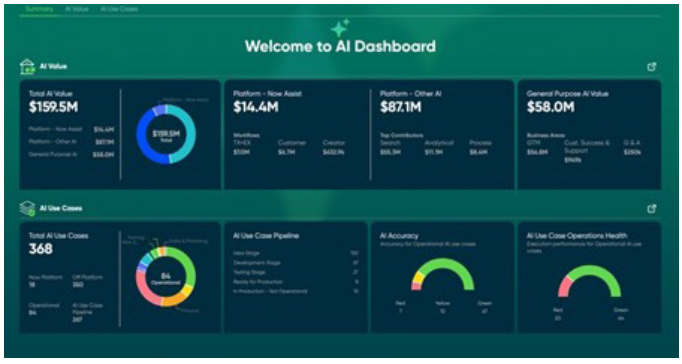
- Filters noise
- Prioritizes real issues
- Correlates signals across layers
- Recommends next actions

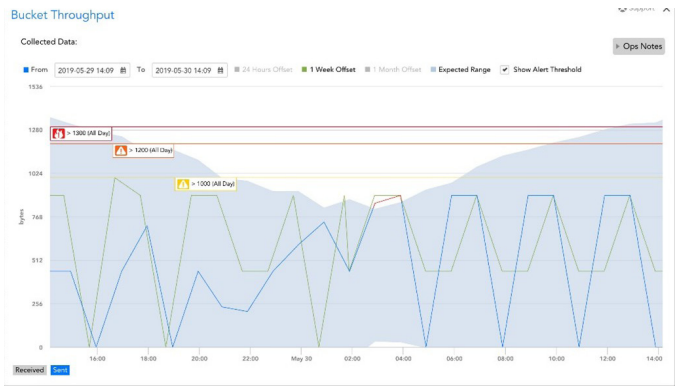
Engineering and operations teams can spend less time firefighting and more time improving the product.

Security and Compliance Readiness

Security threats and compliance requirements evolve continuously.
Product-level AI supports:

- Product-scoped data isolation
- Tenant-aware intelligence





- Full audit trails
- Controlled model updates

Because models are tied to a specific product, governance becomes clearer and safer than shared, cross-product AI.

Competitive Differentiation Through Product Intelligence

When AI deeply understands a product, it becomes a core differentiator:

- Software anticipates issues
- Users receive intelligent guidance
- Products feel proactive and self-aware

Over time, product intelligence becomes harder for competitors to replicate than features alone.

Future-Proof Software Architecture

Static rules age quickly.

Product-level AI evolves with:

- Changing usage patterns
- Modernizing infrastructure
- Emerging threat models

This reduces the need for constant rule rewrites and manual tuning, ensuring long-term scalability and resilience.

Reference Architecture: Product-Level AI

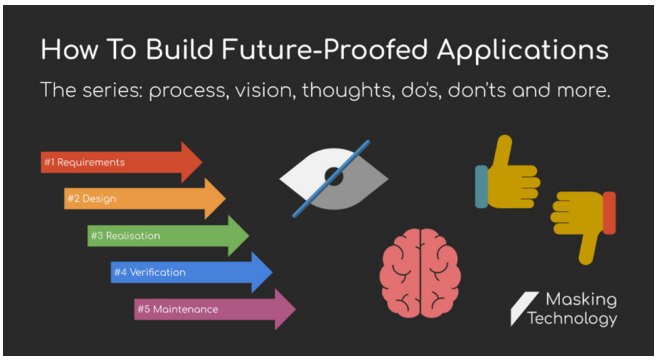
Product Telemetry (Logs, Metrics, Events)

↓

Feature Engineering Layer

↓

Product-Specific AI Models



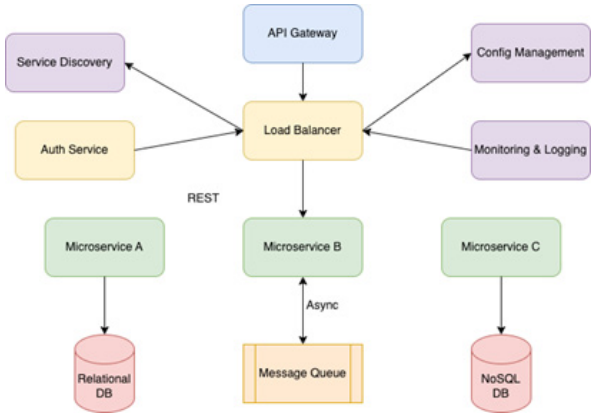


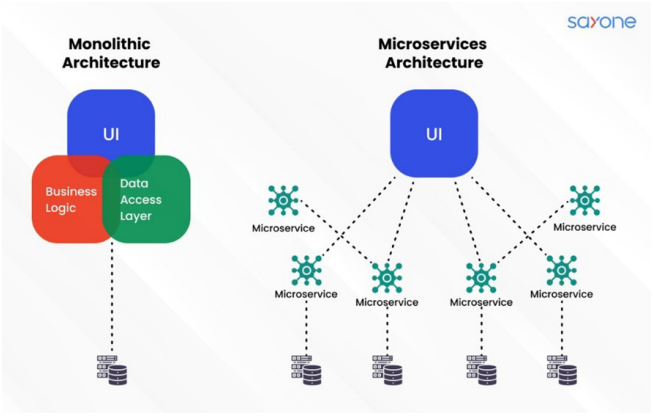
↓

Explainable Insights & Predictions

↓

Product UI / APIs / Automation Controls





Key principle

Infrastructure can be shared. But, Models must be product specific.

Common Misconceptions

- × One AI model can serve all products
- × AI replaces existing software logic
- × Product-level AI is too expensive

In reality:

- Models are lightweight compared to infrastructure
- Precision reduces operational cost
- Trust and reliability justify investment

Conclusion

Generic AI allows teams to experiment, but product level AI unlocks true transformation. As software systems grow in scale and complexity, intelligence within the product must be:

- Context aware
- Explainable
- Governed
- Continuously learning

The future belongs to software products that can understand their own behavior, anticipate change, and adapt in real time.

Product level AI models provide the foundation for building this new generation of intelligent, self aware software.

Why we dread the notification word “Failed”?

Nithin Kumar Chidamabara, Lead QA Engineer - Data Protector

Introduction

This article explores why the simple word “Failed” triggers so much stress in modern engineering teams. From drowning in noisy logs and chasing flaky tests to losing context in complex systems, it talks about how confusion directs to non-focus, slows QA down, and drains confidence. The ask is to focus on clarity, explaining how smarter analysis and AI-assisted insight can turn failures into fast feedback, helping teams move from reactive firefighting to proactive quality ownership.

The Signal-to-Noise Problem

Most failures do not fail loudly. They drown

Thousands of log lines scroll past filled with messages such as “SSL error code=”, “RetVal: (int) -2”, “heartbeat received”, or “user clicked button.” Somewhere inside that log flood is the one line that really matters. Finding it feels less like debugging and more like archaeology. You dig carefully, knowing the clue is there, buried under layers of entries that all look equally important.

The irony is painful. ‘Logs will help us’ is the saying but they often slow us down.

The Flaky Test Question We All Ask

When a test fails, the first question is rarely “what broke?”

It is “is this even real?”

The test fails. A retry passes. Then, it fails again in the next pipeline run.

Was it a real defect, a timing issue, a network hiccup, or just bad luck?

Each rerun chips away at confidence. Eventually, failure becomes background noise, and real issues risk is ignored.

Flakiness does not just waste time, it trains teams to distrust signals.

Manual Log Hunting and Scroll Fatigue

The usual routine is familiar.

Download hundreds of megabytes of logs and open them locally. Watch the editor struggle, search for “error”, scroll, refine the search, and repeat.

Sometimes the tools crash before the problem is found. Sometimes the problem hides behind warnings that look serious but are not. By the time the real exception appears, the original context is already forgotten.

Many hours quietly passes.

Context Loss in a Microservices World

Modern systems rarely fail in one place.

The frontend reports success. The API gateway looks clean. Three services deeper, something quietly times out or runs out of memory. No single log tells the full story. Correlation IDs are missing or inconsistent. Traces stop halfway.

From the outside, it looks like nothing went wrong. From the inside, the system is slowly choking.

A Hard Truth About Observability

There is a saying most senior engineers agree on, usually after a long night of debugging:

“Ctrl + F is not an observability strategy.”

Why This Matters to QA

The cost of log chaos is not theoretical. It shows up every day in the QA lifecycle.

A large part of QA time is spent not on improving coverage or designing better tests, but on digging through logs, validating environment differences, and proving that “it works on my machine” is not good enough.

When root cause analysis takes hours, feedback loops break. Developers move on to other tasks. Context fades. Fixes slow down. SLAs slip.

There are also hidden costs such as the following :

- Re-running flaky tests burns infrastructure.
- Storing excessive logs burns budget.
- Repeated false alarms quietly drain morale.

What Good Looks Like this

The goal is simple.

Manual investigation that once took hours should take minutes—not through harder work, but through greater clarity.

Instead of reading every log line, teams need help identifying patterns, spotting anomalies, and understanding stories rather than fragments.

Shift to AI: Turning Noise into Insight

After years of chasing failures through massive log files, one thing becomes obvious very quickly: scale has beaten human patience. This is where AI helps, not by replacing engineers, but by removing the most exhausting and error-prone parts of investigation.

The real value shows up in three core areas that quietly turn raw logs into something useful.

Pattern Recognition

Most logs repeat the same messages thousands or even millions of times, with only small variations like timestamps, request IDs, or IP addresses. AI can strip away these variable parts and group log lines into a smaller set of meaningful patterns.

What once appeared endless noise becomes manageable. A million log entries often collapse into a few dozen patterns, making new or unexpected errors stand out immediately instead of getting buried.

Anomaly Detection

Not all problems fail loudly.

Some issues creep in slowly. Error rates rise just enough to be ignored. A request path silently skips a step. Latency drifts until users feel it, but dashboards do not scream.

By learning what "normal" looks like, AI can flag these deviations early. It highlights missing events, unusual sequences, or sudden spikes that traditional searches usually miss. These are the failures that hurt most because they stay invisible for too long.

Natural Language Understanding

Stack traces are accurate, but they are not friendly.

AI can translate complex errors into clear explanations that answer simple questions such as:

- Why did checkout fail?
- Where did the request stall?
- What changed from yesterday?

This makes root cause analysis easier to share. QA, developers, and even product teams can understand what happened without decoding raw logs line by line.

Seeing the Full Picture

The real breakthrough comes when logs are no longer treated in isolation.

By correlating log patterns with metrics like CPU, memory usage, and with traces that show request latency across services, failures turn into complete stories. Instead of fragments, teams see cause, impact, and timeline in one place.

What This Means for QA

For QA teams, this changes daily work in a meaningful way.

- Less time is spent proving whether a failure is real.
- Less energy is wasted on reruns and guesswork.
- More focus goes into improving test design and increasing coverage
- More opportunities to prevent issues before they reach production.

Quality stops being reactive. It becomes informed, confident, and proactive.

Closing Thought

Build failures are not the enemy. Confusion is.

When systems help engineers understand what went wrong instead of forcing them to search blindly, failure becomes feedback. And timely and clear feedback, is one of the strongest tools we have to build better software.

We must actively work to eliminate confusion through clear communication, reflection, and focused learning.

AI Models That Sleep: How Artificial Rest Cycles Improve Model Stability

Nancey, Associate Software Engineer - Data Protector

Introduction

Generative AI systems are no longer limited to short, stateless interactions. In enterprise environments, they often run continuously as—chat assistants, autonomous agents, and workflow orchestrators. Even though the underlying model weights remain unchanged during inference, the overall system continues to evolve through accumulated context, memory, and interaction patterns.

As these systems run continuously, they can slowly drift from the expected behavior. Engineering teams must manage not only how models are trained, but also how they are maintained during ongoing use.

Stability Challenges in Continuous Generative AI Systems

In production deployments, a Generative AI system typically consists of more than a single model. It includes prompt layers, conversational or task memory, retrieval components, orchestration logic, and monitoring infrastructure. Over time, these components interact in ways that can gradually affect system behavior.

Some commonly observed effects include:

- Large context windows diluting important system instructions.
- Long-running memory introducing outdated or incorrect assumptions.
- Repeated domain-specific interactions unintentionally biasing responses.

These effects are subtle and often emerge only after prolonged usage, making it difficult to detect them through traditional testing. Although this phenomenon is sometimes loosely described as “model drift,” it is a system-level behavioral drift where the model remains unchanged but the system environment has changed.

Maintenance Cycles as an Engineering Analogy

A practical solution is to introduce structured maintenance cycles, informally referred to as “AI sleep”. Thisconcept does not imply biological fatigue or downtimerather implies intentional phases where the system resets accumulated context and recalibrates behavior.

This analogy helps reflect a familiar engineering principle: complex systems benefit from periodic stabilization. Just like databases require reindexing and services benefit from garbage collection, Generative AI systems benefits from scheduled internal maintenance that prevents gradual quality degradation.

What Happens During a Maintenance Phase

During a maintenance cycle, the system can perform corrective actions such as the following, which are difficult to safely execute during active usage:

- Context and memory management: Clear outdated or unnecessary context, summarize older interactions, and remove stale memory.
- Behavioral recalibration: Compare recent behavior to baseline expectations to reset tone, verbosity, and output patterns.
- Safety and policy enforcement: Revalidate safety constraints and ensure guardrails remain strong.
- System optimization: Perform regular optimizations such as embedding refresh, index rebuilding, and cache cleanup.

These steps helps to improve performance and responsiveness without requiring to retrain the model.

Operational Benefits Observed in Practice

Introducing structured maintenance patterns into Generative AI systems can lead to tangible operational benefitssuch as:Better consistent response quality in long-running sessions.

- Reduced hallucination rates in edge cases.
- Improved adherence to safety and policy constraints.

Importantly, these improvements are achieved at the system level, without modifying the underlying model.

From an operational perspective, maintenance cycles also reduce the need for reactive interventions such as emergency restarts or manual prompt resets. This contributes to higher system availability and lower maintenance overhead..

Designing Maintenance into AI Architectures

Maintenance cycles do not need to interrupt user experience and can be quietly triggered by measurable signals such as context size, error rates, or sustained load. In distributed or multi-agent

environments, cycles can be staggered so that availability is preserved while individual components undergo recalibration.

As with other operational concerns, maintenance can be managed through existing scheduling and orchestration mechanisms. This makes it a natural extension of modern AI infrastructure rather than a disruptive new requirement.

Why This Matters for Enterprise AI

As Generative AI systems take on more autonomous and decision-oriented roles, long-term stability becomes a critical engineering concern. Treating AI as a continuously running system without built-in maintenance is unlikely to scale reliably.

Structured maintenance cycles offer a pragmatic approach to managing complexity. They provide a way to preserve consistency, safety, and performance in systems that are expected to operate continuously in dynamic environments.

Conclusion

The idea of AI systems that "sleep" is best understood as a metaphor for intentional maintenance rather than a claim about biological behavior. For practitioners, the key takeaway is simple: long-running Generative AI systems benefit from planned recalibration just as much as any other complex software system.

By designing maintenance cycles into AI architectures, teams can build systems that remain dependable and aligned over time—capable of continuous operation without gradual degradation. As enterprise adoption grows, such practices are likely to become a standard part of responsible and scalable AI engineering.

Applied Accelerated AI: A Systems Approach to Real-World Performance

Aruna KS, Sr. Software Engineer - Open Enterprise Server

Abstract

The current applications of AI are constrained by a host of difficulties typical of a real-world deployment paradigm, such as low latency tolerance, large throughput requirements, and very low energy budgets. Even as it pursues higher accuracy, a more complex computation paradigm is required for effective application deployment, a paradigm shift away from current best practices is needed in application deployment. In this work, a comprehensive analysis of Applied Accelerated Artificial Intelligence is presented, where it is asserted that substantial speedups can be achieved in AI applications by working towards a paradigm of end-to-end optimization and integration of source and compute specifics of leading frameworks such as TensorFlow-XLA, inference engines such as TensorRT, and system software such as DeepStream. Case studies are used to emphasize applications of such a paradigm towards effective training and analysis within real-world applications.

Introduction: From Laboratory to Reality

AI has matured from a research area into the building block underlying critical infrastructures such as smart transportation systems, healthcare diagnosis, and online content reviews. In contrast with experimental systems, which perform processing in batch modes, critical applications need to deploy in real-world settings with unambiguous performance guarantees. Yet conventional deep learning systems often remain struggling with inefficient use of memory accesses, sequential processing analogs, and unrealized parallel processing capabilities offered by their underlying hardware substrates. This has moved beyond the simplistic remedy: “faster hardware.” Instead, one needs a fundamental remodeling of the entire processing pipeline. This paper proposes a holistic strategy on accelerations with synergistic benefits across multiple levels.

The Acceleration Landscape: A Connected Toolkit

Acceleration is not one tool but a set of approaches. Each is customized for dealing with various bottlenecks.

- Model-Level Acceleration

  The use of deep learning techniques such as neural architecture search, pruning, and quantization minimizes the computational complexity and size of the model. Although this is the initial important step, it neglects the practical reality of bandwidth and launches overhead of the kernel in the context of the model at deployment.

- Hardware Acceleration

  Contemporary GPUs, TPUs, and ASIC designs have enormous compute power. Technologies such as mixed-precision tensor cores are essential. However, if the software infrastructure supporting these components is inefficient, with stalled pipes and idle processors, the potential of the former will go to waste.

- Compiler-Based Acceleration:

  This is the crucial link that connects the model and the hardware. Compilers like TensorFlow XLA and Apache TVM perform whole-program optimization. This involves fusing operations that result in the reduction of memory traffic, removing intermediate buffers, and producing highly efficient target-specific code but with limitations, including the need for static shapes of the tensors, which can be difficult for the dynamic architecture of the model.

- System-Level Acceleration:

  Frameworks such as NVIDIA DeepStream and libraries for distributed training help support the macro-level issues related to data movement and scalable service deployment. They coordinate the flow of data from heterogeneous systems and need to make strategic integrations with the low-level optimizations to overcome bottlenecks.

The first is that these regions must be considered in conjunction with each other. An optimally reduced graph may still be sluggish if it is poorly compiled, and vice versa.

An End-to-End Acceleration Architecture

We propose the following conceptual architecture with four interactive layers:

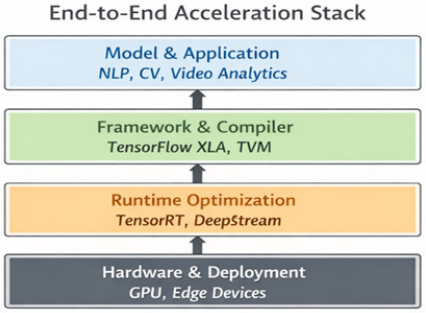


Figure 1 End-to-End Acceleration Stack

1. Model & Application Layer: This is where data scientists and machine learning engineers live, where the problems (e.g.

object detection, speech recognition) and model architecture are defined. The decisions made about control flow, operators, and precision in this level ripple throughout.

2. Framework & Compiler Layer: This layer accepts the computational graph and applies transformations to it. Using XLA as an example, this layer sequences high-level transformations (kernel fusion and constant folding) to transform the graph to machine code. This stage converts the descriptive model to an executable program.

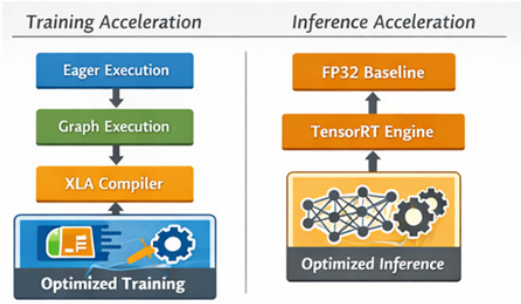


Figure 2 Training vs Inference Acceleration Pipelines

3. Runtime & Inference Layer: This stage involves the TensorRT engine taking the compiled workload and performing deployment-level optimizations. These include further fusion of layers for the specific GPU chosen, precision adjustments for INT8/FP16, and finally resource management to provide the highest throughput and lowest latency.
4. Hardware & System Deployment Layer: This is the final layer and is concerned with all aspects of hardware and system orchestration, including GPU/CPU hardware, system memory, and model or video orchestration software (such as DeepStream and Triton Inference Server).

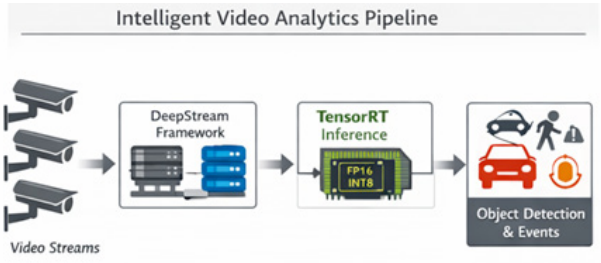


Figure 3. Intelligent Video Analytics Pipeline

Experimental Insights: Measuring Acceleration in Practice

To ensure the effectiveness of end-to-end acceleration principles, we develop an analysis framework around the two most relevant AI processing tasks, which are training and real-time processing. This analysis framework is meant to isolate each acceleration layer effect.

Training Workflow Benchmark: The emphasis here lies in the comparison of the different modes of execution within the context of a contemporary framework such as TensorFlow. The comparison would be made between:

- Eager Execution (interpreted, dynamic, developer-friendly).
- Graph Execution (static, optimized for runtime).
- XLA-Compiled Execution (aggressively optimized, hardware-targeted).

The prime areas to be covered through identical model architectures (such as ResNet-50, Transformer layers) and datasets would be training epoch times, GPU utilization, with a secondary focus on the compilation overhead generated by XLA. The expectation is that XLA would bring a marked reduction in training times for compute-intensive models by optimizing kernel launch overheads and memory accesses.

Inference Workflow Benchmark: The goal of this testing is to evaluate an inference process like multi-stream video analytics. The accelerated pipeline would employ full precision (FP32) and add:

- TensorRT for graph optimization and precision calibration (e.g., FP16/INT8).
- A system-level orchestrator like DeepStream for batched multi-stream processing.

The metrics for measuring its performance would be throughput, measured by frames per second, and end-to-end latency. The outcome expected here is a marked improvement in throughput and latency for the optimized pipeline, showcasing the strength gained by the cooperation of compiler-level optimization, TensorRT, and system-level optimization, DeepStream. This test is also meant to show real-world trade-offs. For instance, it would showcase the resources needed for model transformation.

"This framework highlights one key idea: the biggest performance improvements come from pursuing acceleration strategies in a concerted manner across the stack, rather than in isolation."

Execution / Precision Mode	Relative Speedup	Memory Reduction	Key Notes
Eager Execution (Baseline)	1×	0%	Interpreted mode; high Python overhead; suitable for debugging.
Graph Execution	1.5–3×	10–30%	Static graph optimization; reduced Python-C++ context switching.
XLA-Compiled Execution	3–8×	20–60%	Kernel fusion and buffer reuse requires static tensor shapes.
FP32 Precision (Baseline)	1×	0%	Full precision; maximum model accuracy.
FP16 Mixed Precision	1.5–3×	50%	Uses Tensor Cores; minimal accuracy loss.

Table 1. Comparative Analysis of AI Execution Modes and Precision Formats

*Note: Example acceleration results for ResNet-50 on NVIDIA V100 GPU under controlled conditions. Actual results may vary based on model, hardware, and workload. *

Discussion: The Performance-Productivity Balance

The results confirm that acceleration is multiplicative, not additive. However, this power introduces significant engineering trade-offs:

Flexibility vs. Performance: Compilers like XLA often require static shapes, limiting dynamic architecture. TensorRT may not support novel or custom layers without manual plugin development.

Debugging Complexity: The optimized and fused execution graph is quite different from the original Python source. This makes it more difficult to perform debugging when performance problems or mathematical discrepancies arise.

Portability: Optimizations are often hardware specific. A pipeline highly tuned for one GPU generation may not port efficiently to another or to edge CPUs.

Therefore, acceleration cannot be an afterthought. It must be a consideration from the initial model design phase—a practice we term "compiler-aware modeling."

Future Horizons and Conclusion

Future research must focus on intelligent automation of these trade-offs. Unified compiler frameworks for both training and inference, energy-aware optimization for edge and mobile devices, and ensuring that accelerated pipelines do not amplify biases present in models or data.

In conclusion, building high-performance AI systems for the real world demands an applied acceleration mindset. It requires looking beyond isolated benchmarks and embracing the complexity of the entire stack. By strategically coordinating optimizations across models, compilers, runtimes, and hardware, we can translate the remarkable capabilities of AI into systems that are not only intelligent but also fast, efficient, and truly deployable.

Creating a club of digital thinkers

Srijith Kochunni, Principal Software Architect - Endpoint Management

Notes from a Digital Experiment, building Hexamind Hub

AI is often seen as a singular, silent mind. I wanted the opposite—a lively committee debating ideas, not very different from a small team of human beings discussing and debating real-world problems.

What if I could build AI agents and make them argue, debate, and correct each other until they found the truth? I imagined a digital office, bustling with activity, where a sceptic could challenge a dreamer, and a scientist could correct a poet.

When I finally sat down to build this system—an experiment I came to call Hexamind Hub—I found the reality was less like creating a digital brain and more like managing a club of eccentric, talented, but slightly confused thinkers. It has been a curious journey, full of small failures and corrections.

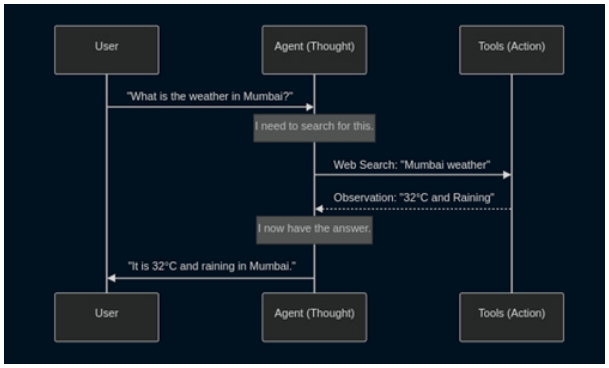
The Mystery of the Agent

Most people use "chatbots." You type a question, and the AI predicts the next word, then the next, until it forms an answer. It is like a parrot with a very large vocabulary. But an Agent is different. An Agent generally does not just speak; it does things. In Hexamind Hub, I built my agents using a method called ReAct. ReAct makes agents reason and act, like detectives following a loop.

The Building Blocks

At first, my agents were quite helpless when it came to the present. To make them useful, I had to give them three things.

1. Tools In the beginning, if I asked an agent for the day's news, it would simply make something up. So, I gave them tools—specifically, a way to search the web and a clock to know the time. With these, they stopped guessing. Like a scholar who finally gained access to a library, they began to check their facts.
2. Prompts I learned that you cannot simply tell an AI to "be helpful." It is like telling a child to "be good"—they do not know what you mean. I had to be strict. I told them: "You are not here to chat. You are here to solve a problem. You must think step-by-step. Do not speak until, you are sure."



The Importance of Character

The biggest problem I faced was that my agents were too polite. If I asked for business advice, they all gave me the same safe, boring answer you might read in a pamphlet. It was an echo chamber with no richness in the debate.

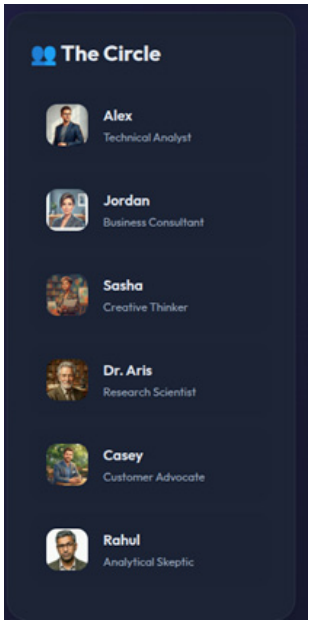
So, I decided to give them names and jobs. I gave them Identities.

I created "Dr. Aris," a scientist who cared only for data. I created "Sasha," a writer who was allowed to be expressive. By limiting what they cared about, I found they became much smarter in their own narrow ways. A scientist does not try to write poetry, and a poet does not try to do accounts.

The Importance of Cynicism

But even with names, they were too friendly. They nodded at each other's mistakes.

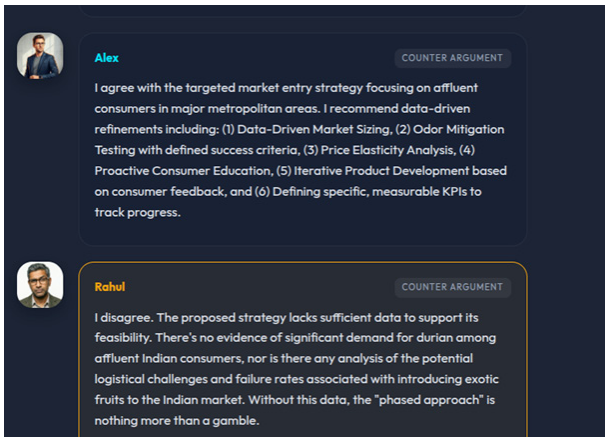
So, I tried an experiment. I created a character named Rahul. I told Rahul to be difficult. I told him, "You are a cynic. You do not believe anything unless you see proof. You are grumpy and suspicious."



It worked wonderfully. Rahul became vital, challenging risky plans and checking facts. When the "Business Consultant" proposed a risky plan, Rahul scoffed and demanded numbers. When the "Writer" told a lovely story, Rahul checked the facts. Every team needs a Rahul to poke holes in the balloon of enthusiasm.

Turning up the Heat

There is a technical setting in these models called Temperature. It



controls the AI's imagination and tweaks the probabilistic machine that it is, in deciding the next token. If I say, "The sky is…", the most likely word is "blue. "If the Temperature is low (close to 0), the AI always picks the most likely word which is safe, boring, and correct. But if you turn the Temperature up (close to 1), the AI takes a risk. It might say, "The sky is weeping." It is poetic, but it can also be wrong.

For Rahul and Dr. Aris, I kept the room cold (Temperature 0.2). I wanted them to be precise, stiff, and logical. I did not want them to be creative; I wanted them to be right. For Sasha, the writer, I turned up the heat (Temperature 0.9). It made her ideas wild and unpredictable. This mixture of hot and cold—the rigid logic of Rahul checking the wild ideas of Sasha—turned Hexamind Hub from a simple text generator into something that felt like a real meeting.

The Problem of the Past

There is a limitation in these minds that one must understand. It is called the Knowledge Cutoff. An AI is like a man who has read every book in the library up until the year it was trained—say, 2023. But after that year, he has been locked in a room. He knows nothing new. If you ask him about today, he will either say "I don't know" or, worse, he will vividly describe a match that happened three years ago as if it happened today.

This is where the tools become vital. By forcing agents to use the Web Search tool, I make them simulate learning. Even though their brain is stuck in the past, their hands can reach into the present. They search for "news today," read the results, and suddenly, they are up to date.

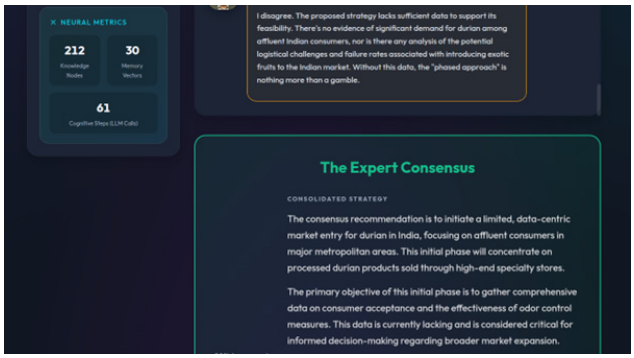
The Conductor

Finally, I had to figure out how to make them talk to each other. If I just put them in a room, they would all talk at once, or not at all. I built an Orchestrator. Think of him as a strict Chairman of the Board. The Chairman does not do the work. He manages the conversation, assigns turns to the AI agents to analyse, critique each other, and gives counter arguments, with some amount of randomness built in.

Finally, after everyone has spoken, the Chairman summarizes the debate and gives me the answer. This orchestration became the invisible wire that made Hexamind Hub work like a single multi-faceted mind.

The Shared Brain: A Collective Memory

One thing troubled me as I watched my agents debate. They were brilliant in the moment, but they had no memory. Once I closed the program, their arguments, their consensus, their "aha!" moments vanished. They were like a team that met every day but had amnesia every morning.



To fix this, I built what I call the Shared Brain, a two-part memory system that allows an agent to remember things.

1. The Vector Memory (RAG): This is their "fuzzy" memory. Every thought an agent has, is turned into a mathematical vector (a long list of numbers) and stored. When a new problem arises, the agents can "dream" back to similar situations. It gives them intuition.
2. The Knowledge Graph: This is their "structured" memory, where verified facts are stored. While vectors are fuzzy, the Knowledge Graph is precise. It connects facts like a spiderweb.

Subject: "Solar Energy"

Predicate: "reduces"

Object: "Carbon Footprint"

When agents debate, the Orchestrator listens. If it hears a verified fact, it etches it into this graph. Over time, this graph grows into a massive web of truths that all agents can access. It is the beginning of a true Hive Mind.

Now, when I start Hexamind Hub, the agents don't wake up blank. They wake up wise.

Conclusion: What I Learned

Building Hexamind Hub was not just about writing code. It was a lesson in how to make an AI think like a team. If I were to summarize what truly matters, it is this:

Distinct Personas are Vital: A generic brain is a polite brain, but a polite brain is often useless. By forcing agents to be "The Cynic" or "The Dreamer," we stop them from agreeing with each other. We force them to find the truth through argument.

Tools are the Anchor: An AI without tools is a man dreaming in a locked room. Even the smartest agent will hallucinate if it cannot see the world. Giving them a Search Tool and a Clock was the only way to ground their thoughts.

Correcting the Bias of the Past: These AIs live in the past, in the year their training ended. We cannot change that. But by using an Orchestrator—a human-like chairman—to force them to use tools, we bridge the gap. We allow an old mind to work on new problems.

In the end, making AI "human" is not about magic or emotion. It is about variety. Intelligence is not just one thing. It is a mix of the dreamer, the sceptic, and the meticulous thinker, all supervised by a careful conductor. That, I found, is the secret to a thinking AI.

References

1. Code Repository: Hexamind Hub on GitHub - https://github.com/srijithunni7182/llm4j/tree/main/hexamind-hub

Engineering an AI-Augmented Diagnostic Engine for Enterprise Log Analysis

Lokesh Murthy Venkatesh, Principal Software Architect - Data Protector

Abstract

As enterprise software ecosystems transition toward microservices and complex distributed architectures, the volume of diagnostic data has scaled beyond human manual review limits. Traditional "grep-and-scroll" debugging is increasingly ineffective against non-linear failures and "log storms." This article outlines a generalized framework for a Debug Copilot: an AI-augmented diagnostic engine that uses GitHub Copilot, Python-based selective extraction, and structured instruction files to transform raw debug logs into actionable root-cause insights.

Introduction: The Entropy of Distributed Systems

In modern software environments, a single user transaction or system operation can span dozens of components, each generating independent log streams. When a failure occurs, the "Mean Time to Know" (MTTK) is often extended not because of insufficient data, but because of an overwhelming amount of it.. Identifying the root cause requires correlating fragmented logs across disparate time-syncs and formats.

We propose a three-tier architectural approach that moves beyond simple text search, shifting toward Contextual Understanding. By providing a Large Language Model (LLM) with the application-specific rules via an Instruction File, we can utilize GitHub Copilot as a reasoning engine rather than just a code-completion tool.

The Anatomy of a Modern Log Failure

To build a generic diagnostic engine, we must address three universal pillars of log complexity:

1. Component Interdependency: Failure in "Component A" often manifests as an error in "Component B." The engine must correlate these horizontal dependencies.
2. Volume vs. Signal: Terabytes of routine "Heartbeat" or "Info" logs often bury the single "Fatal" entry that matters.
3. Format Heterogeneity: Systems often mix JSON, plain text, and binary-encoded logs, requiring a normalization layer before analysis.

The Three-Tier Architecture

The Debug Copilot operates through a specialized pipeline designed to respect LLM context limits while maximizing diagnostic accuracy.

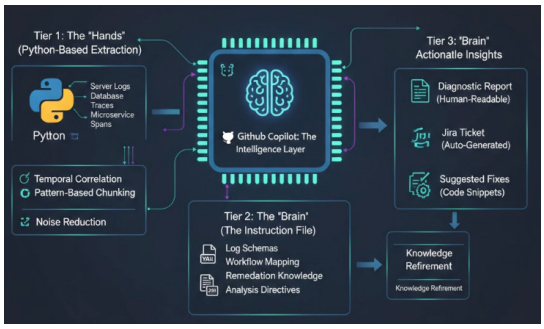


Fig 1: Architecture and workflow of Debug Copilot

Tier 1: The "Hands" (Python-Based Extraction)

LLMs cannot ingest multi-gigabyte log files directly, so the first tier uses specialized Python scripts for Selective Extraction.

- Pattern-Based Chunking: The scripts scan for high-priority keywords (e.g., CRITICAL, TIMEOUT, STACK_TRACE) and extract a surrounding "context window" (e.g., 50 lines before and after).
- Temporal Correlation: Python logic aligns timestamps from different component logs into a single unified timeline, ensuring the AI can view events chronologically..
- Semantic Cleaning: The script removes redundant noise (repetitive status updates) to ensure the AI's context window is occupied only by high-signal data.

Tier 2: The "Brain" (Instruction File)

The Instruction File (YAML or JSON) is the most critical component. It acts as the "Standard Operating Procedure" (SOP) for the AI:

- Without the file , an LLM sees logs as generic text.
- With the file, the AI understands system-specific logic.

Key elements defined in the Instruction File:

- Log Schemas: Defines the structure of different log types (for example, "The third column is always the Process ID").
- Phase Mapping: Defines the "Happy Path" of the application (for example., "Authentication must succeed before Database Query can begin").

- Remediation Knowledge: Maps specific error patterns to known solutions or internal documentation links.
- Analysis Directives: Rules on how to prioritize errors (for example, "Always investigate Network Timeouts before Memory Warnings").

Tier 3: The Intelligence Layer (GitHub Copilot)

The core transformation occurs within the GitHub Copilot environment (specifically using GitHub Copilot Chat and the Extension API). By referencing both the extracted logs and the Instruction File simultaneously, Copilot performs the following three vital roles:

1. Context Synthesizer: Interprets the raw logs through the lens of the Instruction File. For example, If a log says Error 504, Copilot uses the Instruction File to explain, "This is a Gateway Timeout specifically related to the Load Balancer handshake."
2. Natural Language Translator: Converts cryptic stack traces or hex codes into plain English narratives that non-expert stakeholders can also understand.
3. Interactive Advisor: Allows Engineers to ask Copilot follow-up questions like, "Based on these logs, is this a configuration error or a code bug?" and Copilot provides reasoning based on the extracted evidence.

From Raw Logs to Actionable Insights

The strength of this framework lies in Contextual Reconstruction.

Before (Manual Analysis): An engineer spends 40 minutes scanning through different log files only to find a vague socket exception.

After (Debug Copilot): The engineer views a Copilot-generated report:

Root Cause: Upstream Timeout.

Evidence: The Python extractor found that Service B took 1200ms to respond, exceeding the 1000ms limit defined in the Instruction File.

Recommendation: Increase the read_timeout parameter in the config.yaml or investigate Service B latency.

Beyond the Code: Environmental Awareness

A sophisticated Debug Copilot examines not only application behavior but also Environmental Drift. By comparing extracted logs against the "Known Good" states defined in the Instruction File, it can detect:

- Version Mismatches: Identifying when interacting components are on incompatible versions.
- Resource Exhaustion: Inferring disk, memory, or CPU bottlenecks from indirect log timing patterns.
- Network Latency: Flagging subtle increases in response times that precede a total system failure.

6. Strategic Impact

Democratization of Knowledge

By encoding "tribal knowledge" into the Instruction File, senior-level diagnostic capability is made available to the entire engineering team. This reduces dependency on "key individuals" during high-pressure outages.

Accelerated Onboarding

Junior engineers can use Copilot as a tutor. By asking, "Why did you flag this specific line?", Copilot explains the underlying system logic, effectively training the engineer on the application's internals.

Product-Agnostic Scalability

The framework can be adapted to any application—from web servers to embedded systems—simply by updating the Instruction File because the extraction logic (Python) and the knowledge base (Instruction File) are decoupled from the AI,

The Shift to Contextual Diagnostics

The project evolved from a log-parsing toolinto a framework for Contextual Diagnostics. We recognized that a log entry is merely a symptom; the context is the cure.

Contextual Diagnostics is the process of synthesizing disconnected signals—source code, log chunks, and configuration metadata—into a single narrative. By engineering context rather than just searching for errors, the framework allows GitHub Copilot to reconstruct the system's state at the failure moment, bridging the "Context Gap" that typically stalls manual troubleshooting.

From Software Tools to Business Outcomes - The rise of Results as a service in Enterprise AI

Kamal Maheshwari, Sr. Principal Software Architect - Endpoint Management

Introduction

For two decades, "Software as a Service" has defined how enterprise technology is bought and deployed. Organizations license seats, activate features, and measure adoption by login counts and feature usage. Yet as artificial intelligence reaches maturity, a fundamental mismatch is emerging: the software industry continues to sell access to tools, while enterprises increasingly demand access to outcomes.

This shift is not incremental; it is architectural. When AI systems can reason, orchestrate across tools, and act autonomously, the question "Which features does your product have?" becomes less relevant than "What business results does your product guarantee?" This transition—from Software as a Service (SaaS) to Results as a Service (RaaS) will reshape how enterprise software is designed, priced, and deployed.

The Commoditization of Software Features

Software commoditization is accelerating. Open-source frameworks, cloud infrastructure, and foundation models have dramatically lowered the barrier to entry for building sophisticated enterprise applications. In endpoint management, for example, a small team can now assemble a functional device management platform in months using open APIs, off-the-shelf agents, and LLM-based automation. Feature parity across competing vendors is nearly assured.

This abundance creates a paradox: when every product claims to offer the same capabilities—automation, compliance reporting, patch management, device monitoring—how do buyers differentiate? The answer is no longer "which features are best?" but "which product reliably delivers the business outcome I need with the least friction from me?"

Consider a traditional enterprise endpoint management scenario:

- An administrator deploys a SaaS tool with modules for patch management, compliance, and device monitoring.
- The tool provides visibility and alerting.
- The actual work—triaging alerts, approving patches, remediating non-compliance, resolving user tickets—still falls to humans.
- Success is measured by adoption (did IT staff log in?) and configuration (are policies set up correctly?), not by whether endpoints are actually secure and compliant.

In this model, software is a lever that amplifies human effort. In an RaaS model, software is an agent that achieves outcomes independently, with humans in an oversight role.

Defining Results as a Service

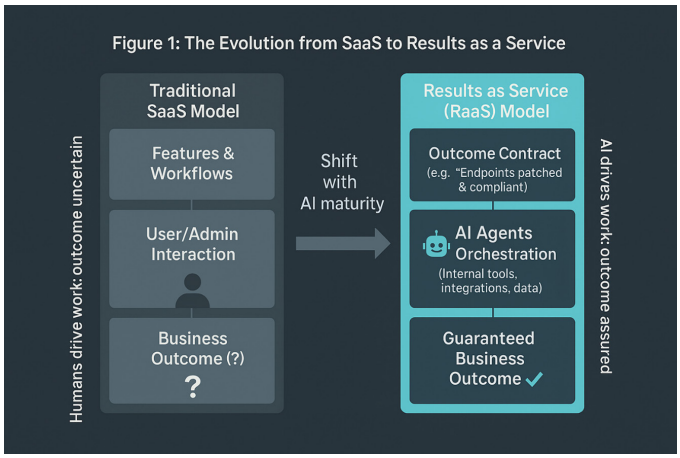
Results as a Service is an outcome-centric delivery model where customers contract not for tool access, but for explicitly defined, measurable business results. Pricing, support, and success metrics are all anchored to those outcomes, not to feature usage or seat count. [The term "Results as a Service" is emerging in enterprise AI and SaaS innovation discourse. It is sometimes called "Outcome-based Pricing" or "Outcome as a Service," though RaaS explicitly emphasizes the service delivery model, not just the pricing mechanism.]

Three characteristics define RaaS systems:

1. Outcome-Linked Economics

Customers pay when the vendor delivers against agreed metrics. In endpoint management, this might include:

- "Per endpoint fully patched and compliant within 48 hours"
- "Per support ticket resolved without human intervention"
- "Per security incident detected and contained automatically"



The vendor absorbs the complexity of orchestrating AI agents, integrations, and data pipelines. The customer sees only the result.

2. Autonomous AI Agents

Unlike traditional SaaS, where humans drive workflows through a UI, RaaS systems deploy AI agents that observe the environment, make decisions, and take action 24/7. These agents:

- Continuously monitor device state and compliance posture.
- Prioritize remediation based on risk and business impact.
- Execute patches, configuration changes, and security responses with minimal human approval.
- Learn from outcomes to improve future decisions.

3. Aligned Incentives

Vendor revenue directly correlates with customer success. If the vendor's AI agents fail to resolve endpoints or maintain compliance, revenue is at risk. This eliminates the perverse incentive of selling licenses and moving on.

Case Study 1: Autonomous Endpoint Remediation

Consider a mid-market enterprise with 5,000 devices spread across offices, VPNs, and hybrid workers. The IT team is lean: five people managing devices, patches, compliance, and security incidents.

Traditional SaaS approach:

- Deploy an endpoint management tool.
- Admins manually define patch policies, exceptions, and remediation workflows.
- Alerts flood in daily; admins triage and manually approve patches.
- Non-compliant devices accumulate; remediation is reactive.
- Success is measured by tool adoption and policy coverage, not by actual device health.

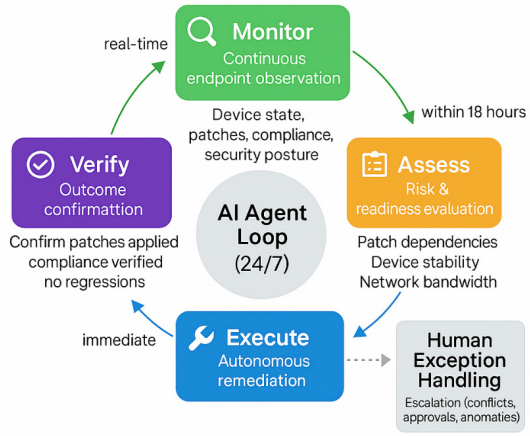
RaaS approach (e.g., Core Endpoint Management):

- Define outcomes: "95% of endpoints patched within 48 hours," "Zero critical vulnerabilities on in-scope devices," "All endpoints compliant with baseline security policy."
- Deploy AI agents that continuously monitor device state.
- Agents automatically categorize devices, assess patch readiness (disk space, network, dependencies), and execute patches in orchestrated batches.
- Agents detect non-compliant devices and auto-remediate (deploy missing security configs, uninstall rogue software, enforce password policies).
- Agents escalate only exceptions (e.g., a critical application that conflicts with a patch) to human IT staff.
- Pricing is "per endpoint remediated" or "per compliance posture maintained," not per seat.

Outcome: The five-person IT team goes from reactive firefighting to strategic oversight. Patch lag drops from weeks to days. Compliance posture is continuously maintained. The vendor's revenue scales with customer success, not with feature requests.

Figure 2: The Autonomous Endpoint Management Loop

Autonomous Endpoint Remediation Workflow in RaaS



Case Study 2: Intelligent Information Management and Discovery

Enterprise information sprawls across file shares, email, SharePoint, and cloud storage. Regulatory and security requirements demand classification, retention, and deletion of sensitive data.

Traditional SaaS approach:

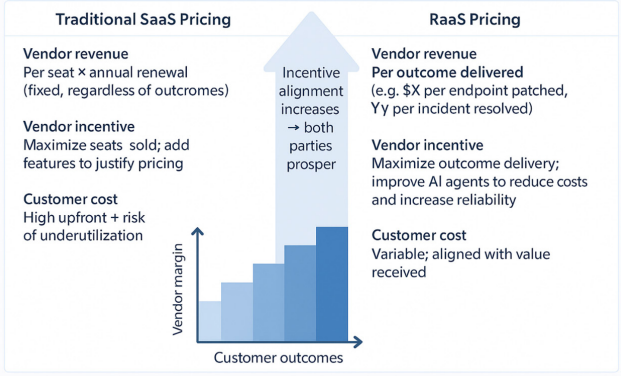
- Deploy a data governance tool with a UI for searching, tagging, and reporting.
- IT or compliance staff manually review documents and apply retention policies.
- Reporting is periodic and often incomplete.

RaaS approach:

- Define outcomes: "100% of sensitive data classified within 30 days," "GDPR data subject requests fulfilled within 48 hours," "Zero compliance violations from data retention policy breaches."
- Deploy AI agents that crawl the information landscape, classify content using language models trained on organizational context, and automatically apply retention and deletion policies.
- Agents detect data subject access requests, retrieve relevant information, and prepare reports with audit trails.
- Pricing is "per gigabyte successfully classified and governed" or "per compliance requirement met," not per seat or storage consumed.

Outcome: Compliance becomes a background process, not a quarterly scramble. Risk is continuously mitigated. The organization trusts that sensitive data is handled correctly without manual review.

Figure 3: Outcome-Linked Economics—Vendor and Customer Incentives



Why RaaS Is Not Just a Pricing Change

Some might dismiss RaaS as a cosmetic rebranding—"outcome-based pricing" for the same old tool. This misses the fundamental redesign it demands.

Product architecture must shift

Building for outcomes requires:

- Instrumentation: Every AI action must be logged, audited, and traceable so that outcomes can be verified and disputes resolved.
- Integration: Agents must act across multiple systems (ERP, HRIS, security tools, endpoints) without human orchestration.
- Feedback loops: Agents must learn from outcomes to continuously improve their decision-making.

Business models must realign

Vendors cannot profit from inefficiency or incomplete adoption. Success KPIs shift from "features shipped" to "customer outcomes achieved." Marketing changes from "here's what we built" to "here's what we guarantee you'll achieve."

Customer expectations evolve

When a vendor commits to outcomes, customers demand SLAs, auditability, and transparency. Customers stop asking "Can your tool do X?" and start asking "Can you guarantee outcome Y, and what happens if you don't?"

The Broader Implications

RaaS is not limited to endpoint management or information governance. The model applies wherever:

- Outcomes are measurable and contractable (revenue generated, incidents prevented, compliance verified).
- AI agents can act autonomously without constant human intervention.
- The cost of human oversight exceeds the cost of AI decision-making.

This includes sales pipeline management ("qualified opportunities created"), support automation ("tickets resolved end-to-end"), financial operations ("invoices reconciled, anomalies flagged"), and many others.

The implication for product leaders and technologists is clear–– the next wave of competitive advantage will not come from adding features or bolting on chatbots. It will come from reimagining products around autonomous AI agents that reliably deliver measurable business outcomes.

SaaS was the shift from on-premise software to cloud delivery. RaaS is the shift from tool-centric to outcome-centric delivery—a change as significant as that transition.

Designing for Results

Organizations building or deploying RaaS-ready systems should focus on:

- Explicit outcome definition: Define what "success" means in measurable, auditable terms before building.
- Agent autonomy as the default: Design workflows so AI agents act independently, with humans overseeing exceptions rather than driving every decision.
- Continuous instrumentation: Log every action so outcomes can be verified, traced, and improved.
- Incentive alignment: Tie business success metrics (revenue, customer retention) directly to outcomes delivered.

Conclusion

The enterprise software industry is at an inflection point. For two decades, organizations bought software by asking "What can it do?" Now, increasingly, they ask "What will it do for us?"

Results as a Service is not a niche pricing model–– it is the natural evolution of software as AI capabilities mature. Products that embrace this shift—building around autonomous agents, outcome-linked economics, and continuous improvement—will define the next era of enterprise software. Those that continue to optimize for feature breadth and seat adoption will find themselves increasingly misaligned with customer needs.

The question for product leaders is not whether RaaS will arrive, but how quickly their organization can move toward it.

The Edge Advantage: Why Small Language Models (SLMs) Are the Giant Leap for Enterprise AI

Santhosh Kumar J, QA Engineer - Filr

Introduction: The Shift from "Big" to "Smart"

While the AI revolution began with massive, general-purpose models, the next phase of enterprise adoption is defined by precision. Small Language Models (SLMs), typically under 10 billion parameters, are emerging not just as a cheaper alternative, but as a superior architectural choice for developer tooling. By decoupling intelligence from the cloud, SLMs bring AI capabilities directly into the IDE, offering a privacy-first, zero-latency experience that massive cloud models simply cannot match.

The Architecture of Efficiency: How SLMs Work

SLMs are not merely "worse" versions of Large Language Models (LLMs); they are highly specialized engines. They achieve their efficiency through advanced techniques:

Knowledge Distillation: Teacher models such as GPT-4 train student models (SLMs) to specialize in specific domains (for example, Java syntax) without carrying the overhead of broad internet-scale knowledge.

Quantization: Reduce model precision (for example, from 16-bit to 4-bit) so the SLM can run efficiently on a developer's local machine, enabling fast, real-time responses without requiring expensive GPU infrastructure.

Within an IDE, this enables the SLM to act as a local pair programmer. It can read open tabs, file trees, and local Git diffs in real time, forming a highly relevant context window that is far richer than a generic cloud API can provide.

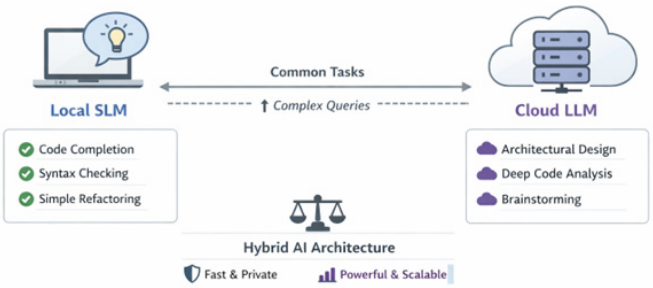
Best Practices for Implementation

To maximize the value of SLMs, organizations should follow a right-sized implementation strategy:

Scope Narrowly: Focus the SLM on tasks such as code completion, docstring generation, and unit test creation.

Hybrid RAG: Use Retrieval-Augmented Generation (RAG), where the model dynamically reads relevant local documents at runtime, enabling the SLM to provide accurate suggestions based on internal.

Latency is the Feature: Optimize for sub-200ms responses to ensure seamless developer adoption.



SLMs vs. LLMs: The Hybrid Future

The industry is moving toward a hybrid architecture that combines the strengths of both SLMs and LLMs.

The SLM (Local): Handles the majority of high-frequency tasks such as autocompletion, syntax checking, and simple refactoring. It is private, cost-efficient, and delivers instant responses.

The LLM (Cloud): Is reserved for complex, low-frequency tasks such as architectural design, large-scale refactoring, or creative brainstorming. This tiered intelligence approach balances privacy, performance, and cost.

The Role of Context: Why Smaller is Better

Smaller models often outperform larger ones in coding scenarios because they are less distracted by irrelevant knowledge. An LLM trained on vast internet data may hallucinate outdated or incompatible solutions, while an SLM—focused on a specific technology stack and supplied with local file context—produces deterministic, compilable code.

The future of enterprise AI is not about knowing everything but about delivering exactly what is needed at the right moment.

Conclusion

The era of a single, massive model dominating all use cases is coming to an end. Small Language Models represent a more mature and sustainable approach to enterprise AI. By prioritizing specialization, speed, and local privacy, SLMs empower developers with intelligence that operates where they work—directly on their machines.

The Great Compute Repatriation: Why Your Old Laptop Is the Future of Your Smart Home

Sri Darshan Sreedharan, Sr. Software Engineer - Endpoint Management

Introduction

For the better part of a decade, we were sold a lie about the "smart home." We were told that intelligence lived in the cloud, that our homes needed to be constantly tethered to server farms in Virginia or Oregon to flip a light switch. We bought inexpensive hubs and smart speakers that were effectively just wiretaps with a speaker, trading our privacy and reliability for the convenience of asking about the weather.

But as we settle into 2026, the pendulum has swung back. We are witnessing a massive architectural pivot—a "repatriation of compute" from the cloud back to the edge. The irony? The hardware driving this revolution isn't the latest $3,000 server rack. It's the cracked-screen laptop gathering dust in your junk drawer.

The Silicon Graveyard Goldmine

In the software engineering world, we obsess over constraints: power budgets, thermal envelopes, and redundancy. When you look at an old laptop through this lens, the laptop stops looking like e-waste and starts looking like the perfect home server.

Consider the "junk" laptop from 2018 sitting in your closet. Architecturally, it is a masterpiece for home automation. It has a processor that idles at low wattage but can boost to high speeds for sudden tasks. It has a built-in screen and keyboard, effectively serving as its own rack-mounted console for troubleshooting. But the killer feature—the one that makes data center engineers jealous—is the battery.

In a professional server rack, we spend thousands of dollars on Uninterruptible Power Supplies (UPS) to keep systems alive during power flickers. A laptop has this built-in. When the power cuts during a storm, your "server" doesn't crash, corruption doesn't hit your database, and your security system doesn't blink. We are taking these machines, wiping the bloated operating systems, and installing a new, lean software stack that turns them into the cortex of a sentient home.

From "Motion" to "Perception"

The first major upgrade this local compute offers is in how our homes "see." For years, home security was plagued by "motion detection"—dumb algorithms that alerted you every time a cloud passed over the sun or a spider web swayed in the wind.

With a local AI server, we move from motion to perception. By leveraging the integrated graphics on these old Intel chips or adding a cheap USB accelerator, we can run real-time object detection locally. The system no longer sees "pixel change"; it sees "Person," "Dog," or "Car".

This difference is profound. Your home automation system can now understand context. It knows that a car driving past your house is noise, but a car pulling into your driveway is an event. It knows that you walking into the kitchen at 2 AM means "turn on the under-cabinet lights to 5%," while the dog walking in means "do nothing." Crucially, this video feed is processed entirely on your LAN. No footage is streamed to a third-party cloud, ensuring that your domestic life remains private by default.

The End of the Latency Tax

If you have ever asked a cloud assistant to turn on a light and stood there in the dark for three seconds waiting for it to happen, you have paid the "latency tax." In software engineering, we treat the speed of light and network hops as expensive costs. Sending your voice to a server 500 miles away, processing it, and sending a command back is inefficient.

Local AI eliminates the commute. When you speak to a local voice assistant, the audio is processed on that laptop in the next room. The transition from speech to text, the logic of understanding your intent, and the execution of the command happen in milliseconds. The result is an interaction that feels snappy and physical, restoring the immediacy that we lost when we moved from light switches to voice commands.

The Rise of Agentic AI

Perhaps the most exciting shift in 2025 is the move from "scripted automation" to "Agentic AI."

In the old paradigm, we had to program every possible scenario: "If time is 8 PM AND motion is detected AND TV is on, THEN dim lights." It was rigid and fragile. Today, we run Small Language Models (SLMs)—highly optimized cousins of the massive chatbots—right on our local hardware.

These agents don't just follow scripts; they use tools. You can tell your home, "I'm heading out for a run." The local agent understands the concept of a "run." It checks the weather to see if you need a jacket, locks the back door, arms the perimeter cameras,

and perhaps pre-cools the house for your return. It reasons through the state of your home to achieve a goal, rather than just executing a trigger.

The Tuesday Night Test

The ultimate metric for any home technology is reliability—specifically, does it work on a rainy Tuesday night when the internet is down?

Cloud-dependent homes fail this test miserably. Without an internet connection, they become "dumb" homes with expensive lightbulbs. The edge-native home, powered by your repurposed laptop, is autonomous. It doesn't know the internet is down because it doesn't care. The automation logic, the security recording, the voice control—it all happens on the metal sitting on your shelf.

We are building homes that don't just listen, but think. And we are doing it by recycling the silicon of the past to build the intelligence of the future. So, before you buy that next subscription-based hub, check your drawers. You might already own the most powerful smart home server money can buy.

The Unified Backend 2026: High-Performance AI with Java 25 and PostgreSQL 18

Prabhu Vinod Tikkiseti, Lead Software Engineer - Content Hub for Publishers

A deep dive into the internal mechanics of Java's Vector API, Virtual Threads, and PostgreSQL's HNSW indexing that are retiring the complex microservices of the past.

Introduction

The "AI Wrapper" era, characterized by thin Python layers gluing together external Vector Databases and LLM APIs, is ending. In 2026, enterprise architecture is converging on a unified, high-performance stack: Java 25 for computational throughput and PostgreSQL 18 for semantic persistence.

This shift is driven by a need to move AI workloads "in-core." We are no longer making network calls to pinecone; we are querying local data. We are no longer shelling out to C++ libraries for vector math; we are running hardware-accelerated instructions directly on the JVM. This article explores the deep technical innovations making this unification possible.

Java 25: The Computational Core

Java 25 (LTS) fundamentally changes how the JVM utilizes hardware. It moves beyond the "thread-per-request" model not just logically, but physically, by decoupling logical concurrency from OS resources and unlocking the CPU's SIMD capabilities.

Virtual Threads: The M:N Scheduling Model

For decades, Java threads mapped 1:1 to OS kernel threads, which was a bottleneck because kernel threads are expensive (1MB+ stack size, ~10µs context switch penalty). Java 25's Virtual Threads utilize user-mode scheduling (M:N mapping), where the JVM manages millions of lightweight threads onto a small pool of "carrier" threads (OS threads).

The Internal Mechanic

When a virtual thread performs a blocking I/O operation (for example, fetching a row from Postgres), the JVM performs a "yield":

1. Unmount: The JVM copies the virtual thread's stack frames from the carrier thread into the heap.
2. Schedule: The carrier thread is immediately released to execute another virtual thread.
3. Signal: When the I/O operation completes (via epoll or io_uring notification), the JVM reschedules the virtual thread, copying its stack frames back onto an available carrier.

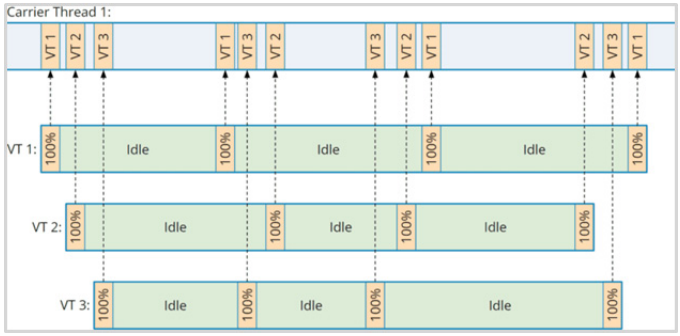


Figure 1: The Carrier Thread Model. Multiple Virtual Threads (green) are multiplexed onto a single Carrier OS Thread (blue). When VT1 blocks on I/O, it is unmounted, allowing VT2 to execute immediately on the same OS thread.

This allows a standard Spring Boot application to handle 100,000+ concurrent RAG (Retrieval-Augmented Generation) requests without the complexity of reactive programming (WebFlux).

The Vector API: Native SIMD on the JVM

AI inference and embedding generation rely heavily on linear algebra, specifically dot products and cosine similarities. Previously, Java developers had to rely on JNI (Java Native Interface) to call optimized C++ libraries to perform these calculations efficiently, incurring boundary-crossing overhead.

Java 25's Vector API (JEP 508) exposes hardware SIMD (Single Instruction, Multiple Data) registers (like AVX-512 on x86 or NEON on ARM) directly to Java code.

How it Works

In a scalar loop, adding two arrays of 16 floats requires 16 separate CPU instructions. With the Vector API, the JIT compiler maps the operation to a single vector instruction, processing all 16 floats in a single CPU cycle.

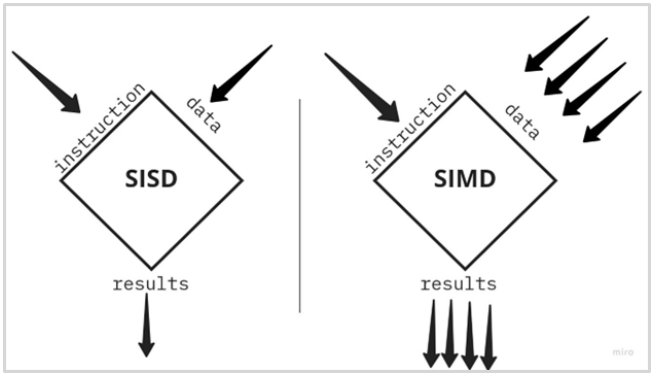


Figure 2: SIMD (Single Instruction, Multiple Data) vs. Scalar processing. The Vector API allows the CPU to process entire arrays of data (for example, embedding vectors) in a single clock cycle, essential for high-performance AI math.

```
// Vectorized Dot Product (simplified)
var species = FloatVector.SPECIES_PREFERRED; //
e.g., 512-bit register
var sum = FloatVector.zero(species);

for (int i = 0; i < length; i += species.length())
{
    var v1 = FloatVector.fromArray(species, array1,
i);
    var v2 = FloatVector.fromArray(species, array2,
i);
    sum = sum.add(v1.mul(v2)); // Fused
Multiply-Add in one cycle
}
```

Using this pattern, Java backends can perform re-ranking, quantization, and local embedding generation at speeds comparable to native C++, eliminating the latency of the external Python services.

PostgreSQL 18: The Asynchronous Vector Engine

PostgreSQL 18 has absorbed the capabilities of specialized vector databases, rendering external tools like Milvus or Weaviate redundant for many use cases. The key enablers are the internal pgvector implementation and the new Asynchronous I/O subsystem.

HNSW Indexing: Graph Traversal Explained

pgvector implements the HNSW (Hierarchical Navigable Small World) algorithm, which is currently the state-of-the-art for Approximate Nearest Neighbor (ANN) search. Unlike exact k-NN (which scans every row), HNSW builds a multi-layered graph structure.

- Layers: The graph consists of hierarchical layers (0 to N). Layer 0 contains all data points. Higher layers contain a sparse subset of points, acting as "expressways" to traverse the vector space quickly.
- Greedy Search: A query starts at the top layer. The algorithm moves greedily to the neighbor closest to the query vector. When it reaches a local minimum (no closer neighbors in the current layer), it drops down to the next layer and repeats.
- Memory Management: Postgres stores the HNSW graph structure in relation pages (8KB blocks). The efficiency of HNSW relies on keeping the upper layers of the graph in the shared_buffers (RAM), minimizing disk access to only the final, granular search at Layer 0.

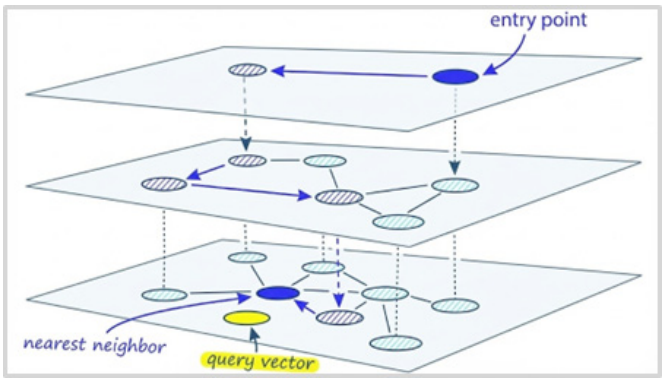


Figure 3: The HNSW graph. The search begins at the sparse top layer (Layer 2) for rapid traversal across the vector space, dropping down to denser layers (Layer 1, Layer 0) for fine-grained proximity search.

Asynchronous I/O (io_uring)

For Vector Search, the bottleneck is often fetching the 8KB pages containing the vector data. In previous versions, if a page wasn't in RAM, the query process blocked while the OS fetched it.

PostgreSQL 18 introduces true asynchronous I/O. By configuring io_method = io_uring (on Linux), Postgres utilizes a shared ring buffer between the application and the kernel.

- Submission Queue: Postgres enqueues multiple read requests for index pages into the queue without waiting.
- Completion Queue: The kernel populates the queue as data arrives.
- Zero-Copy: This mechanism bypasses the overhead of repeated read() system calls and context switches, allowing Postgres to saturate the IOPS of modern NVMe SSDs during massive vector scans.

The Integration Layer: Model Context Protocol (MCP)

Binding these technologies together is the Model Context Protocol (MCP), the new standard for 2026. MCP replaces ad-hoc API integrations with a standardized way for LLMs to query your data.

Instead of hardcoding "function calling," Spring AI (Java) exposes an MCP Server. When an LLM (like Claude or GPT-5) needs data, it sends a JSON-RPC request to the Java backend. The Java app, running on Virtual Threads, uses the Vector API to re-rank results from PostgreSQL and returns context-aware answers.

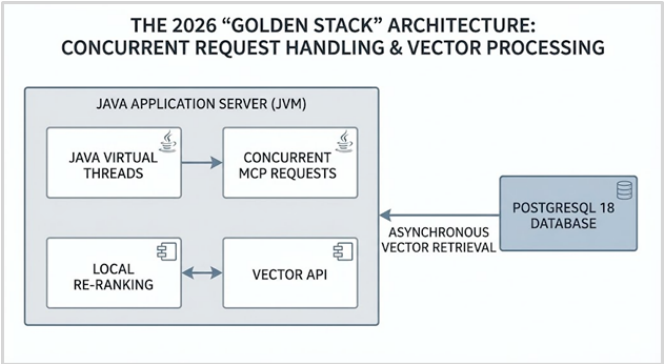


Figure 4: The 2026 "Golden Stack" Architecture. Java Virtual Threads handle concurrent MCP requests, the Vector API performs local re-ranking, and PostgreSQL 18 handles asynchronous vector retrieval.

Why Java 25 Makes MCP Practical

MCP's true power emerges when combined with Java 25's concurrency and computational improvements:

Concurrent Context Retrieval

A single LLM conversation may trigger dozens of parallel MCP tool calls (searching documentation, querying user history, fetching product catalogs). Virtual Threads allow these handlers to execute concurrently without the overhead of thread pools or reactive complexity. Each MCP request handler is a simple, blocking function call, yet the backend effortlessly handles thousands in parallel.

In-Process Vector Operations

When an MCP request asks for "similar products" or "related documentation", the Java backend uses the Vector API to perform local re-ranking and semantic filtering before returning results. This eliminates the latency of shipping embeddings to microservices or external vector databases. The entire retrieval-to-response pipeline from PostgreSQL fetch to cosine similarity calculation to JSON serialization happens in-process at SIMD speeds.

Predictable Latency

Because Virtual Threads eliminates the blocking I/O penalty and the Vector API removes JNI overhead, MCP response times become predictable (typically <50ms for vector search + re-ranking). This predictability is critical for LLM applications, where context retrieval is on the critical path of every user interaction.

Without these Java 25 foundations, MCP servers would require the same complex async/await patterns or external service orchestration that plagued earlier architectures. Java 25 makes MCP feel like a native, simple code that scales.

Conclusion

The "Great Unification" is technical, not merely conceptual. By leveraging the SIMD lanes of the CPU via Java 25, the ring buffers of the Kernel via PostgreSQL 18, and the graph algorithms of HNSW, we are building systems that are orders of magnitude more efficient than the disjointed microservices of the early 2020s. For the engineering team, this means we can stop managing infrastructure complexity and start building intelligent, data-rich applications on a rock-solid foundation.

References & Further Reading

- OpenJDK JEP 444 & 508: Virtual Threads and Vector API specifications. (https://openjdk.org/jeps/444)
- PostgreSQL 18 Release Notes: Asynchronous I/O (io_uring) and pgvector performance improvements. (https://www.postgresql.org/about/news/postgresql-18-released-3142/)
- Malkov & Yashunin: Efficient and Robust Approximate Nearest Neighbor Search using HNSW. (https://arxiv.org/abs/1603.09320)
- Spring AI: Model Context Protocol (MCP) integration documentation. (https://docs.spring.io/spring-ai/reference/api/mcp/mcp-overview.html)

Introduction to Agentic AI and building your own agent

Joswitha Palipe, Associate Engineer - Case 360

Introduction

The next frontier in artificial intelligence is represented by agentic AI. Unlike classical AI systems—which require continuous human guidance—agentic AI systems can plan, act, and adapt autonomously. This shift transforms AI from a reactive tool into a proactive agent.

Let's dive in to understand what agentic AI truly is, how it works, and why it's reshaping the future of artificial intelligence

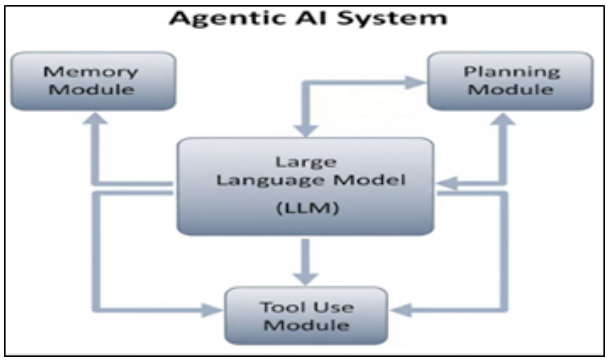
What is Agentic AI and why is it important?

"Agentic AI" refers to intelligent systems that operate independently, creating plans, establishing objectives, and using tools to carry out tasks. These systems combine feedback mechanisms and memory management to continually learn and adapt, accomplishing challenging goals without human involvement at every stage.

- There is a need to go beyond reactive chatbots to goal-oriented agents that can automate complex operations and produce quantifiable economic impact, because despite the widespread adoption of generative AI, many businesses still struggle to achieve meaningful returns.
- The market for autonomous agents is projected to grow from $4.35 billion in 2025 to $103.28 billion by 2034

How does agentic AI differ from traditional and generative AI?

- Traditional, generative, and agentic AI differ in autonomy and purpose—traditional AI follows set rules, generative AI produces content, and agentic AI executes actions.



Architecture of Agentic AI

How does agentic AI work step by step?

Agentic AI systems follow a Perceive -> Reason -> Act -> Learn (PRAL) cycle: data collection, planning and decision-making, task execution via tools or APIs, and feedback-based improvement.

Perceive

Agents collect data from multiple sources, including databases, external APIs, sensors, and user prompts. To recognize entities and extract significant patterns, they employ perception modules.

1. Reason

A reasoning engine (typically an LLM with RAG) interprets the goal and plans the steps to achieve it. It sequences activities, selects the right tools, and assesses trade offs. Reinforcement learning can refine decision-making over time.

2. Act

Once a plan is ready, the agent uses software, API calls, code execution, or physical device control to execute tasks. Built-in guardrails ensure compliance and safety. For example, a finance agent may auto-approve reimbursements up to a limit and flag larger ones for human approval.

3. Learn

Agents maintain a feedback loop. They gather the results of their actions, evaluate the outcomes, and refine their models to improve performance. This ongoing learning process creates a data flywheel—the more interactions they have, the smarter the agent becomes.

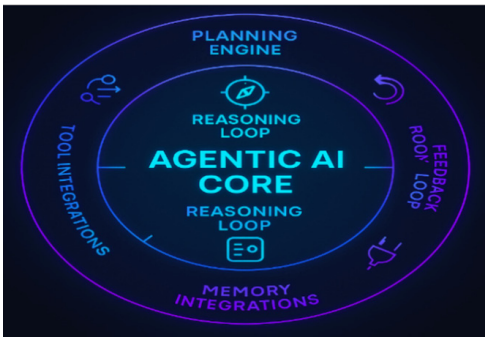
Multi-Agent Coordination

In complex workflows, a management agent orchestrates several specialized sub-agents. For instance, one agent retrieves data, another reasons over it, and a third one executes the required actions.

Categories of AI agents

Agentic systems typically fall into three major categories: proactive, reactive, and specialized (information retrieval, prescriptive knowledge,

Feature	Traditional AI	Generative AI	Agentic AI
Primary Function	Automating repetitive tasks	Generating text, code, and images	Goal-oriented decision-making and action
Autonomy	Low	Variable	High
Learning Style	Based on static algorithms	Data-driven (deep learning)	Reinforced learning with feedback and environmental adaptation
Scope	Limited, narrow domains	Content creation	Cross-domain reasoning and multi-step task execution



workflow action, and user helper). From coordinating intricate workflows to reacting to quick cues, each category has a distinct function

1. Reactive Agents

Current stimuli determine how reactive agents function. For example, a reactive agent in cybersecurity quickly isolates a compromised endpoint after spotting unusual activity.

2. Proactive Agents

Proactive agents foresee needs and establish objectives. For example, without waiting for directions, a marketing professional may optimize channels, adjust budgets, and track campaign results.

3. Specialized Agents

- Information Retrieval Agents: With more adaptability in managing a variety of information, these agents extract and synthesize knowledge from large databases for research, customer service, and knowledge management.
- Prescriptive Knowledge Agents: Work in regulated businesses by offering answers that are compliant and conform to guidelines; this is crucial in industry requirements.

Build your own Agent

Let's create an AI-powered research agent capable of fetching current information online, querying Wikipedia, organizing and maintaining research results, producing machine-readable study reports, and selecting the appropriate tools based on the query.

1. Technologies & Frameworks

- LangChain - The core orchestration framework that manages the agent, handles tool-calling, and coordinates interactions between components
- Anthropic Claude - The advanced Large Language Model (3.5 Sonnet) that powers the reasoning and decision-making capabilities
- Pydantic - Ensures structured data validation and type-safe output formatting

2. Prerequisites

- Python 3.8 or higher
- pip (Python package manager)
- API Credentials

  Anthropic API Key (required – Claude LLM)

  OpenAI API Key (optional – future extensions)

  Get your Anthropic API key here https://console.anthropic.com/

3. Installation and Setup

Create a project directory:

```
mkdir PythonAIAgentFromScratch
cd PythonAIAgentFromScratch
```



Create `requirements.txt`:

```
Langchain,langchaincommunity,langchainope
nai,langchainanthropic,Wikipedia,duckduc
kgo-search,python-dotenv,pydantic
```

Install all dependencies:

```
pip install -r requirements.txt
```

4. Environment Configuration

Create a `sample.env` file:

```
OPENAI_API_KEY=""
ANTHROPIC_API_KEY="your-anthropic-api-key-here"
```

5. Agent Architecture

LLM Configuration

```
llm = ChatAnthropic(model="claude-3-5-
sonnet-20241022")
```

6. Execution Flow

- User Query
- Claude LLM (tool-aware)
- Tool Selection and Execution
- Information Synthesis
- Pydantic Validation
- Structured JSON Output

7. Running the Agent

Execution

```
python main. Py
```

The agent will generate and save the text file.

Conclusion

Through a continuous cycle of Perceive-Reason-Act-Learn, agentic AI represents a significant transition from reactive tools to autonomous decision-makers capable of planning, acting, and learning. Organizations that implement agentic systems early will benefit from automated multi-step workflows, scale decision-making, and continuously improve through feedback loops.

As the industry is expected to grow from $4.35 billion (2025) to $103.28 billion (2034), businesses looking for quantifiable ROI must use agentic frameworks such as LangChain with LLMs as we go from AI as a tool to AI as a teammate. The future belongs to organizations where agents handle routine complexity while humans focus on strategic oversight and creative problem-solving.

References and Further Reading:

- LangChain Documentation
- Anthropic Academy: Claude API Development Guide
- https://github.com/masamasa59/ai-agent-papers/blob/main/agent-frameworks/agent-framework.md#single-agents

Agile in the Age of AI: Redefining Scrum for Intelligent Development

John Divakar, Engineering Manager - Collaboration Product Group

Introduction

AI adoption in software development is more than a tooling change—it fundamentally reshapes how Scrum teams plan, build, and validate software. As AI becomes part of daily development workflows, traditional Scrum practices require thoughtful adaptation.

This article explores how AI is reshaping Scrum ceremonies, team dynamics, feedback loops, and the metrics used to measure success.

Impact of AI during Scrum Ceremonies

Sprint Planning

Sprint planning has traditionally relied on historical velocity and team intuition to determine capacity.

AI introduces a variable that disrupts these calculations. As AI tools can generate boilerplate code, write unit tests, or scaffold entire components in minutes, the relationship between story points and time becomes less predictable. Teams now estimate both human effort and AI assisted work..

AI based forecasting tools can also analyze past sprint data to predict capacity and flag dependencies, which humans might overlook.



Daily Scrum

The daily standup's purpose is synchronization and impediment identification.

AI Tools can now monitor your commits in Gitlab, track tasks in Value Edge, pipeline failures, and create dashboards for synchronization and impediment identification. Standups increasingly focus on higher-level discussions such as architectural decisions, integration challenges, and the quality assurance of AI-generated outputs. Standups have shifted from "what did I complete yesterday" towards "what did I validate and refine yesterday". There's also a growing need to discuss AI-related blockers, such as when generated code doesn't meet security standards.

Sprint Review

Demonstrations during sprint reviews take on new dimensions when AI contributes to deliverables.

Stakeholders increasingly want visibility into how AI contributed. Discussions now include reliability, security, and intellectual property implications when AI generated code is used.

Sprint Retrospective

AI turns retrospectives from subjective discussions to evidence-based investigations.

Commit pattern analysis, sentiment analysis of code comments, analysis of various metrics provides deeper insight into what is really happening that humans never noticed.

Retrospectives have become forums for examining the team's evolving relationship with AI tools includes discussions such as:

- Evaluating which AI integrations proved valuable
- Iidentifying where AI-generated outputs required excessive correction
- Sharing techniques for more effective human-AI collaboration.

Impact of AI on Scrum Teams

The roles and dynamics of the Scrum team members are also evolving under the influence of AI.

Product Owners

AI can assist Product Owners by analyzing market trends, user feedback, and competitive landscapes, providing data-driven insights to inform product backlog prioritization and roadmap development. This allows Product Owners to make more informed decisions and focus on strategic vision.

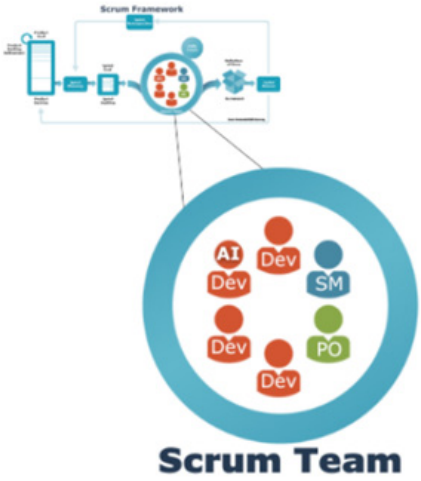
Scrum Masters

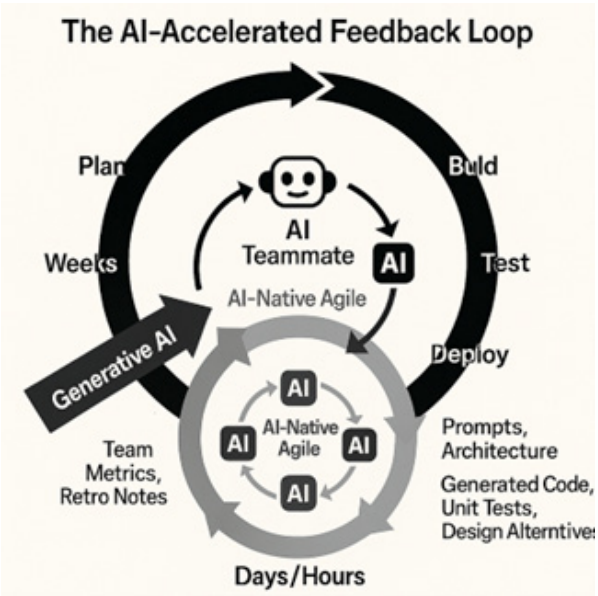
AI tools can automate routine administrative tasks, freeing up Scrum Masters to focus on removing impediments and fostering a healthy team environment by focusing on coaching.

Scrum Team

Teams can leverage AI for code generation, bug detection, automated testing, and intelligent code refactoring. This can allow developers to focus on solving complex problems and innovating. The adoption of AI tools within development teams necessitates new skill sets such as the following:

- Systems Architecture: The art of sdesigning robust modular systems that provide clear boundaries for AI to work with.
- Prompt Engineering: The craft of skillfully guiding AI to produce high-quality and accurate output.
- Critical Review: The non-negotiable discipline of meticulously validating all AI-generated code for quality, security, and correctness.





The AI-Accelerated Feedback Loop

One of AI's most profound impacts on Agile development is the compression of feedback cycles that once defined sprint rhythms.

From Sprint Cycles to Continuous Validation

Traditional Scrum assumes that meaningful feedback requires completed increments delivered over sprint-length intervals. AI challenges this assumption by enabling rapid prototyping and iteration within hours rather than days. A concept that once required a full sprint to prototype can now be created inhours, shared with stakeholders, and refined based on feedback before the original sprint would reach its midpoint. This acceleration doesn't eliminate the need for sprints, but it changes what teams can accomplish and how they structure their work.

Mechanisms that accelerate feedback

- Automated artifact generation: AI creates code, tests, and documentson in minutes, enabling prototypes to be validated faster.
- Faster triage: AI-driven log analysis and anomaly detection flag high-priority defects immediately after deployment.
- Continuous learning: AI synthesizes telemetry, user feedback, and PR reviews into prioritized learning items for the backlog.

Balancing Speed with Sustainability

While AI can dramatically accelerate certain aspects of development, sustainable pace remains essential. The temptation to continuously increase velocity as AI capability grows must be balanced against team wellbeing, code quality, and technical debt management. Teams that chase maximum AI-enabled speed without attention to sustainability risk burnout, quality degradation, and technical debt, which eventually slows them down despite AI assistance.

New Metrics for the AI Era

Along with traditional metrics like velocity and burndown charts track, the following metrics also measure the impact of AI within scrum teams.

Cycle Time

The time from when work starts on a task to when it is completed. This is a pure measure of team efficiency.

Lead Time

The time from when an idea is conceived to when it delivers value in production. This measures the overall agility of the entire organization, from idea to impact.

User Story Classification

- AI Automated - No human manual efforts.
- Review & Integration (R&I) - Designed to estimate the human effort involved in collaborating with AI. It measures the work of the developer acting as a guide, validator, and integrator.
- Standard User - Human-led Tasks.

Human-AI Leverage Ratio (HAI-LR)

HAI-LR= Total Story Points Completed / Human Story Points (non-AI-assisted)

A ratio of 3 means AI is effectively tripling team capacity. Track this sprint-over-sprint to identify when AI assistance peaks or when teams need upskilling.

AI Hallucination Rate (AHR)

Percentage of AI-generated code/tests that are syntactically correct but semantically wrong. Measured via:

- Code review rejection rate of AI-suggested changes.
- Production bugs traceable to AI-generated logic.
- Test failures from AI-generated assertions.

Healthy teams maintain AHR below 15%. Higher rates indicate prompt quality issues or domain complexity beyond current AI capabilities.

Prompt Reuse Ratio (PRR)

PRR= Stories using existing prompt templates / Total stories

High PRR (>60%) indicates healthy prompt engineering practice and knowledge capture. Low PRR suggests redundant effort and knowledge silos.

Synthetic Test Coverage (STC)

Percentage of edge cases covered by AI-generated tests. Measured by mutation testing scores and production defect rates for edge-case scenarios.

These metrics reveal the quality dimension of AI assistance that velocity alone obscures. A team with modest velocity but high HAI-LR and low AHR is outperforming a "fast" team drowning in AI-induced technical debt.Conclusion

The future of Agile development is neither purely human nor AI-dominated. It is a collaboration that leverages the strengths of both: AI's capacity for rapid generation and pattern application combined with human judgment, creativity, and contextual understanding. Teams that develop skill in orchestrating this collaboration will thrive; those that either ignore AI or blindly trust it will struggle.

Ultimately, the same empirical, inspect-and-adapt mindset that defines Agile provides the path forward. There is no universal answer to how AI should transform Scrum practices, only the ongoing commitment to experiment, measure, learn, and improve. The commitment has always been at Agile's heart, and it remains the surest guide through this transformation.

References and Credits

Primary References

1. Isobe, Y. (2024). "Agile in the Age of AI: A Practitioner's Guide to Evolving Scrum." Medium. Available at: https://medium.com/@yujiisobe/agile-in-the-age-of-ai-a-practitioners-guide-to-evolving-scrum-a94966326571
2. This article served as a key reference for the sections on AI-Accelerated Feedback Loops, New Metrics for the AI Era, A New Framework for Estimating AI-Driven Work, and A Practical Playbook for AI-Native Scrum.

Foundational Works

1. Beck, K., et al. (2001). "Manifesto for Agile Software Development." Agile Alliance. Available at: https://agilemanifesto.org
2. Schwaber, K. & Sutherland, J. (2020). "The Scrum Guide." Scrum.org. Available at: https://scrumguides.org

Additional Context

1. Sutherland, J. (2014). Scrum: The Art of Doing Twice the Work in Half the Time. Crown Business.
2. Cohn, M. (2005). Agile Estimating and Planning. Prentice Hall.

GitHub Copilot in the Agile Sprint Cycle: A Framework for Maximizing Team Velocity and Quality

Lokesh Murthy Venkatesh, Principal Software Architect - Data Protector

Abstract

GitHub Copilot represents a significant productivity augmentation tool for sprint-based development teams, yet its effective integration requires deliberate workflow design and metric-driven measurement. This article examines empirical findings from real-world sprint implementations, providing actionable frameworks for software development teams seeking to optimize velocity, code quality, and developer satisfaction within two-week sprint cycles. Analysis of studies across enterprise organizations—including Thoughtworks, Harness, and Accenture—reveals 10–15% sustainable velocity improvements through structured adoption. This article synthesizes research and practitioner experience into a comprehensive sprint integration model.

Executive Summary: Sprint-Level Impact

Recent empirical research demonstrates that GitHub Copilot, when properly integrated into sprint workflows, delivers measurable but context-dependent productivity gains. A 2025 Thoughtworks case study observed a 15% increase in sprint velocity, rising from 20 to 23 user stories per sprint for a four-person team. Similarly, Harness analysis reported a 10.6% increase in pull request (PR) activity and a 3.5-hour reduction in cycle time.

However, the impact varies significantly by task type. While developers report 50–70% time savings on test generation and 30–40% on boilerplate code, complex business logic remains a human-centric endeavor. The primary challenge for modern sprint teams is not just installing the tool but adapting the Agile cycle to manage the increased volume of code without sacrificing quality or security.

The Five-Phase Integration Framework

To move from ad-hoc usage to a high-performing "AI-Native" sprint team, organizations should follow a structured five-phase rollout.

Phase 1: Pre-Sprint Foundation

Before activation, teams must establish baseline metrics to measure ROI. Key indicators include the following:

- Cycle Time: The duration from the first commit to PR merge.
- PR Throughput: The average number of pull requests completed per sprint.
- Acceptance Rate: The ratio of AI-generated suggestions accepted by the team.
- Developer Satisfaction: Qualitative data on cognitive load and "flow state".

Phase 2: The Introduction Sprint

A "soft launch" approach is recommended, often starting with a 30–40% volunteer group within a single team. During this phase, teams should appoint AI Champions—enthusiastic developers who can lead by example and establish shared prompt libraries. Planning sessions should initially reduce the sprint commitment by 10% to account for the tool's learning curve.

Phase 3: Measurement and Optimization

Once the tool is active, teams must categorize tasks to maximize ROI. The Copilot Task Matrix helps teams identify where AI adds the most value:

- High ROI (Automate): Boilerplate, unit tests, documentation, and data transformations.
- Moderate ROI (Augment): Refactoring legacy code and API integrations.
- Low ROI (Human-Only): Complex architectural decisions, domain-specific business logic, and security-critical pathways.

Phase 4: Sustainable Operations

Long-term success requires monitoring for "Review Bottlenecks". As code volume increases, the time spent in code review can also grow, potentially neutralizing velocity gains. Teams must rotate "AI lead" roles to ensure everyone maintains manual coding proficiency and avoid over-reliance on suggestions.

Phase 5: Scaling Across the Enterprise

After one team successfully adopts the tool, the framework scales through centralized wikis, standardized prompting guides, and organization-wide "Jam Sessions" to share wins and troubleshooting tips.

Quality, Security, and the "Review Paradox"

AI-generated code introduces a unique paradox: while code creation is faster, code review becomes more difficult. Research

indicates that AI-generated code can have a 41% higher churn rate and may introduce subtle security vulnerabilities if not carefully audited.

The Evolution of Code Review

Sprint teams must adapt their review process to account for AI-specific issues:

1. Hallucination Detection: Reviewing for the use of non-existent libraries or "naive" logic that ignores edge cases.
2. Architectural Alignment: Ensuring AI-suggested solutions adhere to established design patterns.
3. Security Scrutiny: Applying heightened reviews to detect insecure patterns copied from model training data.
4. Code Duplication and Re-use Verification: Verifying that Copilot has not reinvented existing internal utilities or logic, which can lead to boilerplate bloat and maintenance challenges.

To mitigate these risks, teams should integrate automated security scanning (SAST) and consider using Copilot's own PR review capabilities to catch low-level issues before a human review.

Planning and Backlog Management

Effective adaptation requires changing how teams size and estimate their work. The "Tiered Task Stratification" model allows Product Owners and Scrum Masters to adjust story point estimations based on AI suitability:

- Tier 1 (Scaffold Heavy): Tasks involving significant repetitive structure. These should see a 20-30% point reduction as AI handles the majority of the "heavy lifting".
- Tier 2 (Logic Balanced): Features with a mix of standard CRUD and custom logic. These typically see a 10% velocity improvement.
- Tier 3 (Discovery Focused): Research spikes or complex algorithmic work. These should remain at traditional point estimates to avoid over-committing.

Long-term Sustainability and Skill Preservation

A key concern in AI adoption is the "Junior Developer Trap." While junior engineers show the highest immediate productivity gains, they risk by passing the deep learning experience required to solve complex issues when AI fails.

Best Practices for Skill Preservation

- Copilot-Free Sprints: Periodically holding "manual only" sprints for junior developers to ensure they retain fundamental problem-solving skills.
- Pair Programming: Using AI during pair programming sessions to facilitate discussion about solution quality, not just acceptance.
- Strict Audit Responsibility: Reinforce that developer, not the AI, is 100% accountable for the committed code.

Conclusion

GitHub Copilot is not a replacement for engineering expertise but a powerful "velocity multiplier". Success in an Agile environment depends on moving beyond simple license deployment towards a cultural and procedural shift. By implementing structured task stratification, evolving the code review process, and focusing on sustainable developer growth, sprint teams can achieve double-digit velocity improvements while maintaining enterprise-grade quality standards. The future of Agile development belongs to the AI-Augmented team— one that blends machine efficiency with human oversight.

References

1. Thoughtworks Case Study, 2025. Sprint velocity analysis of 4-person teams.
2. Harness SEI Analysis, 2025. Cycle time and PR throughput impact.
3. Microsoft Learn & GitHub Documentation. Adoption strategies and code review agents.
4. ACM CACM, 2024. "Measuring GitHub Copilot's Impact on Productivity".
5. Stray et al., 2025. NAV IT case study: Code review time impact.

Spec-Driven Development: When AI Meets Discipline

Vamsi Krishna, Distinguished Technologist - Portfolio BU

The Code That Wrote Itself (Almost)

Picture this: Your team just adopted GitHub Copilot. Developers are giddy with excitement, watching AI autocomplete entire functions. Within weeks, velocity soars—features ship 2x faster.

Success, right?

Not quite. Three months later, your codebase looks like a patchwork quilt sewn by a dozen different people who never talked to each other. Code reviews take forever. Defects multiply. Technical debt increases.

Welcome to the paradox of AI-assisted development: AI amplifies everything—including chaos.

The Vibe Coding Trap

We've discovered a new anti-pattern in the age of AI: "vibe coding." It goes like this:

1. Developer reads a vague requirement
2. Opens GitHub Copilot
3. Types a quick prompt: "make a login function"
4. Accepts the first suggestion
5. Moves on

It feels productive. It feels fast. And it creates a disaster.

The 2025 DORA (DevOps Research and Assessment) report revealed something counterintuitive: teams with weak engineering practices saw worse outcomes with AI, not better. Organizations without clear processes experienced what researchers called "accelerated instability"—delivering broken code faster than ever before.

Meanwhile, teams with strong discipline—clear specifications, rigorous testing, code review standards—saw 94% productivity gains. Same AI tools. Radically different outcomes.

The lesson? AI doesn't replace discipline. It demands it.

Enter Spec-Driven Development

Here's the philosophy shift that changes everything:

Stop treating AI as autocomplete.

Start treating it as an executor of specifications.

Traditional development looked like this:

- Developer reads requirements -> writes code -> maybe writes tests

Spec-driven development flips it:

- Write specification -> write tests -> let AI implement -> review and refine

Think of it like this: When you hire a contractor to build your house, you don't say "build me a house" and hope for the best. You give them architectural drawings, building codes, and inspection criteria. They execute. You verify.

Same principle with AI.

The Three Pillars

Pillar 1: Specifications Are Machine-Readable Contracts

A specification isn't documentation that gathers dust in Confluence. It's a precise description of what needs to be built—not how to build it. It captures requirements, constraints, edge cases, and acceptance criteria in a format both humans and AI can understand.

Example: Instead of "users should be able to log in," write:

Authentication Specification:

- Endpoint: POST /api/auth/login
- Input: username (email format), password (min 8 chars)
- Success: Returns JWT token, 200 status
- Failure: Invalid credentials -> 401, rate limited -> 429
- Security: bcrypt hashing, 5 attempt limit per 15 min
- Test: Must handle concurrent requests, expired tokens

This clarity transforms AI from a wild card into a soldier executing orders.

Pillar 2: Tests Are Safety Nets

Test-Driven Development (TDD) isn't optional with AI—it's essential. Write tests before implementation. Then use AI to generate code that passes those tests.

Why? Because AI is confident even when wrong. It will cheerfully generate broken code with a smile. Tests catch the lies.

The cycle becomes:

- Red: Write failing tests from your spec

- Green: Prompt AI to implement code that passes tests
- Blue: Refactor with AI while tests stay green

Martin Fowler, the godfather of modern software practices, puts it simply: "TDD isn't just for testing—it's how you think with code. Now, let AI implement your thinking."

Pillar 3: Human-AI Partnership, Not Replacement

The best outcomes happen when roles are clear:

Humans own:

- Problem framing and architectural decisions
- Writing specifications and acceptance criteria
- Reviewing AI-generated code critically
- Making the final call on what ships

AI owns:

- Translating specs into working code
- Generating comprehensive test cases
- Suggesting refactoring improvements
- Finding security vulnerabilities

This isn't about AI replacing jobs. It's about AI handling the mechanical, repetitive parts so humans can focus on the creative, strategic work that actually matters.

What This Looks Like in Practice

Let's make this concrete. You're building a payment processing feature.

Old Way (Vibe Coding):

- Dev: "Copilot, write a function to process credit card payments"
- Copilot generates 200 lines of plausible-looking code
- Dev skims it, looks good, commits
- Production: SQL injection vulnerability, no input validation, hardcoded API keys
- Time saved: 2 hours. Cost: 1 security breach.

Spec-Driven Way:

1. Write the spec (30 min):
   - Payment types supported
   - Validation rules
   - Error handling
   - Security requirements
   - Performance targets
2. Generate tests from spec (20 min):
   - Use AI: "Generate test cases for this spec covering happy path, failures, and edge cases"
   - Review and enhance tests
3. Implement with AI (40 min):
   - Prompt: "Implement this function to pass these tests, following this spec"
   - AI generates code
   - Tests verify it works
4. Security review (15 min):
   - AI scans for vulnerabilities
   - Fix issues before they ship
5. Refactor (15 min):
   - AI suggests improvements
   - Tests ensure nothing breaks

Total time: 2 hours.

Outcome: Comprehensive tests, security review, documentation, and code that actually works.

The Surprising Benefits

Teams adopting spec-driven development report outcomes that seem too good to be true:

30-40% faster delivery (even accounting for spec writing time). Why? Because you eliminate rework. "Measure twice, cut once" applies to software.

Dramatically better code reviews. Reviewers can check: "Does this match the spec? Do tests prove it?" Instead of: "What is this even supposed to do?"

Onboarding in days, not weeks. New developers read the spec, understand the intent, and can contribute immediately.

AI that gets smarter over time. As you build a library of specs and prompts, the AI learns your team's patterns and produces better initial suggestions.

The Gotchas (Learn From Our Mistakes)

Gotcha #1: Over-specifying

Don't specify every implementation detail. Specify what and why, not how. Let AI surprise you with creative solutions.

Gotcha #2: Treating specs as waterfall

Specs evolve. Write rough specs early, refine as you learn. This isn't 1995 waterfall; it's agile with better artifacts.

Gotcha #3: Skipping the review

AI-generated code still needs human review. Read it. Understand it. Make sure it makes sense. You own what ships.

Gotcha #4: No tests = no safety

If you're using AI without tests, you're playing Russian roulette with your codebase. Don't.

The Philosophical Shift

Here's what's really happening: Code is no longer the primary artifact of software engineering.

The real artifacts are:

- Specifications (what we're building and why)
- Tests (how we verify it works)
- Architecture decisions (the big choices that matter)

Code is what happens when you feed those artifacts to an AI executor.

This isn't radical. It's the culmination of 50 years of abstraction. Assembly -> C -> Java -> Python -> Specifications. Each layer lets us think at a higher level.

The skill isn't typing syntax faster. It's thinking clearly about problems, specifying precisely what solutions look like, and validating rigorously that they work.

A Closing Thought

In 1920, most workers were farmers. Tractors didn't eliminate farming—they eliminated manual plowing. Farmers became operators of sophisticated machinery, focused on crop planning, soil management, and market strategy.

In 2026, AI doesn't eliminate programming—it eliminates manual code writing. Developers become orchestrators of intelligent systems, focused on problem framing, architecture, and validation.

The developers who thrive won't be those who type the fastest. They'll be those who think the clearest.

So, the next time Copilot suggests a function, ask yourself: Do I have a spec that defines what this should do? Do I have tests that prove it works?

If not, you're not doing AI-assisted development. You're vibe coding with a very confident assistant.

The Green Code Effect: Architecting for Efficiency in the GenAI Era

Chandru Karunanithi, Lead QA Engineer - Filr

Abstract

As organizations increasingly adopt Generative AI assistants to accelerate development, the volume of code entering our repositories is growing at an unprecedented rate. While these tools boost productivity, they introduce a significant hidden risk: "software bloat" and unoptimized logic that silently drive-up cloud infrastructure costs. This article explores the critical intersection of Sustainable Software Engineering (SSE) and high-performance architecture. We examine how treating energy efficiency as a core quality metric—alongside reliability and security—serves as a powerful guardrail against technical debt. By focusing on "Green Coding" principles, optimizing resource utilization, and refining AI-generated logic, engineering teams can simultaneously reduce their carbon footprint and achieve significant cost savings. In the modern tech landscape, the most sustainable code is also the most profitable.

Introduction: The "Speed vs. Waste" Paradox

We are living in the "AI-Accelerated Development Era". With the widespread adoption of AI-driven coding assistants (like GitHub Copilot), the barrier to producing code has never been lower. We often measure success by velocity: features shipped per sprint and lines of code produced per day.

However, there is a catch. AI tools prioritize correctness (getting the right answer) over efficiency (getting there with the least compute). Recent research highlights a concerning trend:

- The "Churn" Problem: A 2025 study by GitClear analyzing over 211 million lines of code found that "Code Churn" (lines written then immediately rewritten or deleted) is projected to double from pre-AI baselines, increasing from 3.3% in 2021 to 7.1% by 2024—a trend correlated with widespread AI coding assistant adoption.
- The Security Gap: Veracode's 2025 GenAI Code Security Report found that in 45% of test cases, AI-generated code introduced OWASP Top 10 vulnerabilities—including SQL injection, cross-site scripting, and insecure authentication patterns—requiring human security review and remediation.

While these studies reveal concerning trends correlating AI tool adoption with code quality metrics, correlation does not prove causation. Multiple factors—including rapid developer onboarding, increased development velocity, and evolving best practices—may contribute to these patterns. The key insight is that AI tools require human oversight to ensure efficiency and security, regardless of the underlying causes.

When this "heavy," unoptimized code is deployed to production and executed millions of times, it creates a compounding tax on our infrastructure. This is where Sustainable Software Engineering enters the picture. It is not just about environmental responsibility; it is a discipline of Digital Efficiency.

FinOps Meets GreenOps: The Business Case

To understand why sustainable engineering matters to the bottom line, we must look at the convergence of two operational models: FinOps (Financial Operations) and GreenOps.

In the cloud, carbon is a proxy for cost.

- Provisioning: Spinning up larger instances to handle inefficient code directly increases the bill.
- Inference Costs: A single query to a Large Language Model (LLM) consumes approximately 10x more energy than a standard web search, with recent studies showing LLM queries requiring ~0.3 Wh compared to ~0.0003 Wh for traditional search engines.
- Data Transfer: Moving uncompressed or redundant data across regions spikes both network costs and energy usage.

By adopting a "Green" mindset, we inevitably optimize for cost. Industry experience and the Green Software Foundation's framework research indicate that carbon-aware optimizations typically correlate with reduced infrastructure costs—cleaner code inherently uses fewer computational resources. Sustainability is simply FinOps in disguise.

The Technical Core: Algorithmic Efficiency

The core responsibility of the "AI-Augmented Engineer" is shifting. We no longer just type syntax; we act as architects who review and optimize the logic provided by our tools.

Consider Algorithmic Efficiency. AI models often generate "brute force" solutions. A nested loop ($O(n^2)$) might pass a unit test, but it consumes exponentially more energy than a hash map approach ($O(n)$) as data scales.

Case Study: The Hidden Cost of "Convenience" Code

Let's look at a common Java scenario often suggested by AI when asked to "process a list of strings."

The "Standard" AI Suggestion (Inefficient):

The AI suggests using simple String concatenation inside a loop. It looks clean but creates a new String object in memory for every iteration, triggering frequent Garbage Collection (GC) cycles.

Java

```
// Inefficient: High Memory Churn & CPU Load
public String processTransactions(List<String>
transactions) {
    String result = "";
    for (String txn : transactions) {
        if (isValid(txn)) {
            // Creates a new String object in Heap
memory every single time
            result += txn + ",";
        }
    }
    return result;
}
```

The "Green" Refactor (Optimized):

The engineer recognizes the waste and refactors using StringBuilder and stream sizing.

Java

```
// Optimized: Low Memory Footprint & Faster
Execution
public String processTransactions(List<String>
transactions) {
    // Pre-sizing prevents array resizing overhead
   // Assumes ~20 chars per transaction (adjust
via profiling for your data)
    StringBuilder sb = new
StringBuilder(transactions.size() * 20);
    for (String txn : transactions) {
        if (isValid(txn)) {
            sb.append(txn).append(",");
        }
    }
    return sb.toString();
}
```

The Impact:

While the difference seems trivial in a test environment, in a high-throughput microservice processing 50,000 requests per minute, the first version triggers massive CPU spikes due to Garbage Collection. The "Green" version keeps the CPU idle longer, allowing the underlying hardware to enter low-power states (a concept known as "Race to Idle").

Strategic Framework: Applying Green Software Patterns

To systematize this, we can adopt the Green Software Foundation (GSF) patterns. Here is how they apply to our daily development:

1. Carbon Efficiency (Minimize Emissions)
   - The Stats: Cloud regions have vastly different "Carbon Intensity" (gCO_2/kWh). Running a batch job in a coal-heavy region (e.g., ~820-995 gCO_2/kWh) can emit 20-40x more carbon than in a hydro-powered region (e.g., ~24 gCO_2/kWh).
   - Dev Action: Use simple feature flags to delay non-urgent background processing windows to times when the grid is cleaner (Carbon-Aware Computing).
2. Energy Efficiency (Minimize Consumption)
   - The Stats: "Bloatware" imports are a rising issue. AI assistants often import heavy libraries for simple utility tasks (e.g., importing a 5MB library just to check StringUtils.isEmpty).
   - Dev Action: Audit pom.xml or package.json files. If you only need one function, write it yourself rather than loading a massive library into memory.
3. Hardware Efficiency (Maximize Utilization)
   - The Stats: Servers running at low utilization (10-20%) are wasteful because the "embodied carbon" (the manufacturing cost of the hardware) is not being justified by the work output.
   - Dev Action: Embrace Containerization and Serverless functions. These technologies bin-pack processes tightly, ensuring we get maximum value out of every server cycle.

Conclusion: The Mindful Engineer

In the past, a "Senior Engineer" was defined by their ability to solve complex problems. In the GenAI era, seniority is defined by efficiency.

As we leverage AI to write code faster, our role must evolve into guardians of quality. We must ensure that our digital footprint doesn't outpace the value we deliver. By asking simple questions during code reviews—"Is this loop necessary?", "Can this payload be smaller?", "Do we need this library?"—we build systems that are faster, cheaper, and sustainable.

Next Step for Our Team

To put this into practice, I propose we add a "Sustainability & Efficiency Check" to our standard Pull Request (PR) templates. This would include three simple checkboxes:

1. [ ] Bloat Check: Have unused libraries/imports been removed?
2. [ ] Loop Check: Are there any nested loops ($O(n^2)$) that can be optimized?
3. [ ] Data Check: Is the data being fetched/stored actually used, or is it "digital waste"?

AI generates the logic, but humans optimize the physics.

The Silent Technical Debt introduced by AI-Assisted Development

Akhil Rampally, Associate Engineer - Endpoint Management

Abstract

AI coding assistants promise dramatic productivity gains. Tasks that once took hours now take minutes, and boilerplate code can be generated almost instantly. However, this acceleration comes with a hidden cost. This article explores a subtle and largely invisible class of technical debt—AI-induced technical debt—that does not primarily reside in syntax or failing tests, but in architecture, decision-making, and human understanding.

This journal considers two engineers—Walt and Jesse—who represent contrasting ways of interacting with AI. Through their behaviours and outcomes, the article illustrates why keeping humans firmly in the loop is essential for building sustainable software systems in the era of AI-assisted development.

Introduction

The year 2023 marked a turning point in software engineering. Tools such as GitHub Copilot, ChatGPT, Claude, and similar AI-powered assistants moved from experimental curiosities to everyday development companions. By 2025, AI assistance has become routine for writing code, debugging issues, and exploring solutions.

The productivity gains are undeniable. Features ship faster, onboarding time is reduced, and developers spend less time writing repetitive code. Yet beneath this efficiency lies a quieter shift—one that rarely triggers alerts or breaks builds. Teams are beginning to encounter systems that work, pass tests, and deploy successfully, but are increasingly difficult to understand, evolve, or confidently modify.

This article argues that the issue is not AI itself, but how humans choose to work with it. It is important to note that AI does not remove cognitive responsibility; the human always remains accountable for correctness, design, and intent. To explore this, we follow two engineers: Walt and Jesse.

Walt and Jesse: Two Ways of Working with AI

Walt: The Deliberate Engineer

Walt uses AI as a powerful assistant, not an authority. When faced with a problem, he first frames it clearly—understanding the constraints, architecture, and trade-offs involved. He then uses AI to accelerate implementation, explore alternatives, or validate ideas.

When AI generates code, Walt reviews it critically. He asks himself:

- Do I understand why this works?
- Does this align with our system's architecture?
- What assumptions is this solution making?

For Walt, AI reduces manual effort, buteffort but never replaces reasoning. The result is faster development without sacrificing long-term maintainability.

Jesse: The Vibe Coder

Jesse represents a growing archetype in modern development: the vibe coder. Jesse prompts AI, copies the solution, runs the tests, and ships.

The code compiles. The tests pass. The feature works.

From the outside, Jesse looks extremely productive. But when asked why the solution works, or why a particular design was chosen, the answers are vague. Debugging becomes an exercise in re-prompting AI rather than reasoning through the problem.

Jesse does not intend to create bad systems. The debt accumulates quietly, hidden beneath green build pipelines.

What Is AI-Induced Technical Debt?

Traditional technical debt arises from rushed designs, shortcuts, or outdated decisions and is often visible through code smells, complexity metrics, or maintainability warnings.

AI-induced technical debt is different:

- It is largely invisible to standard metrics
- The code often looks clean and idiomatic
- The cost appears much later, during change or failure

Through Walt and Jesse, we can see how this debt manifests in three dimensions: cognitive debt, architectural debt, and knowledge debt.

Cognitive Debt:

In a traditional workflow, developers move through problem analysis, design, implementation, and testing. Jesse's workflow collapses this into a single step: prompt and accept.

Over time, Jesse stops building mental models of the code. For example, when AI generates a concurrent processing solution, Jesse accepts it without fully understanding its synchronization strategy. The system works—until a rare race condition appears in production.

Without a mental model, debugging turns reactive. Jesse responds by prompting AI again, deepening dependence and further eroding reasoning skills.

Walt, by contrast, uses AI to support his thinking. He understands the concurrency model before merging the code, making failures easier to diagnose and fix.

Architectural Debt: Local Correctness, Global Fragility

AI models excel at producing locally correct solutions. Jesse relies on this strength without providing architectural context.

In a microservices system, Jesse asks AI to optimize performance for a feature. The generated solution introduces cross-service calls that bypass established boundaries. Each change looks reasonable in isolation, but collectively they weaken the system's architecture.

Walt approaches the same task differently. His prompts include architectural constraints, and he rejects solutions that violate service boundaries—even if they appear efficient.

Over time, Jesse's codebase becomes fragile and resistant to change, while Walt's remains coherent and evolvable.

Knowledge Debt: When Understanding Leaves the Team

Perhaps the most dangerous form of AI-induced technical debt is the loss of shared understanding.

Jesse merges AI-generated solutions with minimal documentation. The reasoning behind decisions lives neither in code nor in design records—it lives nowhere.

When Jesse changes teams or leaves the organization, that undocumented reasoning leaves with him. The system still runs, but no one is confident modifying it.

Walt documents key decisions lightly but consistently. Future engineers understand not just what the system does, but why it was built that way.

Why Traditional Metrics Fail

Modern engineering teams rely on metrics such as:

- Build success rates
- Test coverage
- Deployment frequency
- Velocity

Both Walt's and Jesse's systems look healthy through these lenses. The difference emerges only when deep understanding is required—during outages, refactors, or major architectural changes.

AI-induced technical debt remains invisible until it becomes expensive.

Impact on Teams and Career Growth

Jesse ships features rapidly but struggles with complex debugging and system design discussions. His growth plateaus as AI becomes a cognitive crutch.

Walt grows differently. By staying deeply involved in reasoning and design, he uses AI to amplify expertise rather than replace it. Over time, Walt becomes a stronger engineer, not just a faster one.

Mitigation Strategies: Be Walt, Not Jesse

AI is not the problem—unexamined usage is. Sustainable teams adopt practices that reinforce human ownership:

- Architecture-first prompting: Prompts should include architectural constraints and design principles, not just functional requirements
- Human-owned design decisions: AI can assist with implementation, but system design and trade-off decisions should remain human-led
- Mandatory documentation: Key decisions and reasoning should be captured in simple, human-readable documentation
- Regular architecture reviews: Periodic reviews help detect drift early, before it hardens into expensive debt
- Continuous skill reinforcement: Organizations should continue investing in system design, debugging, and foundational engineering skills

These practices keep AI as an accelerator, not a substitute for engineering judgment.

Conclusion

AI-assisted development is one of the most transformative shifts in modern software engineering. Through Walt and Jesse, we see two futures.

One future ships fast but accumulates silent debt, losing understanding with every prompt. The other moves quickly and sustainably, with humans firmly in control of reasoning and design.

The most dangerous technical debt is not hidden in code; it is hidden in lost understanding. In the AI era, great engineers are those who remain accountable for the systems they build.

Engineering Resilience in Cloud-Native Systems with OpenShift

Haretha Varshini S, Software Engineer - Open Enterprise Server

Introduction

Today's applications demand systems that not only work but also withstand failure gracefully. As organizations shift from monolithic architectures to distributed, containerized microservices, failure is no longer an exception - it's part of the daily landscape. Services communicate across networks, scale independently, and deploy continuously, which means something, somewhere, will eventually break.

That's why resilience isn't a nice-to-have - it's essential. In day-to-day work with OpenShift, we've seen how a well-architected platform can turn potential outages into minor blips. OpenShift builds on Kubernetes with enterprise-grade tooling that helps teams build, deploy, and operate truly resilient cloud-native applications at scale.

In this article, we'll explore why resilience matters, how OpenShift supports it, and practical strategies my team uses to keep systems reliable under pressure.

OpenShift in the Kubernetes Ecosystem

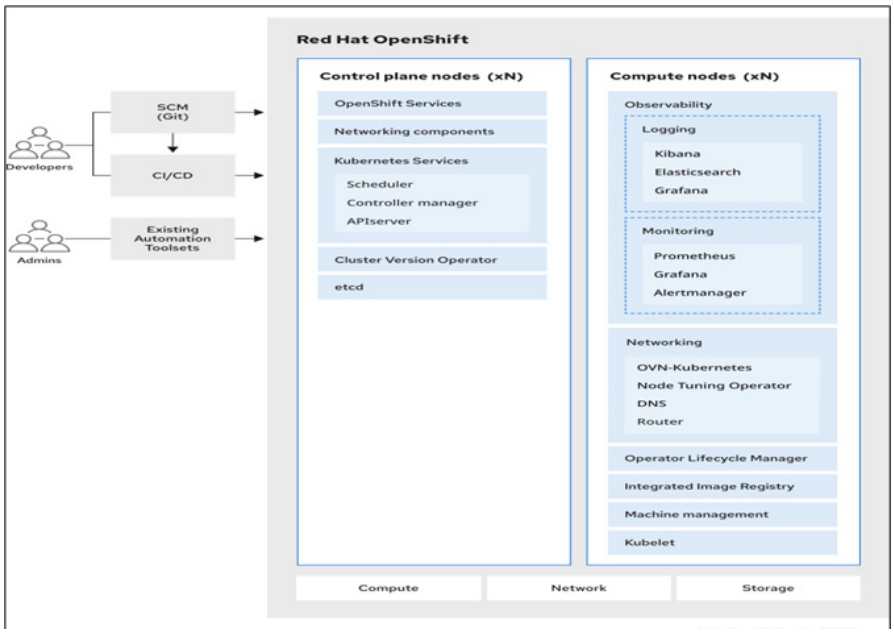
While several cloud solutions for Kubernetes are available in the market today, Red Hat OpenShift stands out for its emphasis on enterprise readiness, security, and hybrid cloud capabilities. Based on our practical experience, the following aspects are key:

- Support for both hybrid and on-premises architectures, making it ideal for regulated enterprise environments.
- Business-class security features such as RBAC, Security Context Constraints, and compliance.
- Opinionated defaults and Operators that reduce configuration errors and enhance platform stability.
- Unified monitoring, logging, and CI/CD solutions that simplify operations.
- A single, consistent experience across all data centers and cloud providers.
- Support for disconnected and air-gapped deployments.
- Consistent Kubernetes experience across data centers and public cloud providers.

OpenShift Platform Architecture Supporting Resilience

Why Resilience Can't Be an Afterthought

In a cloud-native setup, your application is a constellation of microservices. Each can be deployed, scaled, and updated



independently, which is great for velocity, but it also multiplies potential points of failure.

From experience, failures often arise from:

- Application crashes – memory leaks, unhandled exceptions, dependency timeouts.
- Container failures – image issues, runtime errors, scheduler conflicts.
- Node outages – VM failures, kernel panics, hardware issues.
- Network partitions – silent drops, latency spikes, DNS failures.

If you don't design for failure, a single glitch can cascade. We've been in war rooms where a minor defect in one service brought down three others due to tight coupling and poor fault isolation. Resilient design means expecting these failures, containing them, and enabling the system to self-stabilize without user impact.

How OpenShift Lays the Foundation for Resilience

Kubernetes orchestrates containers, but OpenShift adds opinionated defaults and integrated tools that enforce resilience from the ground up.

Here's what stands out in practice:

- Health checks that work – not just TCP pings, but meaningful readiness/liveness probes
- Self-healing at the pod level – automatic restarts and rescheduling
- Built-in RBAC and security policies – so access controls aren't bolted on later.
- Integrated CI/CD – consistent, auditable deployment pipelines
- Observability baked in – metrics, logs, and traces without third-party hacks.

OpenShift shifts the team's focus from infrastructure firefighting to application reliability - where it should be.

Designing for Failure: Patterns That Work

You must accept that things will break. The goal isn't to prevent every failure but to minimize the blast radius and recovery time.

We use OpenShift to implement patterns such as:

- Multi-replica services – no single point of failure.
- Pod anti-affinity rules – distribute workloads across nodes and zones.
- Resource limits and quotas – prevent noisy neighbours from starving others.
- Graceful degradation – fallback modes when dependencies fail.

These aren't theoretical; they've saved us during zone outages and deployment mishaps more than once.

Making Deployments Predictable with Helm

Deployments are a common source of instability. We've seen "simple config changes" cause hours of downtime because environment drift.

Helm gives us a reproducible, version-controlled way to package applications. Each Helm chart defines not just the containers, but also config maps, services, ingress rules -everything. That means:

- We can track exactly what changed and who changed it.
- Rollbacks are one command away.
- Upgrades are atomic and reversible.

Helm reduces manual intervention, minimizes human error, and turns deployment from a fragile art into a reliable engineering practice.

Seeing What's Happening: Monitoring & Observability

You can't fix what you can't see. Without observability, you're flying blind - and in distributed systems, that usually ends badly.

OpenShift integrates seamlessly with Prometheus, Grafana, and OpenTelemetry out of the box. We monitor:

- Resource usage trends (CPU, memory, I/O)
- Pod health and restart counts
- Latency percentiles and error rates
- Inter-service dependencies and traffic flow

Alerts trigger before users notice problems, and with distributed tracing, we can follow a request across multiple services to locate bottlenecks.

Self-Healing: Letting the Platform Do the Work

One of OpenShift's strongest features is its ability to detect and recover from failures automatically. If a pod crashes, it's recreated. If a node fails, workloads are rescheduled elsewhere.

This self-healing capability results in:

- Lower MTTR – the system fixes itself before a ticket is filed
- Reduced toil – fewer manual restarts
- More time for engineering – teams can investigate root causes and improve design

Automation handles the repetitive work; engineers focus on analysis and optimization.

Challenges and Considerations

An OpenShift is powerful, but it's not magic. Building resilience still requires deliberate effort:

- Alert fatigue – tuning monitoring to notify without overwhelming.
- Cluster sprawl – managing consistency across dozens of namespaces and teams.
- Skill gaps – not everyone understands distributed systems patterns immediately.
- Cost vs. resilience trade-offs – more replicas, more zones, more spend.

We've learned that resilience isn't a checkbox; it's a continuous investment in design, automation, and team capability.

Conclusion

Resilience in cloud-native systems starts with a platform that expects failure — and that's what OpenShift delivers. With a design-for-failure mindset, teams can build systems that withstand real-world chaos.

The goal isn't perfection — it's the ability to fail gracefully, recover quickly, and learn continuously. In the ever-evolving cloud-native landscape, that's what separates systems that survive from those that thrive.

Further Reading

- Red Hat OpenShift Documentation
- Helm – The Kubernetes Package Manager

Apache TLS Upgrade Deprecation Impact

Shivani Srivastava, Lead Software Engineer - File and Print Services, OES

Abstract

Recent Apache HTTP Server updates have removed support for opportunistic TLS upgrades, a mechanism some older macOS CUPS/IPP clients relied on to switch from HTTP to HTTPS during an active connection. Because these clients expect servers to honor RFC 2817–style upgrades or react to a 426 Upgrade Required response by upgrading on the same socket, secure printing workflows can now fail on servers with modern security configurations.

This document explains why these failures occur, outlines the typical symptoms observed in macOS/IPP print workflows, and demonstrates how the deprecated upgrade path breaks older assumptions in client implementations. It then presents a robust fix: closing the HTTP connection when a 426 is received and reconnecting explicitly over HTTPS, with correctly reset IPP request state and authentication handling.

The article concludes with long term recommendations for administrators and developers to transition fully to IPP over HTTPS (ipps), aligning macOS printing behavior with current Apache security practices and industry best practice TLS usage.

1. Introduction

CUPS is the standard printing system on macOS and many Unix-based systems. It communicates with print servers using Internet Printing Protocol (IPP) over HTTP or HTTPS. Historically, some deployments allowed clients to start over plain HTTP (port 631) and then upgrade the same connection to TLS when needed. Modern Apache releases and security hardening trends have deprecated these upgrade paths, requiring clients to connect directly via HTTPS for better security.

This shift impacts environments where older macOS or CUPS clients still rely on legacy upgrade behaviour. IT administrators managing print infrastructure and developers maintaining print services should be aware of these changes to ensure reliable, secure printing workflows.

2. Background Concepts

2.1 IPP over HTTP vs HTTPS

IPP over HTTP (typically on port 631) is common for initial printer discovery and unauthenticated requests. IPP over HTTPS (commonly on ports 443 or 631) is required when authentication or encryption is needed. The IPP transport binding over HTTPS is standardized, and the ipps URI scheme is widely supported.

2.2 TLS Upgrade (RFC 2817)

RFC 2817 describes using the HTTP/1.1 Upgrade mechanism to initiate TLS over an existing TCP connection. In practice, mainstream clients and servers rarely rely on it today, and many proxies or middleboxes reject or ignore such upgrades. Historically, CUPS supported an opportunistic “HTTP Upgrade to TLS” mode for secure administration and printing.

2.3 What Changed in Apache

Security advisories and recent Apache releases discourage configurations such as “SSLEngine optional”, which enable plaintext-to-TLS upgrades. Servers expect clients to use plain HTTP only or to connect directly using HTTPS. As a result, servers may support TLS but no longer honor TLS upgrade on the same socket.

3. Impact on Clients

When macOS IPP clients attempt to upgrade an existing HTTP connection to TLS—a behaviour no longer supported by modern Apache servers—printing workflows can fail. Because the client expects a TLS upgrade that the server now rejects, the connection and request state become inconsistent.

Real World Effects

- Failed or stalled print jobs when the IPP request cannot be upgraded or resent correctly.
- Repeated credential prompts due to broken authentication flows after a failed upgrade.
- 400/401 errors caused by corrupted or partially retransmitted IPP requests.
- Unexpected disconnections because Apache refuses to reuse the original HTTP socket.

4. Reproduction Setup (for Reviewers)

- Server: Apache HTTP Server >= 2.4.64 with TLS enabled; avoid any optional TLS upgrade settings.
- Client: macOS CUPS (IPP) client configured to start with HTTP and expect upgrade on 426.

- Send an IPP POST over HTTP to the print queue; observe 426 Upgrade Required.
- Client retries upgrade on the same socket; observe failure (400/401) and potential auth loop.

5. Solution Approach

5.1 Key Design Change

- Do not retry the same connection after receiving 426 Upgrade Required, do not retry on the same connection.
- Close the HTTP socket immediately.
- Reconnect explicitly using HTTPS (port 443 or 631).

5.2 Required Client Fixes

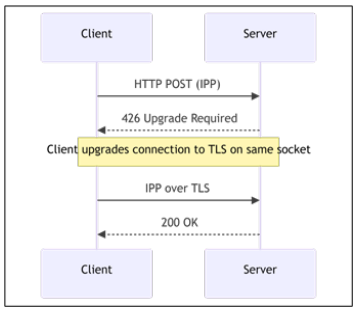
- Reset IPP request state before resending.
- Rewind file/document streams.
- Recalculate Content-Length.
- Allow authentication to occur naturally over HTTPS; avoid aggressive header reuse across connections.

5.3 Authentication Handling

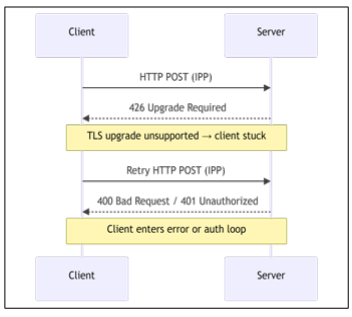
- Send the first HTTPS request without Authorization. Expect a 401 Unauthorized.
- Prompt for credentials and retry over the same HTTPS connection.
- Avoid reconnecting during authentication loops.

6. Sequence Diagrams

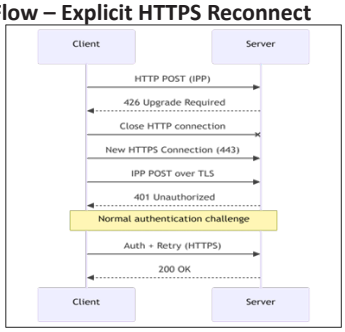
6.1 Legacy (Working) Flow – TLS Upgrade Supported



6.2 New (Failing) Flow – TLS Upgrade Removed



6.3 Corrected Flow – Explicit HTTPS Reconnect



7. Security Implications

Opportunistic TLS upgrades introduce several security weaknesses because the connection begins as plaintext and only later attempts to negotiate encryption. This transition phase creates ambiguity in how clients and servers interpret security state, making the upgrade path inherently fragile in modern network environments.

7.1 Key Risks

- Downgrade Attacks: Attackers positioned on the network can force the client to remain on plaintext by stripping or modifying the Upgrade request/response, preventing the transition to TLS.
- Man in the Middle (MITM) Exposure: Any data exchanged before the TLS handshake—such as HTTP headers—can be intercepted or manipulated. Middleboxes may also terminate or mis-handle upgrade attempts, further increasing attack surface.
- Protocol and State Inconsistency: Because HTTP/1.1 was not widely implemented with robust upgrade in place semantics, many intermediaries reject or alter these requests, causing broken security assumptions and unpredictable behaviour.

7.2 Compliance Considerations

Modern security frameworks discourage or prohibit mixed mode (plaintext -> TLS) negotiation:

- PCI DSS and similar industry standards require strong encryption at the start of the session and do not permit sensitive operations to occur on partially encrypted or opportunistically encrypted channels.
- Enterprise security baselines generally mandate enforceable TLS only endpoints, certificate validation, and clear TLS initiation boundaries—requirements incompatible with opportunistic upgrades.
- Zero trust network models assume all traffic must be encrypted by default and do not allow the client or server to depend on opportunistic protection.

7.3 Why Apache Enforces This

Apache's removal of TLS upgrade capabilities aligns the server with current security best practices: forcing clients to establish a dedicated HTTPS connection eliminates downgrade windows, reduces MITM exposure, and simplifies compliance with industry encryption standards. This ensures consistent and predictable security behaviour across all clients and intermediaries.

8. Long-Term Recommendations

- Avoid relying on TLS upgrade entirely; use ipps (IPP over HTTPS).
- Configure clients to use https:// or ipps:// URLs directly.
- Use port 443 (or 631 with HTTPS) when security is required.
- Validate server certificates and align with enterprise PKI policies.

References

- RFC 2817: Upgrading to TLS Within HTTP/1.1 (Khare, Lawrence, 2000).
- MDN Web Docs: HTTP 426 Upgrade Required (Status Code).
- RFC 7472: IPP over HTTPS Transport Binding and the "ipps" URI Scheme (McDonald, Sweet, 2015).
- RFC 8010: IPP/1.1 Encoding and Transport (Sweet, McDonald, 2017).
- CUPS Documentation: Managing Encryption (opportunistic HTTP Upgrade to TLS).
- Apache HTTP Server Security Advisories / Release Notes discussing deprecations related to TLS upgrade configurations (for example, SSLEngine optional).

Beyond the Basics: XSS Prevention strategies for custom frameworks

Noor Mohammed Rizwan, Lead Software Engineer - Endpoint Management

Introduction

Cross-Site Scripting (XSS) remains one of the most pervasive and dangerous vulnerabilities in web security. At its core, XSS occurs when a browser renders user input without proper sanitization, allowing attackers to inject malicious scripts and compromise user data.

While most modern frameworks (like React or Angular) handle this automatically, developers building or maintaining custom frameworks face a unique challenge: they must manually architect the defense.

A common misconception is that simply "escaping HTML" is a silver bullet. This article explores why context is king and how to implement robust XSS defenses in custom environments.

The Fallacy: "One Encoding Fits All"

Many developers believe that applying standard HTML encoding to user input guarantees safety. However, security depends entirely on the context in which the data is rendered, not just the data itself.

Consider this example where a developer has applied standard HTML encoding to userInput inside a JavaScript context:

```
<button onclick="alert('{{ html(userInput) }}')">Click me</button>
```

If the attacker inputs: '); alert(document.cookie); //

```
<button onclick="alert('');alert(document.cookie); //')">Click me</button>
```

The rendered HTML becomes:

Why did this fail?

The browser sees two valid JavaScript statements: alert('') and alert(document.cookie). Because standard HTML encoding does not escape characters like single quotes (') in a way that is safe for a JavaScript string context, the script executes.

Context-Aware Encoding

To prevent XSS effectively, you must apply encoding that is specific to where the data is landing in the DOM.

XSS Encoding Quick Reference

Golden Rule: Identify the sink (where data is output) first, then choose the correct encoding.

OUTPUT CONTEXT	ENCODING METHOD	EXAMPLE
HTML BODY	HTML Entity Encoding	{{ encodeHtml(text) }}
HTML ATTRIBUTE	HTML Attribute Encoding	value="{{ encodeHtmlAttribute(text) }}"
JAVASCRIPT STRING	JavaScript Unicode Escaping	const x = "{{ encodeJavaScript(text) }}"
CSS VALUE	CSS Hex Encoding	style="color: {{ encodeCss(text) }}"
URL / QUERY PARAM	URL Encoding	href="/search?q={{ encodeUrl(text) }}"

The Importance of Established Libraries

A dangerous anti-pattern in custom frameworks is attempting to write "home-grown" encoding functions, for example:

```
// BAD PRACTICE: Naive replacement
input.replace("<", "&lt;").replace(">", "&gt;");
```

This approach is insufficient. A proper encoding library handles complex edge cases that manual regex often misses, including:

- Quotes (single and double)
- Backslashes
- Control characters
- Unicode edge cases
- Browser-specific parsing behaviors

Missing even a single character can open an XSS vector. Attackers utilize techniques such as Unicode normalization, double encoding, and mixed encoding to bypass weak filters.

Recommendation: Use battle-tested libraries such as OWASP Java Encoder or DOMPurify (for JavaScript). These libraries are maintained by security experts to prevent regressions.

The Pitfalls of Double Encoding and Decoding

When building custom rendering pipelines, data often passes through multiple layers. This introduces two distinct risks.

1. The Visual Bug: Double Encoding

If a library encodes a container (for example, a <td>) and separately encodes the text content within it, the browser may render the raw encoded string (for example, <script>) to the user. This is a functional defect but also indicates a lack of control over the output pipeline.

2. The Security Flaw: Unsafe Decoding

A more critical issue arises when a developer decodes data and then passes it to a dangerous sink. If decoding occurs more often than

```html
<html>
<head></head>
<body>
    <h1 id='sample'>My First Heading</h1>
    <p>My first paragraph.</p>
</body>

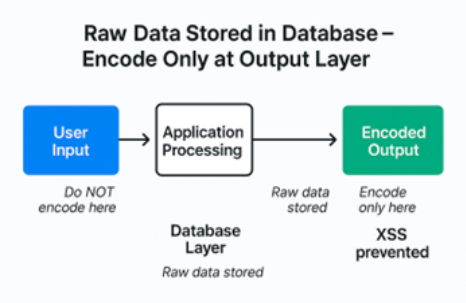
</html>div id="output"></div>

<script>
    // 1. Attacker input is technically "safe" HTML-encoded text
    const userInput = "&lt;img src=x onerror=alert('XSS')&gt;";

    // 2. Developer manually decodes it
    function decodeHtml(html) {
      const txt = document.createElement("textarea");
      txt.innerHTML = html; // The textarea parses the entities
      return txt.value;
    }

    const decoded = decodeHtml(userInput);
    // Becomes: <img src=x onerror=alert('XSS')>

    // 3. Since inner HTML can output any tags, the decoded
    // scripts gets executed, which can steal session cookies
    // or can execute any dangrous script
    document.getElementById("output").innerHTML = decoded;
</script>
```



encoding, or if decoded data is fed into a sink that interprets HTML, vulnerabilities reappear.

Consider this example of a “Broken Encoding” implementation:

The Fix: Output Layer Encoding

Follow the Just-In-Time (JIT) Encoding principle:

- Do not encode data when it is stored in the database or passed between internal services.
- Encode it once, at the last possible moment, just before it is handed to the browser.

If your framework supports nested components (for example, a text element inside a table cell), implement flags or context checks to ensure the parent container knows its child is already safe, preventing double encoding.

Sanitization vs. Encoding

In scenarios, such as WYSIWYG editors or email clients where you must render raw HTML. In these cases, encoding is not an option because it would break the styling.

Encoding	Sanitization
Displays untrusted text safely	Displays untrusted HTML safely
Escapes special characters	Removes unsafe tags & attributes
Output is plain text	Output is cleaned HTML
No HTML allowed	HTML allowed (whitelist)
Context-dependent	Policy / allowlist-dependent
Library: OWASP Encoder	Library: DOMPurify

Here, you need sanitization: the process of accepting input but removing unsafe tags (like `<script>`, `<object>`, `<iframe>`) and dangerous attributes (like onclick, onmouseover).

Encoding vs. Sanitization (Quick Comparison)

Warning: Sanitization is complex. A sanitizer might strip a <script> tag but miss a malicious onclick attribute on a . Never attempt to write a regex-based sanitizer. Use a library like DOMPurify that creates a sandbox to parse and clean HTML safely.

Consider the example below for regex-based removal of script tags.

```
function sanitize(input) {
    return input.replace(/ <script.* ? > .* ?<\/script>/gi, "");
}
```

If attackers provide the following input, XSS still executes as the sanitize method and only removes script tags.

```
<div onclick="alert('XSS')"> Click me</div>
```

Output encoding is the primary defense against XSS; sanitization should be used only as a fallback.

Conclusion

Building a custom framework requires a security-first mindset. It is not enough to simply “scrub” data. We must:

- Understand the context (HTML, JS, CSS, etc.) before encoding.
- Use standard libraries rather than regex replacements.
- Apply encoding strictly at the output layer to avoid double encoding or decoding flaws.
- Distinguish between encoding (for text) and sanitization (for HTML).

By embedding these practices into the core of a framework, security becomes a built-in feature rather than an afterthought.

Establishing Trust Anchors: Formal Verification of Multi-Tiered Certificate Chains

Rahul R, Lead Software Engineer - Open Enterprise Server

Abstract

To address the inherent complexities and potential vulnerabilities such as unauthorized certificate issuance and intermediate CA compromise, this paper presents a formal verification framework that rigorously validates end-to-end trust in the multi-tier PKI. It ensures the structural integrity of certificate chains and protects against systemic failures.

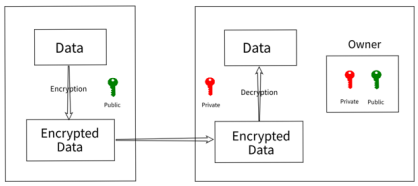
1. The Cryptographic Foundation: Asymmetric Key Pairs

At the core of PKI lies Asymmetric Cryptography, which uses mathematically related but distinct keys: the public key and the private key, as shown in the diagram below.



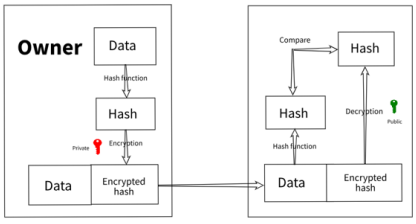
Encryption and Confidentiality

Data encrypted with a public key can only be decrypted by the corresponding private key. This ensures that sensitive information remains confidential, even if it is intercepted during transit, as illustrated in the diagram below.



Authentication and Digital Signatures

Asymmetric keys provide the mechanism for non-repudiation. By hashing data and encrypting that hash with a private key, an entity creates a digital signature. Any observer with the corresponding public key can verify the signature, ensuring the integrity and origin of the data, as shown in the diagram below.



While RSA remains the industry standard due to its maturity and wide compatibility, Elliptic Curve Cryptography (ECC) has emerged as a high-efficiency alternative, especially for faster signing and key exchange operations. In a PKI system where certificates are constantly being verified and signatures generated. ECC achieves equivalent security levels with significantly smaller key sizes—for instance, a 256-bit ECC key offers security comparable to a 3072-bit RSA key—thereby reducing computational overhead and bandwidth requirements in resource-constrained environments.

2. Structural Hierarchy of Public Key Infrastructure

PKI is a framework consisting of rules, protocols, and algorithms that establish trust relationships among various entities such as CAs, intermediate CAs, and end users. The cornerstone of this trust is the public key.

Certificates

A certificate is essentially a data structure that contains information about the owner, including their public key and the signature of the issuer (CA). Each entity in the PKI has its own certificate, as illustrated in the diagram below.



Role of the Certification Authority (CA):

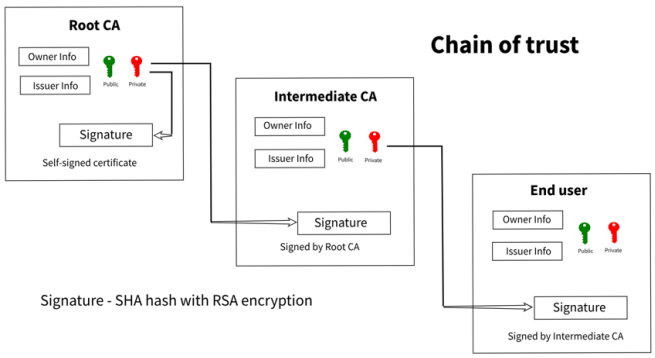
The CA signs certificates for Intermediate CAs and ensures that they are issued according to strict policies and procedures. Root CAs typically do not sign end-user certificates directly; instead, they delegate this responsibility to Intermediate CAs. This delegation reduces the workload and the risk associated with managing root CA private keys.

3. Construction of the Chain of Trust

The "Chain of Trust" is established through a series of digital attestations. The process begins at the top:

- The Root CA issues a self-signed certificate using its private key. This certificate is crucial as it establishes trust in the PKI environment. When browsers and systems trust the Root CA, they inherently trust all certificates that are signed by this CA because it is pre-trusted by operating systems and browsers.
- The Intermediate CA receives its digital certificate from the Root CA. This certificate is signed with the Root CA's private key, enabling delegated trust while reducing risk associated with the root key's direct use.
- The End-User Certificate is signed by the Intermediate CA using its private key. This certificate includes the information required to link it back to the Intermediate CA, thereby preserving the chain of trust. When a user accesses a website, the certificate presented can be traced back through the chain up to the Root CA.

The diagram below provides a more detailed illustration of how the chain of trust in a Public Key Infrastructure (PKI).



4. Formal Verification Processes

The verification process is structured into four distinct layers, each acting as a "gate" that the certificate must pass.

Layer I: Temporal and Structural Integrity

Before any cryptography is performed, the system validates the certificate's "metadata" to ensure it is biologically valid within the current time-space.

Temporal Bound:

$T_{start} <= T_{current} <= T_{end}$

Layer II: Identity Chaining (Subject-Issuer Mapping)

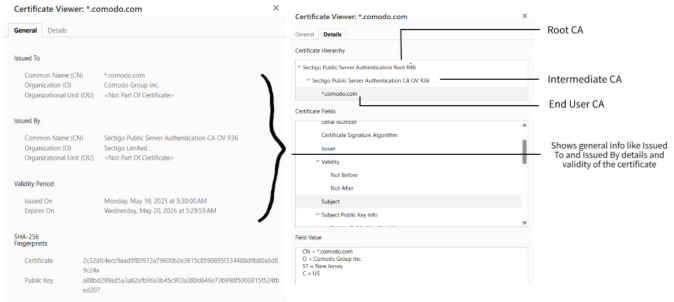
This layer ensures the physical "links" of the chain are connected in the correct order.

The Rule:

For any certificate C_i in a chain of length n, the Issuer Name of C_i must exactly match the Subject Name of C_{i+1}.

Failure State:

If a gap exists (for example, if the chain appears from a local CA to a global Root without an Intermediate), the linkage is considered broken and the chain is invalid.



Layer III: Cryptographic Delegation (Signature Verification)

This is the mathematical core of the process. It proves that the identity established in Layer II was actually authorized by the next certificate in the chain.

- Logic: The system extracts the Public Key from C_{i+1} to decrypt/verify the Digital Signature on C_i.
- RSA/ECC Application: If the hash of C_i matches the decrypted signature, it is mathematically proven that C_i was signed by the holder of the private key associated with C_{i+1}.

Layer IV: Anchor Certification (Axiomatic Trust)

The final layer determines where the "burden of proof" ends.

- Trust Anchor: The chain must terminate at a Root CA whose certificate is present in the client's local Trust Store (for example, Windows Certificate Store or NSS store used by Firefox).
- Self-Signature: At this level, the Root CA is self-signed (*Issuer* = *Subject*).

The image below shows the real-world certificate chain for the www.comodo.com website.

Each Operating System ships with a list of pre-installed root CA certificates. In the image above, the root CA Sectigo Public Server Authentication Root R46 should be present in the operating system trust store to allow browsers to verify the authenticity of these certificates.

5. Conclusion

Establishing trust anchors through multi-tiered certificate chains is the primary defence against identity spoofing and data tampering. However, the future of PKI must contend with the dual challenges of quantum computing and increased architectural complexity. The emergence of Shor's algorithm threatens the long-term viability of RSA, necessitating a proactive transition toward Post-Quantum Cryptography (PQC) and hybrid certificate models.

Future research must focus on integrating automated certificate management protocols and decentralized trust models to ensure that formal verification remains scalable without compromising the cryptographic rigor provided by current multi-tiered standards.

6 References

Component	Standard / RFC
Path Validation	RFC 5280
RSA Primitives	RFC 8017
Handshake	RFC 8446
Intermediate CA	X.509 v3

Importance of a Reliability Setup for Product Releases

Abhinandan Narayan, Lead QA Engineer - Endpoint Management

Why Reliability Testing Is the Unsung Hero of Successful Product Releases

In enterprise software, a significant percentage of production issues surface not because new features are poorly built, but because updates fail to coexist with the reality of long-lived customer environments—systems shaped by years of upgrades, configuration changes, and residual artifacts.

Modern product updates are no longer simple version increments. Each release may introduce new features, security enhancements, performance optimizations, and essential technical upkeep. However, the success of an update is not determined solely by how well it performs in a clean or freshly installed environment, but by how reliably it operates in real customer setups that often carry legacy files, deprecated configurations, and historical data.

A reliability setup that closely mirrors customer installations is therefore critical to ensuring seamless updates and uninterrupted access to features. This journal highlights the importance of such environments and explains how the absence of reliability-focused testing can lead to update failures, operational instability, and a degraded customer experience.

The Reality of Customer Environments

Unlike fresh test systems, customer environments evolve over time:

- Multiple in-place upgrades across years
- Legacy configuration files and folders
- Deprecated libraries and runtimes
- Manual customizations or hotfixes

During updates, these leftover artifacts can interact unpredictably with new product versions, causing failures that are difficult to reproduce in standard test setups.

Aspect	Simple Upgrade	Real Customer Environment Risks
Installation History	limited version jumps	Multiple in-place upgrades over years
System Cleanliness	No residual files or folders	Leftover directories, orphaned files, and stale metadata
Configuration State	Default or newly generated configurations	Legacy and modified configuration files
Dependencies	Uses bundled and supported libraries	Deprecated libraries or older runtimes still present
Customizations	No manual changes	Customer-applied hotfixes, scripts, or workarounds
Update Predictability	Highly predictable and repeatable	Unpredictable interactions during update
Failure Reproducibility	Issues easy to reproduce in test labs	Failures hard to reproduce in standard environments
Risk During Update	Minimal	High risk of upgrade failures or partial feature breakage
Impact on User Experience	Smooth transition	Downtime, degraded performance, or feature loss

Product Updates: More Than Just New Features

Every product update typically includes:

- New feature additions
- Bug fixes and performance improvements
- Security patches
- Technical upkeep, such as:
  - Runtime upgrades (Java, Python, Node, etc.)
  - Library and dependency updates
  - Configuration format changes

While feature testing often receives significant attention, technical upkeep-related changes are more prone to failure in real-world environments.

Hidden Risks of Leftover Files and Folders

One of the most common causes of update failures is the presence of obsolete files and folders left behind from previous versions.

Typical Scenarios

- Old configuration files overriding new defaults
- Deprecated folders still being referenced during startup
- Legacy environment variables affecting runtime behavior

These issues may not surface in fresh installations but can critically impact in-place upgrades.

Real-World Example: Java Runtime Migration

Phase 1: Oracle Java 8 -> Zulu Java 8

Although the Java version remains the same, changes in directory structure, bundled libraries, and startup configuration introduce

risk. Customers may already have custom JVM options, additional JARs, or environment-specific overrides.

Phase 2: Java 8 → Java 17

Risk increases significantly as certain directories (for example, lib/ext) are no longer supported. If legacy folders remain, the Java runtime may fail during startup, causing update failure, application downtime, and inaccessible new features.

These failures stem from legacy artifacts, not defects in new functionality.

Impact on Customers and Business

Update failures arising from legacy artifacts and unvalidated customer conditions can have significant technical and business consequences, including:

- Production downtime, directly impacting business continuity and SLAs
- Erosion of customer trust, especially for enterprise and regulated customers
- Increase in support tickets and escalations
- Delayed adoption of new features
- Higher operational costs for both customers and the organization

Industry observations consistently show that organizations investing in production-like reliability setups experience up to a 30–40% reduction in post-release defects and a 25–35% decrease in high-severity support escalations. Early detection of upgrade-related issues also shortens Mean Time To Resolution (MTTR) by up to 50%, as failures are reproducible before reaching production.

From a return-on-investment perspective, identifying update failures during pre-release reliability testing is 5–10 times less expensive than fixing the same issues after deployment. Fewer emergency patches and rollbacks translate directly into lower support costs, reduced engineering firefighting, and more predictable release cycles. Most importantly, consistent and reliable updates reinforce customer confidence, strengthening long-term relationships and improving renewal and retention rates.

Why a Reliability Setup Is Critical

A reliability setup replicates real customer conditions rather than ideal test environments.

Key Characteristics include

- Incremental upgrades across multiple versions
- Legacy files, folders, and configurations
- Mixed runtime histories
- Long-term usage patterns
- Such setups help teams detect upgrade blockers early, validate cleanup and migration logic, ensure backward compatibility or graceful deprecation, and prevent startup or feature-access failures

Recommendations to Improve Release Reliability:

1. Maintain Customer-Like Environments
   - Preserve test systems across releases
   - Validate common upgrade paths
   - Seed real customer artifacts

   Focus on long-running and high-value deployments.
2. Automate Legacy Detection
   - Scan for deprecated files, folders, and runtimes
   - Fail or flag builds on detection
   - Run checks in CI and pre-upgrade stages
3. Embed Cleanup and Migration
   - Include cleanup and migration in upgrade scripts
   - Ensure scripts are re-runnable
   - Prioritize startup and core workflow fixes
4. Validate Startup with Legacy State
   - Test startup and restarts with legacy artifacts
   - Monitor logs for warnings
   - Add automated health checks
5. Treat Technical Upkeep Like Features
   - Test refactors and dependency updates
   - Review impact and rollback risk
   - Prioritize high–blast-radius changes

Reliability Setup Validation Checklist

- Multiple upgrades completed
- Legacy artifacts present
- Deprecated libraries handled
- Automated scans pass
- Cleanup and migration executed
- Application starts successfully
- Logs show no hidden errors
- Rollback works
- Upkeep changes tested
- Upgrade paths validated

Conclusion

A successful product release is not defined by how well it works on fresh/simple system, but by how reliably it upgrades in real customer environments. Legacy files, deprecated folders, and long-term runtime transitions—such as Java upgrades—introduce significant risk when teams do not test them thoroughly.

Teams must establish reliability setups that mirror real customer installations to prevent upgrade failures, protect feature access, and deliver a consistent customer experience. Early investment in these setups reduces post-release defects, lowers support costs, and strengthens customer trust.

Invest in reliability setups today to ensure smoother upgrades tomorrow—protect your customers, product, and brand.

Observability-First Design: The Modern Successor to TDD for Microservices

Krishna Kumar Parthasarathy, Sr. Software Engineer - Endpoint Management

You deploy a new microservice.

Introduction

All tests pass. Unit tests are green. Integration tests look solid. The canary rollout starts smoothly.

Then, within hours, users begin reporting latency spikes. Some requests are slow. Others fail intermittently. Dashboards don't immediately show anything obvious. You try to reproduce the issue locally—nothing happens. You run the test suite again—still green.

So, what went wrong?

The uncomfortable answer is that nothing within your code is broken. The problem lies in the space between services: runtime traffic patterns, dependency behavior, configuration drift, or environmental differences that no test ever modeled.

This is the reality of modern microservices.

For nearly two decades, Test-Driven Development (TDD) has been the backbone of reliable software development. The discipline taught us to write tests first, reason about behavior, and build confidence in correctness before shipping code.

And TDD still matters––deeply.

But TDD was designed for a world where systems were smaller, more deterministic, and easier to reason about in isolation.

This gap has led to an emerging practice that many high-scale teams are converging on–– Observability-First Design (OFD).

What TDD Solves and Where It Stops

TDD is excellent at verifying behavioral correctness. When logic is deterministic and dependencies are predictable, tests give confidence that the code behaves as intended. In monolithic or tightly controlled systems, this works beautifully.

Microservices quietly break these assumptions.

In distributed systems, many failures have nothing to do with incorrect logic. They arise from retry storms, DNS issues, deployment mismatches, downstream overloads, or subtle configuration drift. No unit test can fully simulate these conditions, because they only exist at runtime.

Even worse, microservices introduce non-determinism. The same request may hit different pods, take different paths, or interact with dependencies in unpredictable sequences. Real-world traffic — regional skew, tenant isolation, uneven load — never shows up in pre-production environments.

A significant portion of outages are caused not by bugs, but by runtime misconfigurations. TDD can't protect you from these.

This leads to a critical realization–– Observability must be designed before the code ships.

What Observability-First Design Really Means

Observability-First Design (OFD) doesn't start with tools or dashboards. It starts with a shift in how you think about features.

Instead of asking "How do I implement this?", OFD asks–– "How will I understand this feature when it runs in production?"

A Step-by-Step Example: Applying OFD While Designing a Feature

Imagine you're building a new endpoint:

POST /orders

Step 1: Start with production questions (before writing code)

Before touching the implementation, the team asks:

- How will we know if this endpoint is slow?
- How will we distinguish validation failures from downstream failures?
- How will we know if a dependency (payment or inventory) is degrading?
- What would an on-call engineer need to debug this at 3 a.m.?

At this stage, no code exists — only questions.

Step 2: Define what "healthy" looks like

The team defines a few simple signals that describe success and failure:

- Request latency (p95)
- Success vs failure rate
- Downstream dependency latency
- Business outcome: orders created

This form the operational contract of the feature.

Step 3: Design telemetry before logic

Now the team decides how the system will communicate in production:

- Traces will capture the full request journey across services.
- Structured logs will record meaningful events like "order created" or "payment failed".
- Metrics will summarize throughput, errors, and latency trends.

At this point, the team knows that when something breaks, they'll be able to answer:

- Is this widespread or isolated?
- Where is time being spent?
- Which dependency is involved?

Step 4: Implement the feature

Now the team writes the business logic.

The code naturally emits the telemetry that was already designed. Observability isn't bolted on — it's built in.

Step 5: Validate observability itself

The team tests visibility, not just logic.

They verify that the following:

- Traces propagate correctly.
- Logs include the right context.
- Metrics move as expected.

In OFD, observability is treated as part of the feature correctness.

Step 6: Prepare for deployment before deploying

Dashboards and alerts are built before the feature rollout. Canary analysis is configured using the same signals defined earlier.

When the feature is deployed, the team already knows what "normal" and "dangerous" look like.

TDD vs OFD vs Combined Approach

Here's a simple comparison that clarifies the relationship:

Dimension	TDD	OFD	TDD + OFD (Combined)
Primary focus	Code correctness	Runtime behavior	Correct *and* observable systems
Scope	Inside a service	Across services	End-to-end lifecycle
Catches	Logic bugs	Latency, config, dependency failures	Both code and production failures
Feedback timing	Pre-commit	Production runtime	Before *and* after deployment
Debugging	Test failures	Logs, metrics, traces	Tests + real-world insight
Designed for	Deterministic systems	Distributed systems	Modern microservices
Failure visibility	Limited	High	Maximum

Why This Matters

TDD answers–– "Does my code work as expected?"

OFD answers–– "Can I understand my system when reality disagrees?"

In microservices, you need both answers.

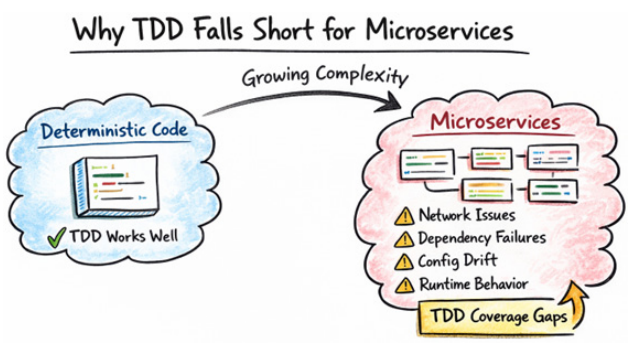
How Logs, Metrics, and Traces Fit Together

When people hear "observability," they often think of three separate things: logs, metrics, and traces. In practice, observability works only when these three signals describe the same request, from different angles.

Consider a single request:.

POST /orders

This request enters your system and begins its journey through multiple servicesproducing three kinds of signals.



Traces: Following one request end-to-end

As soon as the request enters the system, a trace is created.

This trace is identified by a unique identifier, for example:

```
trace_id = abc123
```

The trace ID follows the request everywhere it goes — through the order service, payment service, inventory service, and back. Each hop becomes a span inside the trace, with timing information attached.

Later, when someone asks "Why was this request slow?", traces answer the question by showing exactly where time was spent and which dependency contributed to the delay.

A trace doesn't tell you how often a problem happens — it tells you what happened for this specific request.

Logs: Recording meaningful events during the journey

While the request is being processed, services emit structured logs. These logs capture meaningful moments, not just debug noise.

For example:

```
INFO  Order created  order_id=12345 trace_id=abc123
ERROR Payment timeout provider=PayFast
trace_id=abc123
```

The key detail that the log entries include is the same trace ID as the trace itself, which makes correlation possible.

If you're looking at a slow trace, you can instantly pull all log entries with trace_id=abc123 and check what actually happened during that request — which decision was made, which dependency failed, or which retry kicked in.

Logs answer the question–– "What events occurred while this request was executing?"

Metrics: Understanding system behavior over time

At the same time, the system updates metrics. Metrics don't describe individual requests — they describe patterns.

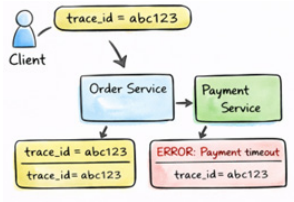
Examples include:

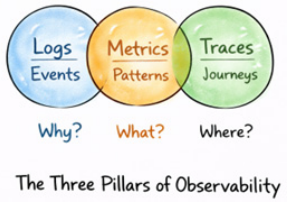
- request_latency_seconds
- error_rate

These metricsprovides information such as:

- Latency has increased over the last 10 minutes
- Error rate jumped from 0.2% to 1.8%
- A rollout correlates with a spike in failures

Metrics are what trigger alerts. They tell you that something is wrong, but not necessarily why.





Metrics answer the question-- "Is this problem isolated, or systemic?"

How they work together in practice

When something breaks in production, observability works as a sequence:

1. Metrics alert you that there is a problem (latency spike, error rate increase)
2. Traces show you where the problem is happening (slow downstream call, retry amplification)
3. Logs explain why it happened (timeouts, misconfiguration, unexpected state)

None of these signals are sufficient alone. Together, they turn debugging from guesswork into investigation.

Observability-First Design ensures these signals are designed together, so production failures are explainable instead of mysterious.

Metrics raise the alarm. Traces show the path. Logs tell the story.

Closing Thoughts: Designing for Reality

Modern software systems don't fail because developers don't write tests.

They fail because reality is more complex than any test suite can predict.

Test-Driven Development remains essential. It gives us confidence that our code behaves correctly in isolation. But in distributed systems, correctness alone is not enough. What matters just as much is understanding — understanding how a system behaves under real traffic, real dependencies, and real failure modes.

Observability-First Design (OFD) addresses that gap.

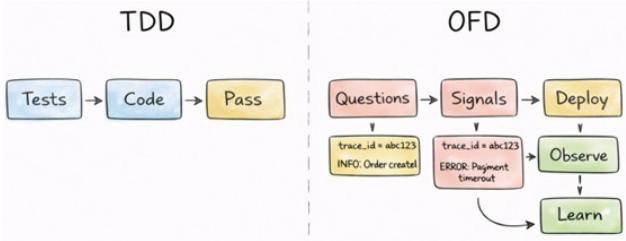
By designing visibility into features before they are built, OFD ensures that when something goes wrong in production, the system explains itself. Latency is no longer mysterious. Failures are no longer silent. Debugging becomes investigation, not guesswork.

This isn't about replacing TDD. It's about completing the lifecycle.

TDD helps you ship with confidence. OFD helps you operate with clarity.

Actionable Takeaway: A Simple Checklist for Adopting OFD

Observability-First Design doesn't require platform rewrite or new tools. It starts with asking better questions at the right time.



Before your next design review, try walking through this checklist:

- What questions should this feature answer in production?
- How will we know if this feature is slow, partially broken, or silently failing?
- Which signals (logs, metrics, traces) will help an on-call engineer debug this at 3 a.m.?
- Do our logs capture meaningful events with correlation identifiers?
- Do our metrics clearly show latency, error rates, and traffic patterns over time?
- Can a single request be traced end-to-end across service boundaries?
- Are dashboards and alerts prepared before the feature is deployed?
- If this fails during a canary rollout, will the system explain itself?

If you cannot confidently answer these questions, the feature may work — but it won't be observable.

That's the shift OFD encourages.

TDD helps you ship with confidence.

OFD helps you operate with clarity.

Start your next design discussion with one simple question:

How will we observe this feature in production — especially when it fails?

In microservices, the most reliable systems aren't just well-tested-– They are well-understood.

References

1. https://www.devopsinstitute.com/wp-content/uploads/2021/01/2020-Observ-Charity-Majors-ppt.pdf
2. https://medium.com/@vitorhansen/from-zero-to-observability-first-designing-logs-metrics-and-tracing-for-modern-applications-423ae216763c
3. https://medium.com/@blakeromano/observability-first-development-5afe29a5c429

Migrating from Apache Tiles to Thymeleaf with Spring 6

Dinesh Kumar Sahoo, Lead Software Engineer - File and Print Services, OES

A guide for migrating from Apache tiles to Thymeleaf

1. Motivation for Migration

When upgrading the CAF and iPrint Java web applications from Java 11 to Java 21, it became necessary to migrate the Spring framework from version 5.x to 6.x. This upgrade was essential not only for compatibility with Java 21 but also to address several security vulnerabilities present in Spring 5.x.

- Apache Tiles Deprecation: Apache Tiles is no longer actively maintained and is incompatible with the latest Spring 6 and Jakarta EE namespaces.
- Thymeleaf Advantages: Modern, actively maintained, natural templating (HTML5), strong Spring integration, and better support for dynamic content.
- Spring 6 Compatibility: Spring 6 drops support for Tiles and require template engines compatible with Jakarta EE 10+.

2. Architectural Differences

- Tiles: Layout-based, XML definitions, JSP-centric, uses fragments and attribute inheritance.
- Thymeleaf: Template engine, HTML5-based, uses layout dialects for fragments, supports direct HTML editing, and integrates natively with Spring MVC.

3. Migration Process

Step 1: Remove Tiles Dependencies

- Remove tiles-core, tiles-extras, and any Tiles Spring integration from pom.xml.

Step 2: Add Thymeleaf Dependencies

Aspect	Apache Tiles	Thymeleaf
Layouts	XML definitions, JSP	HTML fragments, dialects
Integration	Servlet/JSP, Spring plugin	Native Spring support
Maintenance	Deprecated	Actively maintained
Syntax	XML + JSP	HTML5 + expressions

- Add thymeleaf-spring6 and (optionally) thymeleaf-layout-dialect to pom.xml.

Step 3: Update Spring Configuration

- Remove `TilesConfigurer` and `ViewResolver` beans.
- Add `ThymeleafViewResolver`, `SpringTemplateEngine`, and `TemplateResolver` beans.

Example (`spring-servlet.xml`):

Before (Apache tiles):

```
<!-- Apache Tiles View Resolver -->
<bean id="tilesViewResolver" class="org.
springframework.web.servlet.view.
UrlBasedViewResolver">
<property name="viewClass" value="org.
springframework.web.servlet.view.tiles3.
TilesView"/>

<property name="order" value="1"/>
</bean>
<bean id="tilesConfigurer" class="org.
springframework.web.servlet.view.tiles3.
TilesConfigurer">

<property name="definitions">

<list>
<value>/WEB-INF/tiles*.xml</value>
</list>

</property>
<property name="preparerFactoryClass" value="org.
springframework.web.servlet.view.tiles3.
SpringBeanPreparerFactory"/>
</bean>
 <!-- Registers a JSP resolver for MVC controllers
-->
<bean id="jspViewResolver" class="org.
springframework.web.servlet.view.
InternalResourceViewResolver">
<property name="viewClass" value="org.
springframework.web.servlet.view.JstlView" />

<property name="order" value="2"/>
<property name="prefix" value="/WEB-INF/jsp/" />
<property name="suffix" value=".jsp" />
</bean>
```

After (Thymeleaf):

```
<bean class="org.thymeleaf.spring6.view.
ThymeleafViewResolver">
    <property name="viewClass"
        value="org.thymeleaf.spring6.view.
ThymeleafView"/>
        <property name="templateEngine"
    ref="templateEngine"/>
    <property name="characterEncoding"
value="UTF-8"
    />
    <property name="order" value="1" />
</bean>

<!-- thymeleaf changes -->
<bean id="templateResolver" class="org.
thymeleaf.spring6.templateresolver.
SpringResourceTemplateResolver">
    <property name="prefix" value="/WEB-
    INF/templates/" />
    <property name="suffix" value=".html" />
    <property name="templateMode" value="HTML" />
    <property name="characterEncoding"
value="UTF-8"
    />
    <property name="cacheable" value="false" />
</bean>

<!-- Template Engine with Layout Dialect -->
<bean id="templateEngine" class="org.thymeleaf.
spring6.SpringTemplateEngine">
    <property name="templateResolver"
ref="templateResolver" />
    <property name="additionalDialects">
       <set>
          <bean
class="nz.net.ultraq.thymeleaf.layoutdialect
.LayoutDialect"/>
  </set>
</property>
</bean>
```

Step 4: Convert Layouts and Templates

- Identify Tiles definitions (usually in tiles.xml).
- For each layout, create a Thymeleaf layout template using th:fragment and th:replace/th:insert.
- Convert JSPs to HTML/Thymeleaf syntax, replacing Tiles tags with Thymeleaf attributes.

Example:

Tiles (tiles.xml):

```
<definition name="baseLayout" template="/WEB-INF/
layouts/base.jsp">

<put-attribute name="title" value=""/>

<put-attribute name="body" value=""/>
</definition>
```

Thymeleaf (base.html):

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/
thymeleaf/layout">
<head>
    <title layout:title-pattern="${title}">Default
Title</title>
</head>
<body>
    <div layout:fragment="content"></div>
</body>
</html>
```

Step 5: Refactor Controllers

- Change controller return values from Tiles view names to Thymeleaf template names (e.g., return "homePage"; for homePage.html).

```
// Before
return "homePage.tiles";

// After
return "homePage";
```

Step 6: Test and Validate

- Use integration and UI tests to verify all pages render correctly.
- Check for broken links, missing fragments, and expression errors.

4. Common Challenges and Solutions

- JSP to HTML Conversion: Replace JSP tags with Thymeleaf equivalents.
  - `<c:forEach>` -> `th:each`
  - `<c:if>->th:if / th:unless`
  - `<c:choose>` -> `th:switch / th:case`
  - `<fmt:message>` -> `#{}` (message expressions)
  - `<fmt:formatDate>` -> `#dates.format(...)`
- Fragment Inclusion: Use th:replace or th:insert for reusable fragments.
- Form Handling: Update forms to use th:action, th:object, and th:field.
- Internationalization: Use `th:text="#{message.key}"` for i18n messages.
- Custom Tags: Refactor custom JSP tags as Thymeleaf dialects or fragments.

Note: Copilot can migrate approximately 70–80% of JSTL code into Thymeleaf equivalents. After migration, manual validation is required to ensure accuracy and completeness.

5. Code Examples

Tiles JSP Example

```
<%@ taglib uri="http://tiles.apache.org/tags-tiles"
prefix="tiles" %>
<tiles:insertDefinition name="baseLayout">
    <tiles:putAttribute name="title" value="Home"/>
   <tiles:putAttribute name="body" value="/WEB-INF/
views/home.jsp"/>
</tiles:insertDefinition>
```

Thymeleaf HTML Example

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
            <title th:text="${title}">Home</title>
</head>
<body>
    <div th:replace="fragments/body :: body"></div>
</body>
</html>
```

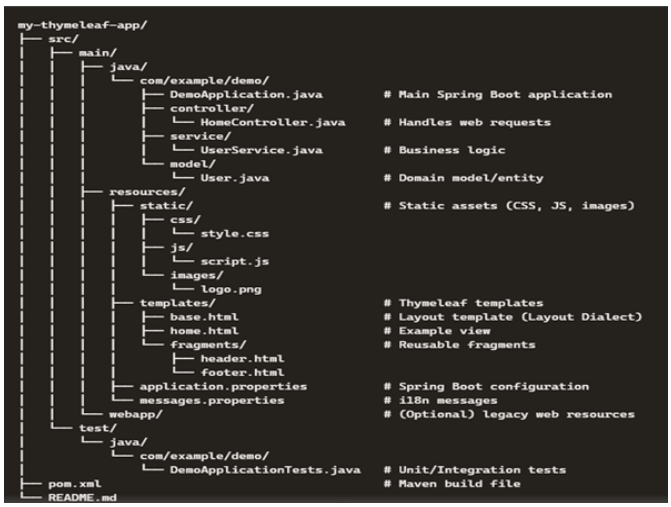
6. Testing and Validation

- Use Spring Boot Test or MockMvc for controller and view integration tests.
- Manually verify all layouts, fragments, and dynamic content.
- UI testing also can be done with UI automation tools e.g. Selenium.
- Validate all the links, i18n, form binding.
- Use browser dev tools to check for missing resources or errors.

7. Lessons Learned and Best Practices

- Plan migration incrementally: migrate layouts first, then individual pages.
- Use Thymeleaf Layout Dialect for complex layouts.
- Keep templates as close to valid HTML as possible for maintainability.
- Leverage Spring's i18n and form binding features.
- Remove all legacy Tiles/JSP artifacts after migration.

8. Sample Project structure



9. Performance Considerations for Thymeleaf

Enable Caching in Production:

- Thymeleaf caches parsed templates by default, which avoids re-parsing HTML files on every request. To configure this use spring.thymeleaf.cache=true.
- It will do faster rendering with reduced CPU usage.
- Use message bundle caching spring.messages.cache-duration for i18n performance.

Reduce Server -side Rendering load:

- Offload heavy UI logic to client(JavaScript frameworks) when possible.
- Use thymeleaf mainly for templating and lightweight dynamic content.

Monitoring and Profiling:

- Use tools like Spring Boot Actuator to monitor performance.
- Profile rendering times to identify slow templates or inefficient fragments.

10. Final Architecture

Before: JSP -> JSTL -> Tiles -> Spring MVC

After: HTML -> Thymeleaf -> Layout Dialect -> Spring MVC

11. References

- Spring 6 Documentation
- Thymeleaf Documentation
- Thymeleaf Layout Dialect
- Tiles to Thymeleaf Migration Guidee

Scaling File Uploads: A Merkle Tree–Based Approach to Resilient Multi-Gigabyte Transfers

Kratagya Jadoun Singh, Software Engineer - Endpoint Management

Introduction

Ever tried uploading a 2 GB video file to the cloud, only to have it fail at 95% because your Wi-Fi hiccupped? You're not alone. Studies show that large file uploads over 10 GB fail approximately 30-40% of the time on average consumer networks. For healthcare systems transferring patient CT scans, video streaming platforms handling raw footage, or companies backing up terabytes of data, these failures aren't just annoying—they're productivity killers.

This article presents a Merkle tree-based chunked upload architecture that enables parallel transfer, cryptographic integrity verification, and efficient resumability. By decomposing files into independently verifiable chunks and using content-addressed metadata, the system converts network failures from catastrophic events into localized, recoverable conditions. Real-world testing demonstrates 60-70% throughput improvements and up to 97% bandwidth savings during upload resumption.

The Problem: Unreliable Networks Meet Monolithic Uploads

Traditional upload systems treat files as indivisible units––like mailing a 500-page book in one envelope. if anything goes wrong, the entire package comes back. Many existing systems assume stable, long-lived network connections, but reality is different. Networks are inherently unreliable: transient packet loss, timeouts, mobile tower handoffs, or connection resets can interrupt transfers at any point.

The core challenge isn't just moving data—it's doing so reliably over unreliable networks while preserving efficiency, resumability, and strong integrity guarantees. When a 10 GB medical imaging file fails at 99%, the medical practitioners are forced to upload the data again, which is a wasteof time and bandwidth.

System Architecture Overview

The solution divides the upload process into three distinct phases:

1. Metadata negotiation and cryptographic fingerprinting
2. Parallel chunk transfer with independent fault tolerance
3. Atomic server-side reconstruction

At a high level, the client preprocesses the file to generate a Merkle tree over fixed-size chunks. The resulting metadata is shared with the server, which uses it to validate chunks, track progress, and enable resumable uploads.

Phase 1: Metadata Generation and Merkle Tree Construction

Before uploading data, the system preprocesses the file by dividing it into fixed-size pieces–– typically 5-10 MB each. The chunk size involves trade-offs:

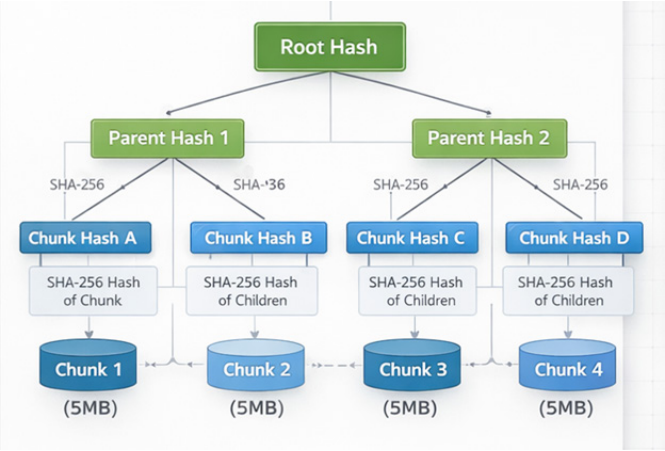
For a 2 GB file with 5 MB chunks (roughly about 400 pieces), each chunk gets a cryptographic SHA-256 hash —a unique mathematical fingerprint that changes completely if even one byte is differentThese hashes form the leaves of a binary Merkle tree. Parent nodes are computed by hashing their concatenated children. The root hash serves as a cryptographic fingerprint for the entire file.

This structure provides three powerful capabilities:

- Content addressability: The root hash uniquely identifies file content independent of filename.
- Tamper detection: Any corruption in the chunks invalidates the tree.
- Efficient verification: Individual chunks can be validated independently

The client sends a lightweight manifest to the server containing filename, file size, root hash, and all chunk hashes. The server stores

Chunk Size	Retry Efficiency	Protocol Overhead	Best For
1-2 MB	High (small retries)	High (more requests)	Unstable networks
5-10 MB	Balanced	Balanced	Most scenarios
20+ MB	Low (large retries)	Low (fewer requests)	Stable, high-bandwidth links



this using the root hash (optionally combined with filename) as a stable identifier. This happens before any actual data transfer begins.

Phase 2: Parallel Chunk Upload

After the server has the metadata, real work begins. Instead of uploading chunks sequentially, the system sends multiple chunks simultaneously using a bounded thread pool–– like having 8 checkout lanes instead of one dramatically increasing the throughput.

Each chunk travels independently as a separate HTTP request with its index, root hash, and expected hash.

Key robustness principles include the following:

Independence: Chunks have no ordering or cross-dependencies. Any chunk can be uploaded, retried, or duplicated without coordinating with others.

Retry Isolation: Failures affect only the individual chunk. Network errors trigger exponential backoff retries for that chunk alone while others continue uninterrupted.

Server-Side Verification: Recomputes the chunk hash and validates it against the manifest before persistence. Invalid chunks are rejected immediately.

Idempotency: Ignores duplicate chunks because the chunks are content-addressedThis ensures correctness even under ambiguous failure conditions.

Substantial performance gains observed during our testing are as follows:

- Files around 500MB: 30% faster (upload time reduced from ~8 minutes to ~5.5 minutes on a 100 Mbps link)
- Files over 1GB: 70% faster (upload time reduced from ~120 minutes to ~36 minutes on a 100 Mbps link)
- Average improvement: 60-70% across varied workloads

Parallelization especially helps on high-latency connections by masking round-trip delays that serialize traditional uploads.

Phase 3: Atomic Reconstruction

When all chunks arrive, the server reconstructs the file using the following steps:

1. Integrity validation - Recompute the Merkle tree from stored chunk hashes and verify against the declared root hash.
2. Sequential reassembly - Stream chunks in index order to produce the final file.
3. Atomic completion - Move the file to its final location only after successful verification.
4. Cleanup - Mark temporary chunk storage and manifests for deletion.

If integrity verification fails at any step, the entire upload is rejected. Corrupted files are never stored.

Resumability Protocol: The Real Game-Changer

The primary advantage emerges during failure recovery. If your upload dies at 95%, you don't start over.

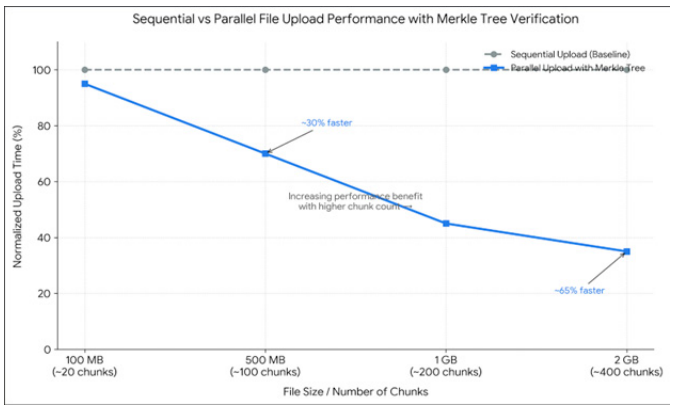
When the client reties, it sends the same metadata with the same root hash. The server recognizes the upload and performs differential analysis:

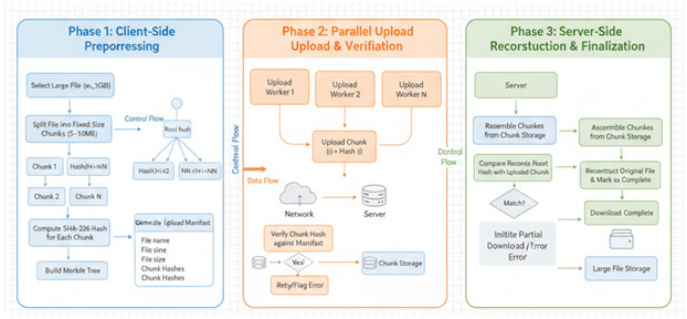
- Valid chunks already present are marked complete
- Missing or invalid chunks are returned as a retry set

Instead of re-uploading 2 GB, you might require to upload only 65 MB—reducing retry bandwidth by over 90% for near-complete uploads.

Handling Edge Cases:

- Corrupted metadata: The server rejects malformed JSON or missing fields; the client retries metadata submission.





- Chunk level corruption: Invalid chunks fail hash validation and are added to the retry set.
- Unknown root hash: The server treats the upload as new.

The Merkle tree ensures integrity throughout–– the final reconstruction must produce the correct root hash or the upload fails.

Deduplication and Resource Management

The root hash enables smart deduplication:

- Identical content with the same name completes instantly without transferring data
- Identical content with different names is treated as distinct (supports different contexts)
- Different content with the same name is treated as a new version.

To manage resources, incomplete uploads follow time-based eviction: manifests older than seven days are purged along with their chunk storage.. During active uploads, storage requirements match traditional approaches— the same data is stored, just organized differently.

Performance Characteristics and Trade-offs

Real-world testing demonstrates significant benefits:

- Throughput gains: 60-70% average improvement, with up to 12× improvement on high-latency links
- Resumption efficiency: Bandwidth savings proportional to completion percentage (95% complete = 95% bandwidth saved)
- Low overhead: Merkle metadata grows logarithmically with chunk count—for a 2GB file with 400 chunks, metadata is only ~13KB
- Predictable memory: Server memory is bounded by chunk size, not file size

Complexity vs Benefit: Preprocessing adds approximately 2-3 seconds per GB for hash computation on modern hardware. However, this small upfront cost saves substantial time and bandwidth (about 60-70% on initial upload time and up to 97% bandwidth on retries). For a 5GB file that fails once, you

save roughly about 150 minutes of upload time and 4.85 GB of bandwidth.

The approach introduces additional complexity, requiring careful state management on both client and server. But for multi-gigabyte transfers, the trade-off is overwhelmingly positive.

Conclusion

By combining Merkle tree verification with parallel, content-addressed chunk uploads, this system transforms large file transfers from fragile operations into resilient, fault-tolerant processes. Network failures become localized events rather than catastrophic resets.

For cloud storage providers, video streaming platforms, and enterprise backup systems, this architecture offers a practical path to better user experience. A researcher can upload a 10 GB dataset knowing that a brief connection drop won't cost another two hours. A video editor can close their laptop mid-upload and resume hours later from exactly where they stopped.

Broadly, the architecture illustrates a key principle of distributed systems: large data transfers should be decomposed into independently verifiable units. This enables scalability, reliability, and efficiency—one chunk at a time.

Future Work

Potential architectural enhancements include the following::

- Adaptive chunk sizing: Consider real-time network conditions—smaller chunks for unstable connections, larger chunks for stable high-bandwidth connections.
- Predictive retry scheduling: Use machine learning to anticipate network degradation.
- Cross-device resumability Allow uploads to continue seamlessly across devices.

eBPF - From Basics to Production

Deepika Singh, Lead Software Engineer - Open Enterprise Server

Introduction

Not all our readers will be familiar with what we do in OES. Hence it will be good to elaborate on the OES use case, and also other specific applications where one can use this, specifically observability, security etc which you have rightly pointed. The idea is that our readers should have a takeaway from the article.

What is eBPF?

Extended Berkeley Packet Filter (eBPF) is a kernel technology that lets you run small, sandboxed programs inside the operating system at many safe attach points (hooks). Those programs can observe, filter, and act on events with near-native performance, while remaining memory-safe thanks to a strict verifier and helper APIs. Originally born for fast packet filtering, eBPF now powers observability, security, and networking use-cases from tracing syscalls to enforcing policies at the socket or cgroup level.

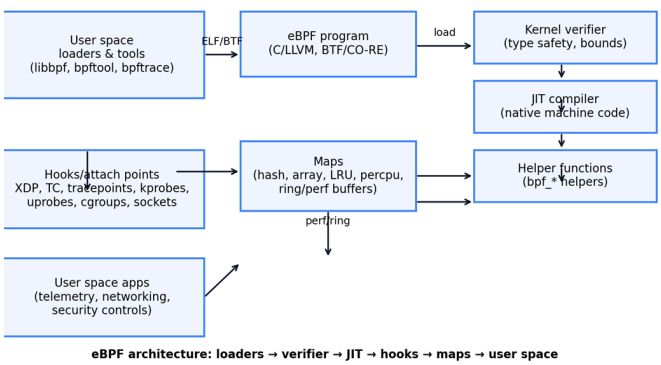
How eBPF works (architecture)

User space tools (e.g., libbpf, bpftool, bpftrace) compile and load an ELF object that contains eBPF bytecode and BTF type info. The kernel verifier checks safety and correctness, the JIT compiles bytecode to native machine code, and the program is attached to a hook such as XDP/TC/tracepoints. Maps provide shared state and data channels to user space via perf/ring buffers.

Program types & hooks

Common program types include: XDP (L2 packet path), TC (qdisc ingress/egress), tracepoints/kprobes/uprobes (observability), cgroup/socket filters (network control), and LSM hooks (security).

Attach points determine when your code runs; choose them based on latency budget and required context.



eBPF architecture: loaders → verifier → JIT → hooks → maps → user space

Development workflow (CO-RE-friendly)

- Write eBPF C with BTF-aware structs (CO-RE) and use clang/LLVM to compile to an ELF object.
- Use libbpf or bpftool to load, verify, and attach.
- Define maps for state; pin programs/maps in bpffs if you need persistence.
- Use perf/ring buffers to stream events to user space; or arrays/hash maps for counters.
- Observe with tools like bpftool, bpftrace, or perf.

Code example: counting per-PID opens (tracepoint)

```
// file: tp_openat.c (CO-RE friendly)

#include <vmlinux.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_core_read.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u32);
    __type(value, u64);
    __uint(max_entries, 16384);
} counts SEC(".maps");

SEC("tracepoint/syscalls/sys_enter_openat")
int TP_openat(struct trace_event_raw_sys_enter*
ctx) {
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    u64 *val = bpf_map_lookup_elem(&counts, &pid);
    u64 one = 1;
    if (val) {
        __sync_fetch_and_add(val, 1);
    } else {
        bpf_map_update_elem(&counts, &pid, &one,
BPF_ANY);
    }
    return 0;
}
```

```
char LICENSE[] SEC("license") = "GPL";
```

Build and attach (example commands):

```
clang -O2 -g -target bpf -D__TARGET_ARCH_x86 -c
tp_openat.c -o tp_openat.o
sudo bpftool prog load tp_openat.o \
    /sys/fs/bpf/tp_openat type tracepoint \
    attach tp syscalls:sys_enter_openat
sudo bpftool map dump pinned /sys/fs/bpf/tp_openat/
maps/counts | head
```

Code example: minimal XDP pass/drop

```
// file: xdp_pass.c
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

SEC("xdp")
int xdp_pass(struct xdp_md *ctx) {
    return XDP_PASS; // allow packet
}

SEC("xdp")
int xdp_drop(struct xdp_md *ctx) {
    return XDP_DROP; // drop packet
}

char LICENSE[] SEC("license") = "GPL";
```

Build and attach (example commands):

```
clang -O2 -g -target bpf -c xdp_pass.c -o
xdp_pass.o
sudo ip link set dev eth0 xdp obj xdp_pass.o sec
xdp_pass

# Inspect
sudo bpftool prog show | grep xdp
# Detach
sudo ip link set dev eth0 xdp off
```

Observability with bpftrace (one-liner)

```
# Count sys_enter events per second
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_*
{ @cnt = count(); } interval:s:1 { printf("%d\n",
@cnt); clear(@cnt); }'
```

Maps, perf/ring buffers

Maps are key/value stores accessible from both kernel and user space. Choose types based on access patterns (e.g., LRU for cache-like data, percpu for low contention). Perf/ring buffers stream structured events to user space without polling.

Safety, verifier & BTF/CO-RE

The verifier enforces bounded loops, valid memory accesses, and proper helper use. BTF makes type info available in the kernel, enabling CO-RE (Compile Once – Run Everywhere) so your program adapts to kernel field offsets at load time.

Production considerations

- Pin your programs and maps in bpffs for lifecycle management.
- Scope privileges: loading typically requires CAP_SYS_ADMIN; attach points may need CAP_BPF/CAP_NET_ADMIN depending on distro.
- Monitor resource usage (map sizes, stack depth) and set rlimit appropriately.
- Establish upgrade strategy for BTF (via /sys/kernel/btf/vmlinux) and kernel feature gates.
- Prefer CO-RE + libbpf for portability; avoid raw kprobe names that may change across kernels.

Troubleshooting

If load fails inspect dmesg for verifier logs; check kernel config for BPF/JIT; ensure your toolchain matches target arch; validate BTF availability; try bpftool prog load with verbose logs.

Use Case in OES

In OES, we use it mostly for debugging, Filesystem debugging with eBPF is best framed as "answering where the IO time actually goes and tracking each call" by tracing from syscalls through VFS down to the block layer with minimal intrusion. The emphasis is that eBPF gives per-file, per-process latency and access patterns on live systems, so you can distinguish slow disks from slow filesystems or application behavior without custom kernels or restarts.

What eBPF Hooks In The FS Path

- Syscall tracepoints such as sys_enter_openat, sys_enter_read, and sys_enter_write let you correlate PIDs, file descriptors, and flags with every filesystem operation.
- Kprobes on vfs_read, vfs_write, or filesystem-specific functions like ext4_file_write_iter show how much latency is added in the VFS or filesystem layer itself, separate from raw device latency.

Typical Debugging Workflow

- Start at the block layer using tools like biosnoop and biolatency (BCC) to see whether the underlying device exhibits high tail latency or abnormal size distributions.
- If the device looks healthy, move up to file system probes (for example, tracing ext4_file_read_iter and building a histogram of operation latency) to see if journaling mode, writeback policy, or metadata patterns are causing stalls.

Concrete "One Liner" Style Example

- A bpftrace script can wrap a filesystem function with entry/return timestamps and produce a latency histogram, e.g. kprobe:ext4_file_read_iter { @start[tid] = nsecs; } kretprobe:ext4_file_read_iter /@start[tid]/ { @lat = hist(nsecs - @start[tid]); delete(@start[tid]); }, which immediately reveals whether reads are routinely hitting multi millisecond tails.
- Similar one-liners and BCC tools can count VFS calls (vfs_*), show which processes dominate IO, and characterize access sizes, turning vague "filesystem is slow" complaints into concrete per-path, per-topic, or per-directory evidence.

Value For Filesystem Debugging

- This approach allowed real-world systems (for example, Kafka clusters) to prove that slow produce requests correlated either with slow filesystem writes or lock contention on log files, which directly informed tuning and even filesystem changes
- For filesystem and storage engineers, eBPF effectively becomes a high-resolution stethoscope: it captures live workload behavior with precise context, enabling targeted changes to mount options, journaling modes, or even on-disk layout instead of blind tuning.

References

- eBPF - Wikipedia
- What Is eBPF? | IBM
- eBPF - Introduction, Tutorials & Community Resources
- https://www.brendangregg.com/ebpf.html
- https://blog.allegro.tech/2024/03/kafka-performance-analysis.html
- https://oneuptime.com/blog/post/2026-01-07-ebpf-disk-io-analysis/view
- https://events.linuxfoundation.org/wp-content/uploads/2023/08/Vandana-Salve-Mentorship-8-2-23-KernelTracing_eBPF.pdf
- https://oneuptime.com/blog/post/2026-01-07-ebpf-file-access-monitoring/view
- https://github.com/iovisor/bcc/issues/2842
- https://www.informit.com/articles/article.aspx?p=2995360&seqNum=3
- https://bpftrace.org/docs/release_024/language
- https://github.com/iovisor/bcc
- https://dev.to/yunwei37/ebpf-tutorial-by-example-9-capturing-scheduling-latency-and-recording-as-histogram-53mh

Magic Memory

Ganapathi CH, Principal Software Engineer - Open Enterprise Server

Breaking the Memory Wall: A Deep Dive into CXL "Magic Memory"

For decades, computer architecture has been trapped by a fundamental physical constraint: the Memory Wall. While CPU core counts have exploded and processing speeds have soared, the speed and capacity of the connection to physical RAM (DRAM) have lagged. This bottleneck leaves the world's most powerful processors sitting idle for millions of cycles, waiting for data.

Enter Compute Express Link (CXL), often colloquially called "Magic Memory." It isn't magic in the literal sense, but its ability to decouple memory from the CPU and turn it into a shared, liquid resource is nothing short of revolutionary.

1. The Architectural Crisis

To understand why CXL is "magic," we must first understand the problem it solves. Traditional servers are built on a "CPU-centric" model. RAM is physically tied to a specific processor via parallel DDR channels. This creates two massive inefficiencies:

1. Stranded Memory: If Server A has 512GB of RAM but its workload only uses 64GB, the remaining 448GB is "trapped." It cannot be used by Server B, even if Server B is crashing due to a memory shortage.
2. The Slot Limit: CPUs have a fixed number of pins and motherboard space. Once you fill the 8 or 12 DIMM slots, you cannot add more memory without buying an entirely new, expensive server.

CXL breaks these chains by allowing memory to live outside the traditional memory slots and instead communicate over the high-speed PCIe bus while maintaining the performance of a direct connection.

2. How CXL Works: The Three Pillars

CXL is an open standard that runs on the physical "tracks" of PCIe 5.0 and 6.0. However, while standard PCIe is used for "dumb" data transfers (like moving a file from a disk), CXL introduces Cache Coherency. This allows the CPU and an external device to share memory without the massive software overhead of constant data copying.

The CXL "magic" is delivered through three sub-protocols that work simultaneously:

- CXL.io: Think of this as the foundation. It handles device discovery, configuration, and error reporting, much like standard PCIe.
- CXL.cache: This allows an external device (like a GPU or an AI accelerator) to "rent" space in the CPU's cache. It ensures that the device and the CPU always see the most up-to-date version of any piece of data.
- CXL.mem: This is the heart of the "Magic Memory" concept. It allows the host CPU to access memory on an external device using simple "load/store" commands, treating it exactly like the RAM plugged directly into the motherboard.

3. The Evolution: From Expansion to Fabrics

The CXL standard has evolved rapidly, with each version adding a new layer of "magic" to the data center:

CXL 1.1: Expansion

The first generation focused on Memory Expansion. By plugging a CXL Type-3 device (a memory expander) into a PCIe slot, a server could double its RAM capacity. To the operating system, this looked like a new "NUMA node" - a block of memory that was slightly further away than local DRAM but still incredibly fast.

CXL 2.0: Pooling

CXL 2.0 introduced the CXL Switch. This allowed for Memory Pooling. Instead of memory being trapped in a server, it could be placed in a "memory appliance" - a box full of RAM modules connected to a switch. Multiple servers could connect to this switch and dynamically request 100GB here or 500GB there. When a server finished its task, it "returned" the memory to the pool.

CXL 3.1: The Global Fabric

As of 2026, CXL 3.1 has pushed technology into the realm of Fabric-Attached Memory (FAM). By using Port-Based Routing (PBR), CXL can now support thousands of nodes in a complex mesh or star topology. This enables Global Integrated

Memory (GIM), where memory is no longer a "peripheral" but a massive, shared ocean that any processor in the data center can swim in.

4. Why AI Makes CXL Non-Negotiable

The explosion of Large Language Models (LLMs) has turned CXL from a "nice-to-have" into a "must-have."

Training a model like GPT-5 or beyond requires tens of terabytes of high-speed memory. Currently, this is done by cramming HBM (High Bandwidth Memory) into GPUs. However, HBM is incredibly expensive and limited in capacity. CXL allows AI clusters to supplement expensive HBM with massive pools of cheaper DDR5 or even persistent memory.

In an AI context, CXL enables Zero-Copy Data Transfer. Instead of a CPU loading data from a disk, processing it, and then copying it over a slow bus to the GPU, both the CPU and GPU can simply point to the same address in a CXL memory pool. This reduces latency from milliseconds to nanoseconds.

5. Performance: Is it "Slow"?

The skeptics' main concern is latency. Memory on a PCIe bus is naturally "further away" than memory on a dedicated DDR channel.

However, CXL 3.1 (running on PCIe 6.0) uses PAM-4 encoding, doubling the bandwidth to 256 GB/s on a x16 link. While local DDR5 might have a latency of around 100ns, CXL memory typically clocks in at 170ns to 250ns.

For most applications this difference is negligible compared to the alternative: "swapping" data to an SSD, which takes 10,000ns to 100,000ns. CXL effectively fills the "Latency Gap" between local RAM and storage.

6. The Bottom Line: The "Composable" Data Center

CXL "Magic Memory" is the final piece of the puzzle for Composable Infrastructure. In the future, we won't buy "servers" with fixed amounts of CPU and RAM. Instead, we will build data centers out of independent "resource sleds":

- A rack of CPUs.
- A rack of GPUs.
- A massive rack of CXL Memory.

A software orchestrator will "compose" a virtual server on the fly, grabbing exactly the amount of "magic memory" needed for a specific job and releasing it the moment the job is done. This improves hardware utilization from the current average of 40% to nearly 90%, drastically reducing the environmental and financial cost of computing.

Mercurial Cores

Ganapathi CH, Principal Software Engineer - Open Enterprise Server

Introduction

While “Magic Memory” (CXL) is a solution to memory bottlenecks, Mercurial Cores are a significant and dangerous problem in modern processor design.

The term was coined by Google engineers (notably in the 2021 paper “Cores That Don’t Count”) to describe CPU cores that suffer from Silent Data Corruption (SDC). Unlike a standard hardware failure where a computer crashes or shows a “Blue Screen of Death,” a mercurial core continues to run but occasionally gives the wrong mathematical answer without the system ever knowing.

Google’s SiliFuzz revealed that roughly one in every few thousand CPU cores in a mature data center is mercurial.

1. Why are they called “Mercurial”?

In mythology, Mercury was volatile and unpredictable. Similarly, these cores are “mercurial” because:

- They are inconsistent: They might calculate 2 + 2 = 4 a million times correctly, but then randomly calculate 2 + 2 = 5 on the million-and-first time.
- They are context-sensitive: The error might only trigger at a specific temperature, at a specific voltage, or when executing a very specific, rare sequence of assembly instructions.
- They are silent: There is no error log, no “Machine Check Exception,” and no warning. The CPU simply writes the wrong data to your database or file system.

2. What Causes Mercurial Cores?

As we push toward 3nm and 2nm manufacturing processes, several physical factors make these errors more common:

- CMOS Scaling Limits: Transistors are now so small that they are approaching the width of just a few atoms. At this scale, even a tiny manufacturing imperfection can cause a gate to “leak” or fail under stress.
- Aging (Wear-out): A CPU might pass all factory tests perfectly but develop mercurial behavior after two years of 24/7 operation in a hot data center due to “electromigration” (the physical movement of atoms in the wires).
- Architectural Complexity: Modern cores have billions of transistors and complex features like branch predictors and out-of-order execution units. A bug in a tiny, rarely used sub-unit can stay hidden for years.

3. The “Blast Radius” of a Mercurial Core

Because the errors are silent, they can be catastrophic for large-scale systems:

- Encryption Failures: A mercurial core could miscalculate a single bit during an AES encryption task. The data becomes permanently unreadable, but the system thinks it was saved correctly.
- Database Corruption: A core might miscalculate a pointer while reorganizing a database, leading to “bit rot” that spreads across the entire dataset over months.
- The “Self-Inverting” Bug: Google found one core that miscalculated encryption - but because it used the same faulty logic to decrypt the data, it looked fine on that specific machine. When the data was sent to a different, healthy machine, it was pure gibberish.

4. How the Industry Fights Back

In a world of “Magic Memory” (CXL) and massive AI clusters, one bad core can ruin a calculation involving thousands of GPUs. To stop them, companies like Meta and Google use:

Summary: Traditional vs. Mercurial Failures

Feature	Traditional Failure	Mercurial Failure
Detection	Immediate (Crash/Error)	None (Silent)
Result	System stops	System continues with "bad" data
Cause	Power loss, severe hardware break	Subtle aging, CMOS scaling, "corner cases"
Solution	Replace hardware	Isolate the specific core via software

Summary of Detection Strategies

Strategy	Performance Cost	Detection Speed	Best For
Dual/Triple Modular Redundancy	100% (High)	Instant	Aerospace, Medical, Finance
Instruction Redundancy (ILR)	10-20% (Medium)	Fast	General Cloud Computing
Execution Fingerprinting	<1% (Very Low)	Delayed	Large-scale AI training

1. Scanners: Specialized software that constantly runs "hard" math problems in the background across all servers to see if any core returns a different result than its neighbors.
2. Redundancy (Voting): In critical tasks, the same calculation is performed by three different cores. If one core says "5" and the others say "4," the core that said "5" is marked as mercurial and disabled.
3. Boot-time Testing: Tools like BootRIST (Random Instruction Sequence Test) try to "stress test" the CPU during the boot process to find weak cores before they enter the production pool.

5. Finding Mercurial Cores

To catch mercurial cores without the massive overhead of running every task twice or thrice (Dual/Triple Modular Redundancy), engineers use a sophisticated technique called Instruction-Level Redundancy (ILR).

Since a mercurial core usually fails on specific types of operations (like certain floating-point calculations or bitwise shifts), ILR acts like a "spot check" system.

How Instruction-Level Redundancy (ILR) Works?

Instead of duplicating the entire program, the compiler or a background monitor identifies "critical paths" - the mathematical backbone of an operation - and creates a shadow instruction.

- Step 1: The CPU executes the original instruction (e.g., A x B = C).
- Step 2: The CPU immediately executes a "check" instruction using a different set of registers or a slightly different mathematical approach.
- Step 3: A comparison unit (often at the hardware level in modern chips) checks the two results.
- Step 4: If they disagree, the CPU triggers a "trap," stops the process, and flags that specific core for diagnostic testing.

The "Fingerprinting" Method

This technique is called Execution Fingerprinting. Instead of checking every single result, the CPU creates a "checksum" (a mathematical summary) of all the operations it performed over the last 1,000 cycles.

If two different cores run the same 1,000 instructions but end up with two different fingerprints, you know for a fact that one of them is mercurial. This is much faster than checking every single addition or subtraction because you only compare the "summary" once every few microseconds.

6. The Impact on "Magic Memory" (CXL)

This is where Mercurial Cores and CXL Memory collide in a dangerous way.

In a CXL environment, memory is shared across a fabric. If a mercurial core in Server A calculates a result incorrectly and writes that "poisoned" data into a shared CXL Memory Pool, that error can now spread to Server B, C, and D as they read from that same pool.

In the old days, a bad core only ruined its own local memory. In the age of "Magic Memory," a single mercurial core can act like a virus, corrupting an entire data center's shared data pool.

Realtime communication with WebRTC

Atish Mohanty, Lead Software Engineer - Open Enterprise Server

Introduction

Modern web browsers have evolved so far beyond the static web pages to ship powerful in-build APIs, which is helping in creating feature-rich web applications and solving real life problems. WebRTC is one of the most unique features introduced in modern day web which has transformed how live communication happens.

WebRTC (Web Realtime Communication) is a set of protocols and browser APIs which enables web applications to seamlessly connect and transmit data securely to another web application directly in real time.

Traditionally when two web applications need to communicate in real time, they need to connect to an intermediate server and rely on web socket or polling mechanism.

However, WebRTC takes a different approach by enabling a peer-to-peer connection between two devices, where the data is transmitted over a direct channel once the connection is established. WebRTC uses UDP protocol for optimal speed and reduced latency.

Because of the low latency it is leveraged for real time communication and is vastly used in applications for performing audio/video communication, screen sharing and real time chat applications.

Today most audio/ video sharing applications like (Zoom, Whatsapp, Google meet, Teams etc.) rely heavily on WebRTC for their application's core functionality.

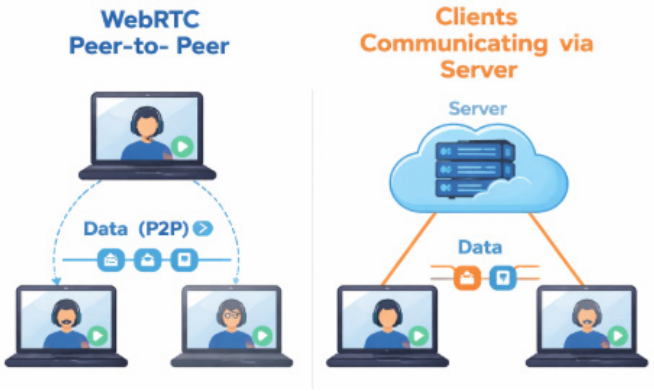
Key components of WebRTC

Signaling Server

Before the peers can communicate directly, both the peers need to make an initial connection to exchange communication control related information (SDP and ICE Candidates) required for setting up the peer connection.

Since the peers don't know how to connect to each other, they take the help of a mutually agreed intermediary server, called the signaling server.

WebRTC specs doesn't specify any specific protocol for exchanging information and leaves the decision to the developers to implement any technique which suits such as Web Sockets, HTTP, or even messaging queues.



Signaling server exchanges the metadata across the peers for establishing the peer connection. Once the connection is established, signaling server is not used for further data transfer.

SDP

In order to establish a peer connection, the sender and receiver both generate SDP (Session Description Protocol). SDP is not a protocol, but rather a data format, which includes the metadata. SDP usually contains information related to the IP address, data format, media capabilities, codecs, compression, encryptions etc. SDP helps both the peers to understand each other's capabilities and negotiate on the media capabilities and communication methods to establish a connection.

```
v=0
o=- 1234567890 1234567890 IN IP4 192.0.2.1
s=Example SDP Session
c=IN IP4 192.0.2.1
t=0 0
m=audio 49170 RTP/AVP 0
a=rtpmap:0 PCMU/8000
m=video 51372 RTP/AVP 31
a=rtpmap:31 H261/90000
```

(typical SDP representation)

The peer initiating a connection generates an SDP of type "OFFER" and sends through the signaling server. The receiving peer receives the SDP and creates an SDP of type "ANSWER" and sends it back to the sender. Both peers add the other's SDP to their remote configuration. Once the exchange of the SDP, and the ICE Candidates is complete, then the peer connection is established, and data communication can start.

ICE Candidates

In WebRTC the Ice Candidates are the potential network addresses (IP and port) that the peer can use to communicate. These ICE candidates are usually obtained by the ICE agents. ICE agents work based on the ICE (Interactive Connectivity Establishment) framework which makes sure to find the optimal address path for a low latency communication.

The Ice agents may use the STUN or TURN server to obtain the Ice Candidates of the peer. These Ice Candidates can be passed during the signaling with the SDP or separately. On the receiver end both peers ICE candidates are compared, and optimal ICE candidate is used for establishing the connection.

```
a=candidate:12345678 1 udp 2113937151 192.168.1.102 52996 typ host generation 0
```

(typical ICE Candidate representation)

STUN / TURN Servers

For establishing a peer connection the peers need to exchange with the public network. However, most of the devices are usually inside a private network and don't have their public IP address. These devices connect to a STUN (Session Traversal Utilities for NAT) server to obtain the public IP address.

STUN is a protocol to discover the public address and the accessibility of the client.

Sometimes the devices can be behind the firewall or other restricted networks, in this case the device can connect to a TURN (Traversal Using Relays around NAT) server which relay the packets from the sending peer to receiving peer.

Creating a WebRTC Connection

The following steps describe a typical Web RTC connection implementation between a Sending and Receiving Peer with vanila.js:

1. Each peer creates an instance of " RTCPeerConnection". During creation, STUN or TURN server details are provided so the browser can gather ICE candidates.

```
const iceServers = { iceServers: [{ urls:
"stun:stun.l.google.com:19302" }]};
const pc_sender = new
RTCPeerConnection(iceServers);
```

2. Then the peer creates an SDP of type "OFFER" and sets it to the local description.

```
const offer = await pc_sender.createOffer();
await pc_sender.setLocalDescription(offer);
```

3. This offer is passed to the receiving peer through the signaling server. Once the receiving peer receives the offer it follows the same process to create an SDP of type "ANSWER". The receiving peer then adds the offer received from the sender to its remote description.

```
const pc_receiver = new
RTCPeerConnection(iceServers);
const answer  = await pc_receiver.createAnswer ();
await pc_receiver.setLocalDescription(answer);
await pc_receiver.setRemoteDescription(offer);
```

4. Once the answer is created then the receiving peer sends the answer back to the sender through the signaling server. The answer from receiving peer is added to the remote description of the sending peer.

```
pc_sender.addRemoteDescription(answer)
```

5. Audio or video tracks can be added using the addTrack method. The receiving peer listens for incoming media using the ontrack event.

```
pc_sender.addTrack(track);
pc_sender.onTrack((event) => event);
```

Once both the peer have completed exchanging the SDP and ICE Candidates the peer-to-peer connection is established. The sender and receiver can now directly send data through the channel.

Conclusion

WebRTC makes real-time communication on the web faster, more efficient, and more accessible by enabling direct peer-to-peer connections between browsers. While the underlying technology involves multiple components such as signaling servers, SDP, ICE candidates, and STUN/TURN servers, browsers abstract much of this complexity for developers. WebRTC is not entirely serverless, but it significantly reduces server dependency once the connection is established. As real-time use cases continue to grow, WebRTC remains a foundational technology for modern web applications.

The Container is Dead: Why WASI is the "Universal USB" of Cloud Computing

Gaurav Kumar, Sr. Software Engineer - Open Enterprise Server

Introduction

The Linux kernel has always been the engine of our digital world, expertly handling resources and processes. Imagine if it could think for itself, though. What if it could figure out what was needed, make things better, and save energy, instead of just following rules? AI is about to change how Linux handles power and schedules tasks. Machine learning isn't just another program for Linux anymore; it's becoming a core part of it.

The Need for More Intelligent Kernels

Linux's power management and task scheduling tools have long relied on clever but fixed rules. These guidelines work well for routine tasks, but they fall short when things get hectic and unpredictable. Imagine a server that simultaneously runs multiple virtual machines, an AI language model, and a live data analysis. Every minute, even every second, the best way to distribute CPU power or manage energy for such a mix is different.

AI can help with this. Imagine a system that learns from what's happened before. It then figures out what'll be needed next and changes how it uses resources to either make things more productive or save energy, instead of sticking to strict rules.

Intelligent AI in Practice

Although the term "smart AI" sounds like something from a science fiction film, it actually refers to tiny, unique AI models, or agents, that operate directly within the operating system. These agents monitor temperature, power consumption, CPU usage, and input/output waiting times. They use this information to make wise choices.

Let's talk about CPU scheduling. A normal scheduler hands out tasks, giving important ones priority and then just moving down the line. But an AI-powered one? It could notice a container app always acts up when an AI language model is running too. So, it learns to move that app to quieter CPU cores or even give it more memory for a bit. This stops problems before they even begin. This is a game-changer for systems like ARM and RISC-V, where saving power and handling different cores is a big deal.



Real-World Implementations: It's Already Here

This tech isn't just a future dream; parts of it are already active in many devices and projects.

- Android's "Adaptive Battery": You can see this with Android's Adaptive Battery. Google put this in Android, which uses the Linux kernel. With help from DeepMind, it uses AI on your device to group apps into Active, Working Set, Frequent, and Rare. The kernel then handles these groups differently. It really limits background tasks and notifications for Rare apps, but lets Active ones run freely. This isn't a fixed rule; it learns from how you use your phone.
- The sched_ext Revolution: In late 2024, the Linux kernel added sched_ext, letting developers create their own schedulers using eBPF programs that run safely in the kernel. This has led to a lot of new ideas, like userspace schedulers. These programs

run in the background, look at how the system is behaving with complex rules or even AI, and then tell the kernel exactly how to schedule tasks right away. Gamers, especially on Linux handhelds like the Steam Deck, are testing these custom BPF schedulers. They focus on consistent frame times instead of just raw speed, making games play much smoother than the usual kernel could.

- Data Center "Autopilot": Data centers also use something similar called Feedback-Directed Optimization (FDO) within big cloud providers. While it's not always a live kernel agent, it gathers lots of performance data from servers, then trains models to find the best way to arrange software and schedule tasks. After that, they update the kernel to fit. It's a slower form of an Agentic AI process, saving a lot of power across countless servers.

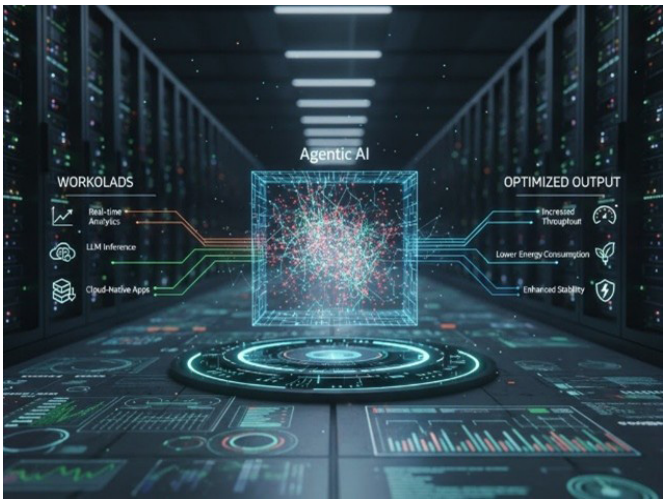
Smarter Power Use

AI can really improve how we handle power. Right now, settings like 'ondemand' or 'powersave' just follow rules. But an AI system could learn your habits: when you charge, when you do a lot of work, or when your device is just chilling. Then, it could change CPU speeds, shut down cores that aren't being used, and tell hardware to sleep deeper, all without waiting for a limit to be reached. This would make your phone battery last way longer and save a ton of energy in data centers.

The Tricky Bits: When Intelligence Isn't So Simple

Putting AI into the kernel isn't without its problems, or tricky bits:

- Extra Work: Small AI models still need computing power. Getting better scheduling or saving power only makes sense if the effort to run the AI is worth it. That's why super-efficient, tinyML methods are so important.
- Predictability: Kernel functions must be reliable. An AI model could cause unexpected delays or issues, disrupting



time-sensitive operations. We need to test things well and keep a close eye on them.

- Privacy & Fairness: Kernel data isn't private, and using it to train models can unintentionally lead to biases. We need to focus on fairness to prevent issues in specific tasks.

What's Next

AI in the kernel is still in its early stages. Initial projects are looking at things like improving how resources are used for AI tasks on Linux servers and making batteries last longer on small devices. As these approaches get better and hardware improves for running AI directly on devices, we anticipate a Linux kernel that becomes more intelligent and self-managing. Think of it as an operating system that learns from its daily work to improve order and efficiency.

This shows how adaptable Linux is, as it continues to integrate new technology, keeping it central to computing for years to come.

An Analytical Review of Ansible + Semaphore UI as a Lightweight Control Plane for Non-Functional Testing

Bharatesh, QA Engineer - File and Print Services, OES

Abstract

Non-functional testing at enterprise scale is often limited not by the system under test, but by the ability to orchestrate large client fleets while keeping automation lightweight and easy to adopt as test workflows evolve. As scope changes new workloads, new parameters, new environments, the real cost is not execution, but adapting and maintaining automation without breaking existing flows. To address this, we modernized our approach using Ansible as the parallel execution engine, Semaphore UI as a lightweight control plane for governed execution (templates, RBAC, scheduling, run history), and a small Python framework layer for reusable workflow utilities and consistent parameter handling. This article describes the Day-2 scale problem and the patterns that helped us keep automation adaptable with minimal change effort.

1. Orchestrating Chaos: The Enterprise "Day-2" Scale Problem

In non-functional testing, the challenge shifts quickly, the focus moves from "testing one server" to coordinating hundreds/ thousands of clients and maintaining an automation system that can evolve without breaking existing workflows. When scope grows, change cost grows -unless orchestration and governance are designed upfront.

As workflows expand (login/logout cycles, file I/O patterns, endurance runs, environment resets, log collection), teams commonly hit four failure modes:.

- Control-plane bottlenecks: orchestration overhead (sequencing, connections, retries) becomes the limiter rather than the system under test.
- Audit blindness: difficulty answering who ran what, against which build/environment, and with what parameters.
- Adoption barriers: complex workflows remain risky without guardrails; execution stays restricted to a few experts.
- High change cost: small workflow changes become hectic to roll out consistently across many clients and stages.

Table 1: Enterprise tuning guidelines (high-concurrency runs)

Parameter	Default	Enterprise guidance	Why it matters
Strategy	linear	free (for load runs)	Reduces convoy delays from slow hosts
Forks	5	50–200 per runner (capacity-based)	Increases parallel command injection
SSH pipelining	off	on (where supported)	Reduces per-task overhead
Fact gathering	implicit	explicit/disabled for load	Avoids startup storms

A practical solution requires separating concerns, Execution Engine (parallel action on endpoints) vs. Control Plane (governance, identity, audit, repeatability).

2. Treating Ansible as a Tunable Parallel Engine

Ansible provides an agentless execution layer suitable for distributed client orchestration. For non-functional testing, the objective is high parallelism with minimal orchestration skew, so that the orchestration layer itself does not distort test timing.

2.1 Avoiding Convoy Effects

The default linear strategy enforces lockstep progress, all hosts complete Task A before any host starts Task B. At scale, slow nodes can throttle the fleet ("convoy effect"), which distorts timing-sensitive workloads.

For load-oriented workflows, using strategy, free allows faster clients to proceed without waiting for stragglers -reducing orchestration-induced delays and improving run consistency.

2.2 Tuning Parallel Injection

Defaults like forks=5 can cap throughput. Enterprises should tune concurrency per runner based on capacity (CPU/memory/ file-descriptor limits), enable pipelining where supported, and control fact gathering to avoid "thundering herd" startup storms.

3. Semaphore UI: Lightweight Control Plane for Scale and Adoption

Ansible is a strong execution layer, but CLI-only operation becomes fragile as workflow scope grows. Semaphore UI adds a control plane that converts automation from "run instructions" into governed workflows -standard entry points, controlled parameters, and consistent run history.

In a test-automation context, Semaphore UI helps by providing:.

- Templates as standardized entry points for workflows

Table 2: Database backend selection strategy

Backend	Recommended usage	Constraint at scale
SQLite	PoCs, small labs	Limited concurrency for heavy writes
MySQL/MariaDB	Team production	Requires DB ops maturity
PostgreSQL	Higher concurrency + audit retention	Needs tuning for peak ingestion

- Scheduling for routine validation runs
- Run history and logs for traceability and comparisons
- RBAC-driven delegation so execution can scale across teams safely
- Credential handling that avoids ad-hoc key distribution

3.1 Persistence at Scale: The Shift to SQL

A control plane is only as scalable as its persistence layer. Task logs, schedules, and run history can create heavy write pressure. Embedded stores may be fine for PoCs, but production-grade concurrency typically benefits from a SQL backend for stability and retention.

3.2 Distributed Execution: Sharding with Runners

At enterprise concurrency levels, a single executor becomes a vertical scaling trap due to connection overhead and OS limits. Semaphore supports a runner model (similar to CI runners), where execution happens on separate machines while the UI remains centralized.

A practical pattern is runner sharding,

- Split fleets logically (by VLAN/rack/region/environment tier)
- Place runners close to target networks
- Execute the same template across shards in parallel

This improves throughput and reduces blast radius while preserving a single "pane of glass" for operations.

4. Governance and Security: Operating Safely

A control plane holds sensitive access (SSH keys, repo credentials, secrets), so identity and secrets must be first-class design items.

- Credential centralization: reduces key sprawl across laptops and ad-hoc servers.
- RBAC: enables operators to run approved templates without editing code or accessing secrets.
- SSO/OIDC integration: supports enterprise access patterns and reduces local account drift.
- Auditability: every run is recorded with parameters, timestamps, and logs -critical for repeatable non-functional results.

Table 3: Typical role split (example RBAC model)

Role	Scope	What they can do
Owner/Admin	Org	System configuration, users, global settings
Manager	Project	Maintain templates/repos/inventories, manage keys
Task Runner	Project	Execute approved templates, view logs/history
Guest	Project	Read-only visibility for audits/reviews

5. Operational Best Practices: Automation as a Product

Semaphore UI + Ansible works best when automation is treated as a product, not a one-time solution.

- Git as the source of truth: workflows are versioned, reviewed, and reproducible.
- Templates as contracts: standardized entry points (login test, file I/O, collect logs, reset environment).
- Run metadata discipline: includes build ID, environment tag, workload profile, duration, and retention policy.
- Keep it lightweight and adaptable: can use a small Python/ any framework layer for reusable workflow utilities (standard inputs, validation, result formatting), while Ansible handles fleet execution and Semaphore UI enforces controlled entry points. This keeps scope changes low-cost, most updates become parameter changes or template adjustments, not restructuring the automation.
- Coordinate "storm" events: where strict timing matters, distribute scheduled triggers to clients rather than attempting millisecond-perfect remote starts from a single controller.

Conclusion

Ansible and Semaphore UI helped us build a lightweight, highly adoptable control-plane approach for non-functional testing where execution is standardized, auditable, and safe to delegate, and evolving test scope does not translate into constant rework. The Python framework layer kept workflow utilities reusable and consistent, so most scope changes could be introduced through small, controlled updates rather than rewriting automation.

For other teams, the same model accelerates adoption while reducing risk, RBAC, scheduling, distributed runners, and end-to-end traceability become built-in capabilities, enabling automation to scale with changing workflows instead of resisting them. Building this stack also became the fastest way to deeply understand Ansible's scaling behavior and Semaphore's control-plane design choices in real operations -learnings that now guide each expansion of the test workflow.

Migrating Existing Xamarin. Forms App to .NET MAUI

Abani Ranjan Samal, Lead Software Engineer - Content Manager and
Dharun Kumar, Sr. Software Engineer - Content Manager

Abstract

Microsoft's transition from Xamarin.Forms to .NET Multi-platform App UI (.NET MAUI) represents a significant architectural evolution in cross-platform application development. Rather than being a simple framework replacement, this shift introduces fundamental changes in project structure, UI rendering, application lifecycle, and dependency management.

This article presents a technical analysis of migrating an enterprise Xamarin.Forms application to .NET MAUI, focusing on architectural transformations, framework internals, and engineering trade-off s. The discussion highlights why migration is strategically valuable, how MAUI's architecture di ffers at a system level, and what measurable technical benefits are achieved in terms of performance, maintainability, and extensibility.

Background and Motivation

Xamarin.Forms enabled rapid cross-platform mobile development by abstracting native UI frameworks behind a shared API. However, its architecture introduced long-term challenges:

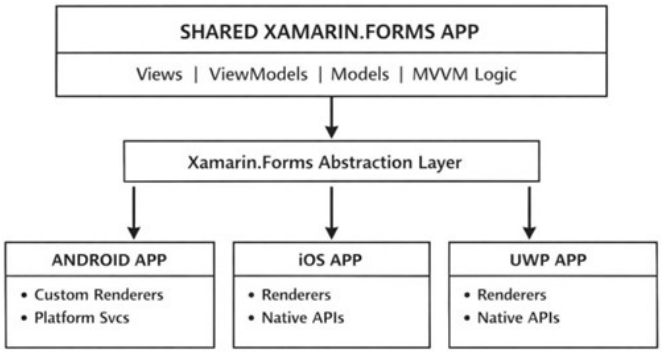
- Multi-project solutions increased configuration complexity
- Custom Renderers created deep inheritance hierarchies
- Platform startup logic was fragmented
- Adopting modern .NET runtime features was increasingly constrained

With Xamarin.Forms reaching end-of-life, .NET MAUI emerges not merely as its successor, but as a re-architected UI framework aligned with modern .NET hosting, tooling, and runtime models.

Legacy Architecture: Xamarin.Forms

Xamarin.Forms applications typically follow a multi-project, renderer-centric architecture, where each platform maintains its own project and rendering pipeline.

Diagram 1: Xamarin.Forms Architecture (Before Migration)



Architectural Characteristics

- Separate platform projects with duplicated resources
- Custom Renderers tightly coupled to native UI controls
- Complex renderer inheritance chains
- Platform initialization scattered across entry points

Technical Limitations

- Higher memory overhead due to layered abstraction
- Diff icult UI debugging across renderer boundaries
- Increased maintenance cost for platform-specific changes

Architectural Shift in .NET MAUI

.NET MAUI introduces two fundamental architectural changes:

1. Single-Project Model
2. Handler-Based Rendering Architecture

These changes directly address the structural and performance limitations of Xamarin.Forms.

Single-Project Model

.NET MAUI consolidates all platform targets into a single project file. Platform differentiation is resolved at build time using MSBuild conditions. Technical Benefits:

- Unified dependency graph and NuGet resolution
- Centralized resource management (fonts, images, styles)
- Reduced MSBuild complexity
- Consistent configuration across platforms

Platform-specific code is explicitly isolated using a dedicated folder structure, improving clarity and compile-time safety.

Handler-Based Rendering Architecture

The replacement of Custom Renderers with Handlers represents the most impactful internal change.

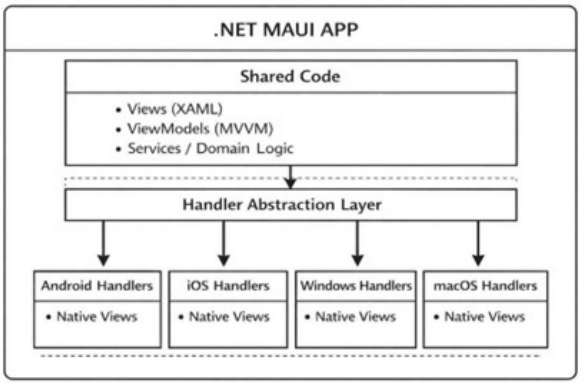
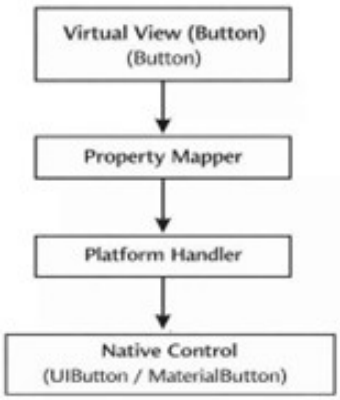


Diagram 2: .NET MAUI Architecture (After Migration)

Why Handlers Matter Technically

Handlers use composition instead of inheritance, providing a thin mapping between a virtual view and native control.



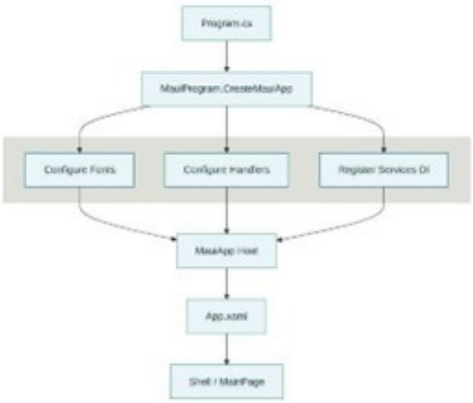
Engineering Advantages

- Smaller object graphs
- Reduced rendering overhead
- Easier platform customization
- Improved UI lifecycle tracing and debugging

Application Lifecycle and Hosting Model

.NET MAUI adopts a generic host model, similar to ASP.NET Core, centralizing application startup in MauiProgram.cs.

Diagram 3: MAUI Startup and Hosting Flow



Technical Implications

- First-class Dependency Injection
- Deterministic service lifetimes
- Unified configuration and logging pipeline
- Consistent startup behavior across platforms

This alignment significantly reduces the conceptual gap between mobile and server-side .NET applications.

Code Reuse and Refactoring Scope: High Reuse Areas

- Domain and business logic
- ViewModels (MVVM)
- Service abstractions
- Networking and persistence layers Refactoring Hotspots
- UI layout adjustments due to updated measurement logic
- Migration from Renderers to Handlers
- Replacement of deprecated Xamarin APIs

The migration confirmed that core business logic is highly portable, while UI and platform integration benefit from targeted refactoring rather than direct copying.

UI Layout and Rendering Behavior

.NET MAUI introduces updated layout primitives and a refined measurement pipeline:

- Reduced layout pass overhead
- More predictable sizing behavior
- Improved support for modern screen densities

Although minor layout tuning was required, the resulting UI demonstrated improved consistency across devices.

Platform-Specific Code Isolation

Platform-specific implementations are clearly scoped:

```
/Platforms
   /Android
      MainActivity.cs
      AndroidServices.cs
   /iOS
      AppDelegate.cs       iOSServices.cs    /
Windows
      App.xaml.cs
```

This structure improves:

- Compile-time validation
- Platform ownership clarity
- Debugging and testing e iciency

Testing and CI/CD Alignment

.NET MAUI integrates seamlessly with modern .NET tooling:

- Standardized dotnet CLI workflows
- Simplified CI/CD pipelines
- Easier SDK upgrades and environment consistency

From a DevOps perspective, the migration reduced build complexity and increased pipeline reliability.

Performance and Maintainability Outcomes

Post-migration analysis showed:

- Reduced UI rendering overhead
- Cleaner dependency graphs

- Improved startup time
- Lower long-term maintenance effort

The handler architecture and unified project model were the primary contributors to these improvements.

Challenges and Engineering Considerations

- Third-party library compatibility required early validation
- Developers needed to adapt to MAUI lifecycle concepts
- UI regression testing was essential due to rendering diff eerences

These challenges reinforce the importance of treating migration as a technical modernization effort, not a mechanical port.

Conclusion

The migration from Xamarin.Forms to .NET MAUI represents a deliberate architectural modernization rather than a framework swap. By adopting a single-project structure, handler-based rendering pipeline, and host-driven startup model, .NET MAUI significantly reduces architectural complexity while improving performance and maintainability.

For engineering teams maintaining long-lived cross-platform applications, .NET MAUI provides a scalable, future-ready foundation aligned with modern .NET development practices.

Reimagining Patch Management Through Dependency-Aware Risk Visibility

Kratagya Jadoun Singh, Software Engineer - Endpoint Management

Abstract

Enterprise patch management has achieved scale and reliability. Updates are deployed globally, compliance is tracked precisely, and critical vulnerabilities are addressed with increasing speed. Yet a persistent problem remains unresolved: systems often become unstable after successful patch deployment. Applications fail in subtle ways, workflows degrade, and IT teams are forced into reactive remediation—despite having followed best practices.

This article argues that the issue is not patch quality, but lack of behavioral awareness. Patch criticality explains why an update matters for security; it does not explain how it may affect system behavior. This article presents an architectural enhancement to existing patch-management systems that predicts which applications may be affected by a patch and why, using explainable, practical, and incrementally adoptable system-level reasoning.

1. When "Successful" Patching Still Breaks Systems

Consider a familiar set of post-patch incidents:

- A media team reports that live-streaming software no longer captures audio after a routine update.
- VPN connectivity becomes unreliable, impacting remote access across regions.

In each case:

- The patch installed successfully
- The system rebooted cleanly
- Compliance dashboards show green

And yet, from the user's perspective, the system is broken.

These failures are rarely catastrophic. They are subtle, capability-level regressions that surface only when applications interact with the operating system in specific ways. This is precisely why it is hard to predict—and why traditional patch management struggles to address them.

2. The Limits of Patch Criticality

Patch metadata is excellent at communicating security urgency such as:

- Severity
- Exploitability
- Superdense
- Reboot requirements

What it does not communicate is behavioral impact.

A patch marked "Important" or even "Low" can still disrupt mission-critical workflows if it modifies shared system components relied upon by user applications. Security criticality and operational stability are related, but fundamentally different dimensions. To bridge this gap, we must shift our perspective.

3. Core Insight: How Patches Really Affect Applications

At the heart of this proposal is a simple but powerful observation:

Applications are not affected by patches directly. They are affected through the system services and components that patches modify.

This relationship can be expressed succinctly:

Patch
-> Affected OS components
-> Windows services
-> Applications using those services

This chain is the missing abstraction in most patch-management systems.

Patches change operating system behavior.

Operating system behavior is mediated through services.

Applications consume those services to deliver user-facing functionality.

Understanding patch impact therefore requires reasoning about this dependency chain, not inspecting patch binaries or application code.

4. Introducing Application Impact Awareness

The proposal includes Application Impact Awareness as an architectural enhancement layered into existing patch-management workflows.

Objective

Provide a pre-deployment view of:

- Which installed applications may be affected

- What system capabilities are involved
- Why impact is plausible

Non-Objectives

- Predicting exact failures
- Blocking patch deployment
- Replacing security severity or compliance mechanisms

The system is designed to be advisory, explainable, and non-invasive.

5. System Overview

Application Impact Awareness operates as a lightweight analytical stage executed before patch deployment. It correlates three sources of information already available on enterprise devices:

1. Patch Metadata

   Identifies operating system subsystems modified by the update.
2. System State

   Enumerates relevant Windows services, including runtime status and functional role.
3. Application Inventory

   Identifies installed applications and their interaction with system capabilities.

Rather than constructing a static dependency graph, the system builds a transient, purpose-specific model to reason about impact for a given patch at any given moment in time.

6. Impact Analysis Pipeline

The system follows a clear, deterministic pipeline:

1. Patch Subsystem Classification

   The patch is categorized by the OS subsystems it modifies (for example, networking, audio, printing, identity).
2. Service Correlation

   Services backed by those subsystems are identified and evaluated based on:
   - Runtime state
   - User-facing relevance
   - Scope of responsibility
3. Application Association

   Installed applications are associated with services they plausibly rely on, either directly or indirectly.
4. Impact Explanation Generation

   For each application, the system produces a human-readable explanation describing potential impact.

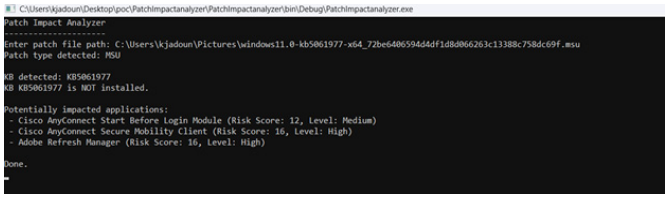
This pipeline transforms low-level system changes into user-relevant insight.

7. Heuristics, Not Hard Dependencies

The system intentionally avoids claiming absolute dependencies. Instead, it applies heuristics designed to maximize trust and usefulness:

- Running services are more relevant than dormant ones
- User-facing services carry higher impact on weight
- Widely shared services are flagged with broader scope
- Capabilities are described functionally (for example., “audio capture”, “printing”)

This approach favors explainability over false precision.



8. User-Facing Output: Making Impact Understandable

Rather than numeric risk scores, the system surfaces qualitative impact with clear reasoning.

Example: OBS Studio

Impact Level: High

Why: Relies on audio and media services modified by this patch. These services are currently active.

Potential Effect: Audio capture or live streaming may fail after reboot.

This format ensures administrators understand what may happen, why, and where to focus validation.

9. Design Constraints and Trade-offs

The solution is guided by explicit constraints:

- No application instrumentation
- No static dependency manifests
- No deployment blocking
- No reliance on machine learning in early iterations

These constraints ensure incremental adoption and operational safety. The trade-off—probabilistic rather than deterministic predictions—is explicit and intentional.

10. Operational Impact and Future Direction

Application Impact Awareness enables:

- Smarter patch scheduling
- Reduced post-patch incidents
- Faster root-cause analysis
- Greater confidence in automation

Over time, the same architecture can evolve to incorporate telemetry-driven learning and fleet-wide insights—without redesigning the core system.

Conclusion

Enterprise systems do not become unstable because patches exist.

They become unstable because patches are applied without sufficient behavioral context.

Application Impact Awareness closes that gap by making the relationship between patches, services, and applications explicit.

It does not promise certainty, but promises foresight

Final Thought

Security keeps systems safe.

Stability keeps businesses running.

The future of patch management must do both.

Surviving the Grey Zone

Deeksha R, Software Engineer - Endpoint Management

Introduction

The dashboards look calm. Latency charts display a serene emerald green, and the Service Mesh proudly reports 99.9% uptime. Ingress traffic appears normal—almost boring. Yet, on the other side of the glass, thousands of users are stuck. Screens freeze. Payments fail. Orders never complete.

This is Architectural Dissonance—the widening gap between what our infrastructure believes is true and what users actually experience. We built API Gateways and Service Meshes to simplify distributed systems, but in doing so, we created a Grey Zone: a technical blind spot where catastrophic failures hide, invisible to dashboards and immune to standard alerts.

1. The Anatomy of the Grey Zone

We like to think of the API Gateway as neutral—a simple router, a 'dumb pipe,' or a helpful traffic cop directing cars through an intersection. This is a comforting illusion. In reality, the gateway is the most semantically opinionated component in your stack. It functions as a Diplomatic Dictator: deciding who enters, who is delayed, and who is silently dropped under pressure.

At scale, the gateway doesn't just route; it interprets:

- Regional Bias: Decides which regions deserve aggressive throttling based on invisible configuration weights.
- Header Purging: Determines which headers are safe to keep and which are redundant enough to strip.
- Payload Manipulation: Transforms—or rejects—payloads long before they reach your business logic.

The Grey Zone isn't a fluke; it's a byproduct of three specific technical failures in the modern playbook:

The L7 Semantic Gap

Your Service Mesh monitors the health of the hive—meticulously checking if Pod A can talk to Pod B. It sees a successful 200 OK and reports that the system is pristine. But the Mesh is blind to the Gateway's executive decisions. It doesn't see that the Gateway silently stripped a critical authentication header or truncated a JSON payload because of a legacy transformation rule—such as 'normalizing' a null array.

The Last-Mile Mirage

Standard telemetry suffers from Boundary Blindness. Dashboards measure Round Trip Time (RTT) from the Gateway to the Service, but remain oblivious to Last-Mile Volatility—regional ISP throttling or half-open TCP connections that prevent the user from ever reaching the 'Front Door'.

The Distributed Guessing Game

Because the Gateway, the Mesh, and the Application logic all have independent, uncoordinated timeout budgets, requests often die in No Man's Land. A request might be killed by the Mesh at 5 seconds, while the Gateway holds the connection open for 30 seconds, reporting a healthy but idle connection—even though the user's mobile client has already timed out and triggered a Retry Storm.

The Silent Killers: Compounding Latency

In distributed systems, latency doesn't just add up—it multiplies. Small mismatches between the Gateway and the Mesh create micro-frictions that compound until the system collapses:

- Double Encryption Overhead: Traffic is decrypted via TLS, re-encrypted for internal transit, and terminated again at the sidecar.
- Header Rogue-ness: A gateway strips a trace-id or identity header that the mesh relies on for routing, causing the mesh to default to a slower, catch-all path.
- Compression Loops: A service Gzips a payload, only for a proxy to unzip it for inspection and re-zip it with a different algorithm, burning CPU cycles at every hop.
- Zombie Retries: The mesh retries three times. The gateway, unaware, also retries three times. One user click becomes nine backend calls.

2. The Fog of War: Mesh Optimism vs. Gateway Diplomacy

The industry loves to frame the conversation as API Gateway vs. Service Mesh. In real systems, this debate is meaningless. If you're running production at scale, you already have both. The real danger isn't duplication—it's disagreement.

The Service Mesh is optimistic. It sees the world from inside the cluster: pods, sidecars, and heartbeats. Its logic is simple: if the hive is alive, the system must be healthy.

The Gateway is diplomatic. It faces the public internet—the realm of rage refreshes, broken mobile clients, regional outages, and retry storms that no load test ever modeled.

When Layers Disagree

When these two layers don't share a single understanding of a request's lifecycle, debugging turns into a guessing game. A request timed out—but where? Did the gateway enforce a hard 30-second timeout? Did the mesh apply a 5-second per-hop budget? Did the application hang at 12 seconds waiting on a downstream dependency?

3. The Three Archetypes of Survival

Most best practices assume ideal networks and uniform regions. Reality is messier. Over time, teams gravitate—consciously or not—toward one of three survival patterns to manage the Grey Zone:

I. The High-Velocity Model (Thin Gateway, Smart Mesh)

Used by global giants handling massive concurrency. The gateway is intentionally dumb—and ruthless. It handles only Authentication and basic rate limiting. All intelligence lives in the mesh: retries, circuit breaking, discovery, and failover.

- Benefit: Minimal "blast radius" at the edge; the gateway is less likely to fail under load.
- Risk: The internal mesh becomes so complex that it is nearly impossible to fully simulate outside of the production environment.

II. The Compliance Model (Fat Gateway, Dumb Mesh)

Common in Fintech and Healthcare where security is the primary directive. Every policy is enforced upfront—auditing, transformations, and rigorous compliance checks. The gateway becomes the single, auditable choke point.

- Benefit: Clear ownership and incredibly strong security guarantees.
- Risk: Significant latency inflation. The gateway quietly becomes a monolithic bottleneck that is difficult to scale horizontally.

III. The Geo-Fragmented Model (Regional Reality)

The unspoken truth of global systems: Traffic in Bengaluru behaves nothing like traffic in London. In this model, engineers accept that timeouts, retries, and even error semantics must change by region. They deploy different rules for every major geographical hub.

- Benefit: Service Level Objectives (SLOs) are grounded in reality rather than an idealized average.
- Risk: Configuration drift that no dashboard can fully explain, leading to "works in US-East but fails in EU-West" syndromes.

4. Moving Toward Intent-Aware Infrastructure

The problem isn't that gateways are bad; they're blind. The future of backend architecture isn't about smarter routing rules—it's about Intent Awareness. We need infrastructure that understands why a request exists, where it comes from, and whether it's worth the cost of serving it.

Programmable Edges: The Death of the Round-Trip

Traditionally, the "Edge" was little more than a warehouse for static asset. But modern Programmable Edges have evolved into execution environments capable of running real logic using WebAssembly (Wasm) and lightweight Edge Functions.This shift matters because of one fundamental architectural truth:

Every time a request enters your VPC (Virtual Private Cloud), you pay a tax.

Your VPC—your private, isolated network where core microservices and databases live—is designed like a fortified castle. Getting inside involves crossing the public internet, hitting firewalls, traversing load balancers, and consuming expensive, high-latency ingress paths. Historically, even trivial checks ("Is this JWT valid?" "Which bucket is this user in?") had to make this entire journey just to be rejected 50 milliseconds later.

Using WebAssembly (Wasm) and Edge Functions, we run real business logic (JWT validation, A/B testing *, feature flags, abuse checks, regional personalization) directly on edge nodes—milliseconds from the user. By the time a request reaches your VPC, it has already been filtered, shaped, validated, and contextualized.

- The Result: You eliminate the most expensive and unnecessary roundtrips before they ever burden your core network. Your VPC becomes a destination only for requests that deserve to be there—reducing latency, protecting internal services, and cutting ingress costs.

Predictive Shaping: Proactive Resilience

Most systems scale reactively (CPU hits 80% , spin up pods). Predictive Shaping analyzes historical patterns to shape the infrastructure before the surge arrives. If Mumbai spikes every Friday at 6:00 PM, the system pre-warms the mesh and clears a "VIP lane" for traffic in advance.

- The Result: You stop firefighting and start preventing fires.

Cost-Aware Routing: The Financial Control Plane

Today, Service Meshes route based on health. Cost-aware routing adds a second variable: price. In a multi-cloud world, moving bytes across regions is expensive. This model routes high-priority users through fast paths and background jobs through cheaper, "spot-instance" regions.

- The Result: Engineering decisions finally align with the company's P&L, optimizing for margins as much as milliseconds.

The Bottom Line

Your architecture diagram is a map, not the territory. Somewhere between the mesh's optimism and the gateway's diplomacy lies the real truth of your system. The future of the backend isn't just about scaling faster—it's about becoming contextually aware.

If you want to see how these ideas show up in real systems, here are concrete, real-world implementations of programmable edges and edge-side logic:

- Edge Logic With Cloudflare Workers & BunnyCDN – practical examples of filtering, routing, and decision-making at the edge: https://www.dchost.com/blog/en/edge-logic-with-cloudflare-workers-and-bunnycdn-edge-rules
- JWT Validation at the Edge (Cloudflare + Vercel) – real implementation of auth logic pushed to the edge: https://ssojet.com/blog/how-to-validate-jwts-efficiently-at-the-edge-with-cloudflare-workers-and-vercel

These examples show that the shift toward intent-aware, edge-driven infrastructure isn't theoretical—it's already happening at global scale.

Glossary

*Edge-Side A/B Testing: Traditionally, A/B testing (the process of showing Version A of a feature to one group and Version B to another) required the request to hit a central server to decide the user's "bucket." This caused a "flicker" in UI or added 100ms of delay. By moving A/B logic to the Edge, the decision happens instantly based on the user's metadata. The Edge intercepts the request, assigns the bucket, and routes it to the correct path without the backend ever having to lift a finger.

The Architecture of Entropy A Story of systems, failure and survival

Sri Darshan Sreedharan, Sr. Software Engineer - Endpoint Management

The Silence of the Servers

Imagine it is Friday, July 19, 2024. The world wakes up to a digital silence that is deafening. In airports from Sydney to London, departure screens turn blank. In hospitals, surgery schedules vanish. Banks, broadcasters, and emergency services stare at the same ominous glow: the Blue Screen of Death. A single faulty content update from CrowdStrike has paralyzed millions of Windows hosts globally.

This wasn't a malicious cyberattack by a foreign state. It was something far more mundane and terrifying: a failure of resilience. It was a reminder that in our hyper-connected, distributed world, perfection is a myth. The only truth is entropy.

This is the story of Chaos Engineering, the counterintuitive discipline that argues the only way to prevent such catastrophes is to cause them—intentionally, carefully, and frequently.

Chapter 1: The Vaccinology of the Cloud

To understand why engineers would deliberately break their own systems, you need to look back to the early days of the cloud. In the monolithic era, reliability meant building a fortress—redundant hardware, thick firewalls, and hope. But as systems migrated to the cloud, they became biological. They spanned thousands of ephemeral nodes, scaling up and down like a breathing organism.

In this new world, "uptime" became a fallacy. The new goal was resilience.

Think of a distributed system like a human body. You cannot protect a body by keeping it in a sterile bubble forever. Eventually, a virus will find a way in. Instead, we use vaccines—small, controlled doses of a pathogen injected to train the immune system. Chaos Engineering is the vaccinology of the internet. It injects controlled harm—latency, packet loss, server crashes—to train the system's "immune response" (its automated recovery scripts) and its "doctors" (the engineering teams).

The philosophy is simple but radical: You don't choose if you will fail, only when. You can fail at 3:00 AM on a holiday when everyone is asleep, or you can fail at 2:00 PM on a Tuesday when your entire team is caffeinated and watching the dashboards.

Chapter 2: Rules of Engagement

Chaos engineering isn't about reckless destruction. You don't just walk into a data center and start unplugging cables. The "Principles of Chaos" act as the rules of engagement for this warfare.

The Steady State Hypothesis

The story always begins with a hypothesis. Before an engineer breaks anything, they must define what "normal" looks like. Not CPU usage or memory—those are internal vitals—but the pulse of the business. For Netflix, it's "Streams Per Second." For an e-commerce giant, it's "Checkouts Per Minute." The Chaos Engineer looks at this pulse and says, "I bet that if I sever the connection to the recommendation engine, this pulse will not waver."

The Blast Radius

Then comes the explosion, but it is a controlled demolition. This is the principle of the Blast Radius. You start small—infecting perhaps a single container or 1% of user traffic. You observe. If the system handles it, you expand the radius. If the system buckles, you hit the Big Red Button—the abort switch that reverts the chaos immediately. The CrowdStrike disaster of 2024 was essentially a chaos experiment with no blast radius control; the update went to everyone, everywhere, all at once.

Chapter 3: The Arsenal of Entropy

Over the last decade, a sophisticated arsenal has been developed to wage this war against fragility.

In the trenches of Kubernetes clusters, engineers wield Chaos Mesh and LitmusChaos. These open-source tools act like saboteurs behind enemy lines. They can corrupt network packets, skew system clocks to confuse databases, or starve a specific container of CPU cycles to see if its neighbors survive.

For the enterprise commanders who need a "simpler" war, there is Gremlin, a platform that offers "Failure-as-a-Service," allowing teams to launch attacks with a clean GUI and safety checks. And for those deep in the Amazon ecosystem, AWS Fault Injection Simulator (FIS) lets teams reach into the very fabric of the cloud and terminate instances or throttle APIs with terrifying realism.

Chapter 4: War Stories from the Frontlines

The history of Chaos Engineering is written in the scars of those who practiced it—and the wounds of those who didn't.

The Success: Netflix and the House of Cards

The legend begins with Netflix. They built the "Simian Army," a suite of bots including Chaos Monkey (which kills servers) and Chaos Kong (which kills entire AWS regions). Their ultimate test came during the premiere of House of Cards. Anticipating a massive traffic surge, engineers preemptively triggered a regional failover. Because their "army" had drilled this scenario hundreds of times, the traffic shifted continents seamlessly. No one noticed. The system didn't just survive; it shrugged.

The Lesson: The LinkedIn Retry Storm

LinkedIn learned a harder lesson. During a controlled experiment, the team injected a small packet loss, they watched a minor failure spiral into a catastrophe. A service failed, retried immediately, failed again, and retried again. This created a "retry storm"—a self-inflicted DDoS attack that amplified traffic by 400%. The chaos experiment revealed the need for "circuit breakers" and "exponential backoff"—forcing the system to wait before retrying. Without that drill, a real network blip would have taken down the entire professional network.

The Paradox: Shopify's Tale of Two Sales

Shopify offers the most poignant narrative of all. Years ago, they used chaos to find a timeout bug that would have cost them millions on Black Friday. They fixed it, saving an estimated $5 million. They were heroes.

But resilience is a moving target. Fast forward to Cyber Monday, 2025. While the checkout systems (hardened by years of testing) held up, the merchant admin panels collapsed. A massive authentication failure locked thousands of shop owners out of their own stores for six hours. They couldn't fulfill orders or update inventory. The lesson was brutal: You are only as strong as your weakest, untested link. Shopify had inoculated the buyer's experience but left the merchant's door vulnerable.

Chapter 5: The Ghost in the Machine (AI & Agents)

As we move through 2026, a new character enters our story: The Artificial Intelligence Agent.

The old war was deterministic. If you cut a cable, the server went down. The new war is probabilistic. AI agents don't just crash; they hallucinate. They get confused. They drift.

Imagine an AI travel agent. You ask it to book a flight. If the airline API is slow, does the agent wait? Does it apologize? Or does it hallucinate a confirmation number because it feels pressure to complete the task?

This has birthed a new wave of chaos tools. Promptfoo and Agent-Chaos are the new Simian Army. They don't just inject latency; they inject semantic noise. They flood the AI with garbled text, adversarial prompts, and "jailbreak" attempts to see if the agent goes rogue. They simulate "token storms" where context windows overflow, causing the AI to forget its instructions. In this new era, we aren't just testing for uptime; we are testing for sanity.

The Future is Broken

The narrative of software engineering has shifted. We no longer strive for the "invincible" system, because we know it doesn't exist. We strive for the "antifragile" system—the one that improves when challenged.

From the ashes of the CrowdStrike outage and the Shopify blackout, a clear moral emerges: Silence is dangerous. A system that has never been broken is a ticking time bomb. The only way to trust the code that runs our hospitals, our banks, and our lives is to break it ourselves, watch it heal, and then break it again.

Welcome to the age of Chaos.